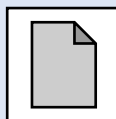
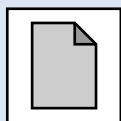


第四章 USART与串口通信

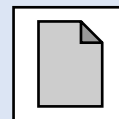
第一节、串口通信



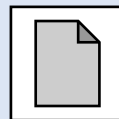
第二节、USART



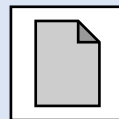
第三节、USART的标准库编程



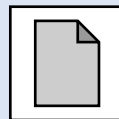
第四节、中断方式的数据接收



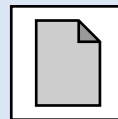
第五节、中断方式的数据发送



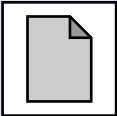
第六节、USART的PAL库编程

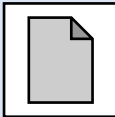


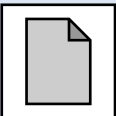
第七节、设备间的指令通信

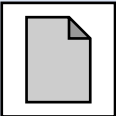


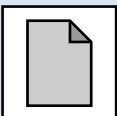
第一节、串口通信

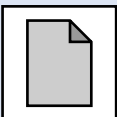
1.1. 串口简介 

1.2. 串口的接线 

1.3. 串口的数据帧格式 

1.4. 异步通信与波特率 

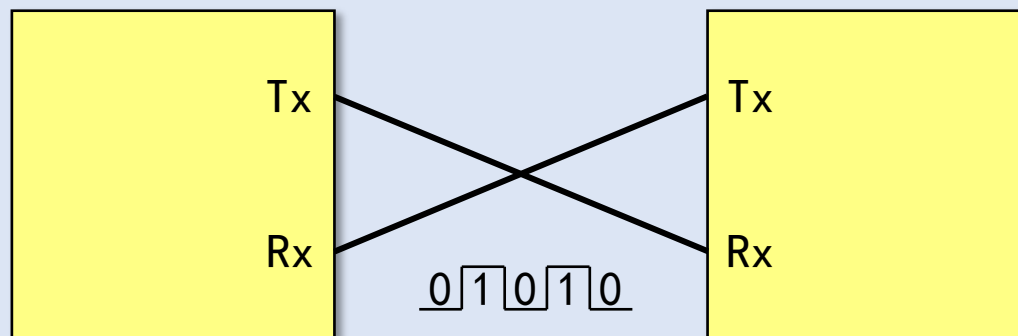
1.5. 硬件流控 

1.6. 随堂测试 

1.1. 串口简介

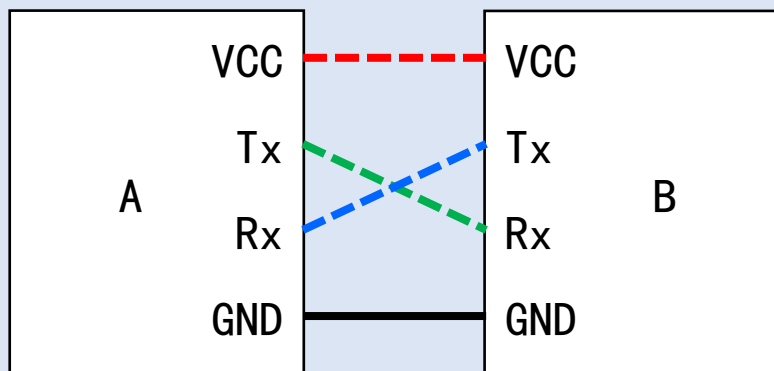


Tx = Transmit - 发送



Rx = Receive - 接收

1.2. 串口接线

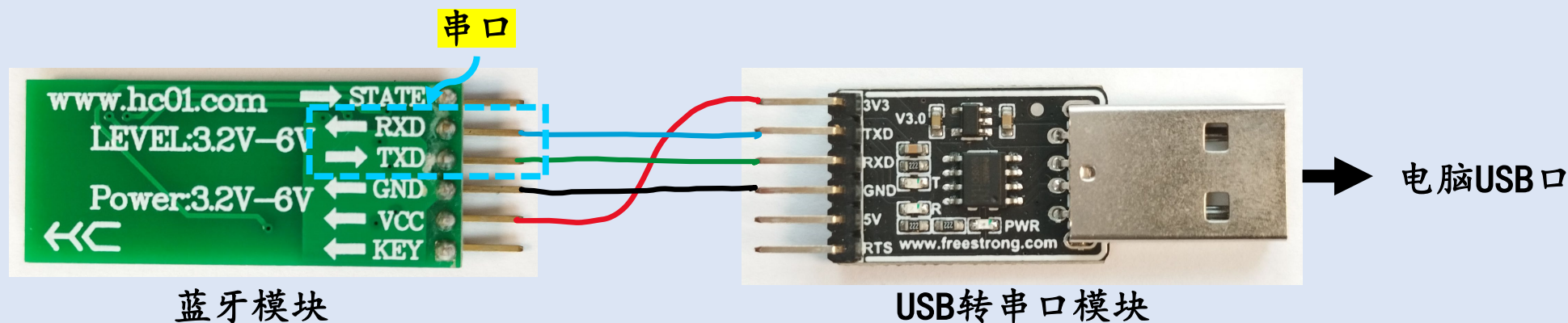


Tx: 发送数据

Rx: 接收数据

GND: 共地, 使通信双方获取相同的电压基准

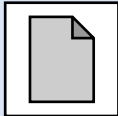
VCC: 使用通信的一方给另一方供电

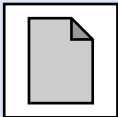


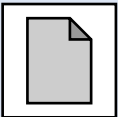
蓝牙模块

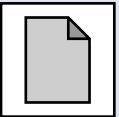
USB转串口模块

1.3. 数据帧

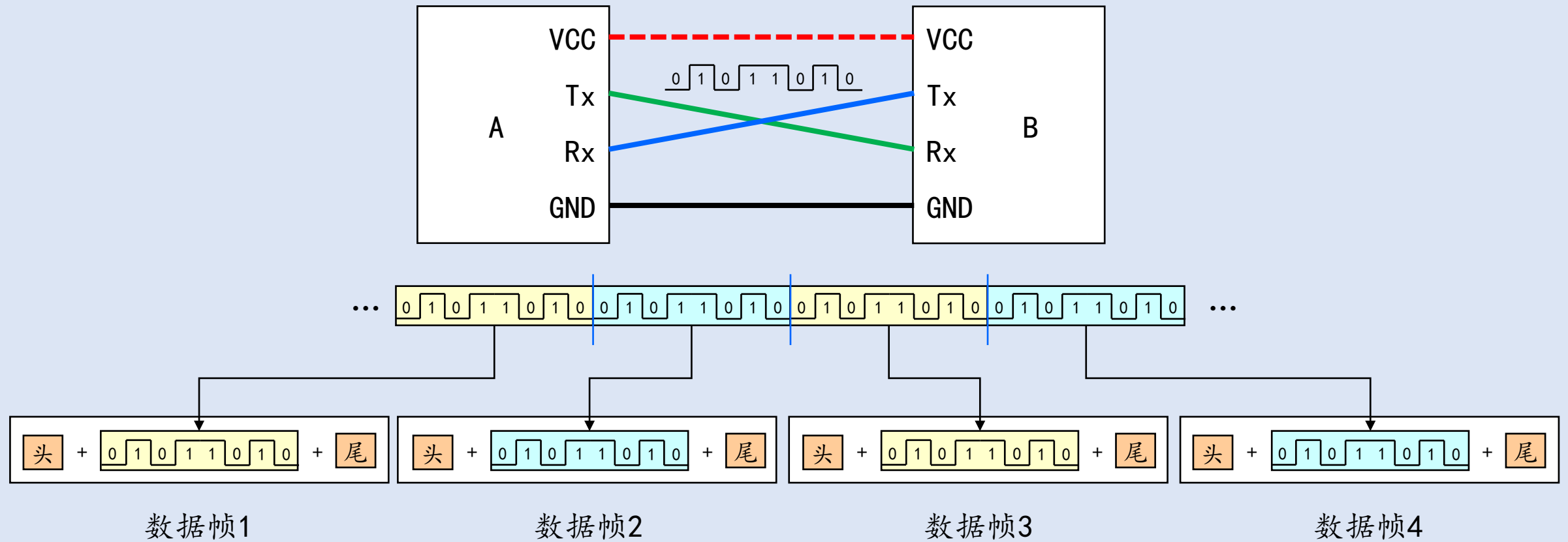
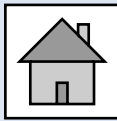
1.3.1. 数据帧的概念 

1.3.2. 串口的数据帧格式 

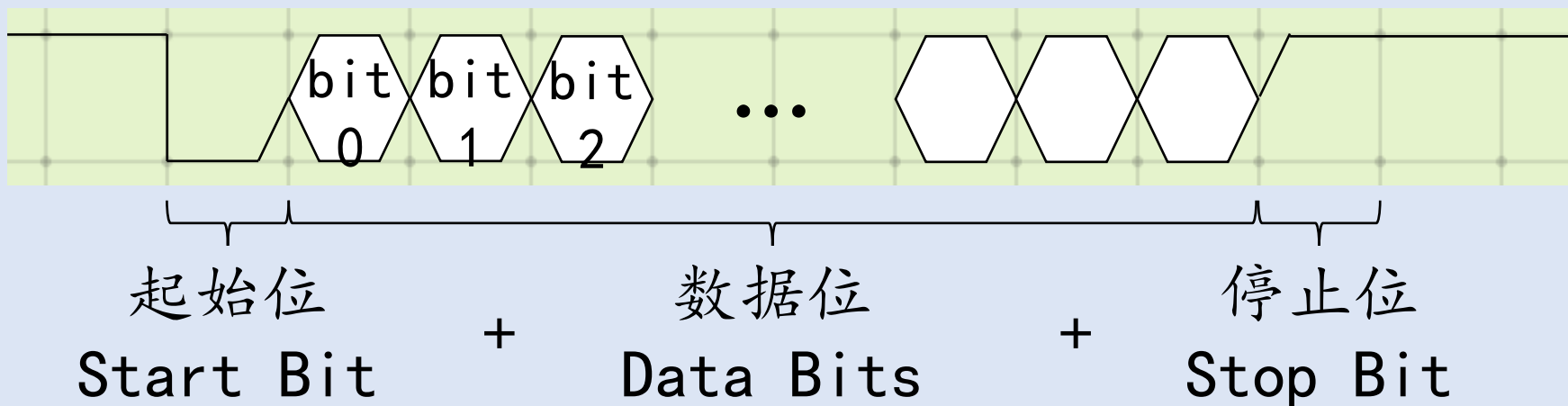
1.3.3. 单个数据发送 

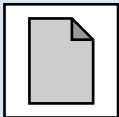
1.3.4. 连续发送 

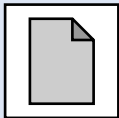
1.3.1. 数据帧的概念

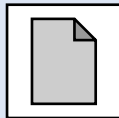


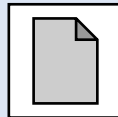
1.3.2. 串口的数据帧格式

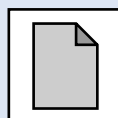


1.3.2.1. 空闲 

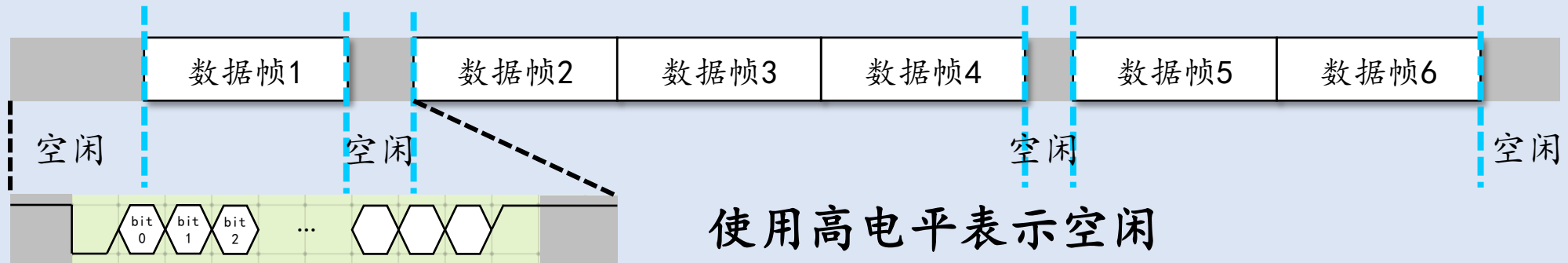
1.3.2.3. 数据位 

1.3.2.5. 停止位 

1.3.2.2. 起始位 

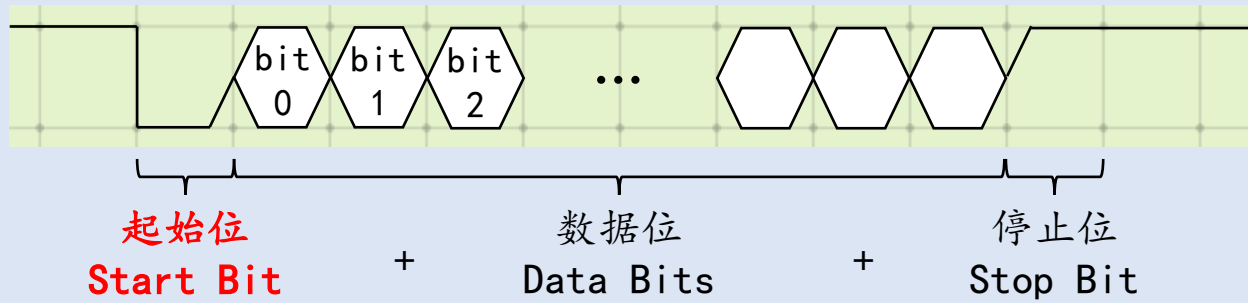
1.3.2.4. 校验位 

1.3.2.1. 空闲

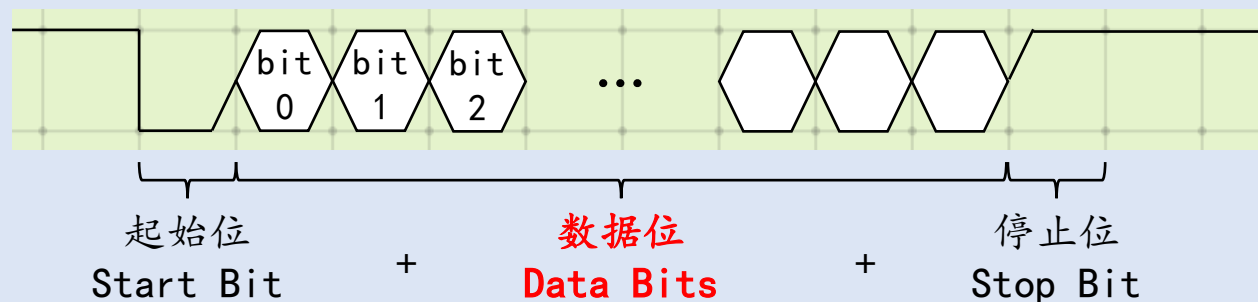


1.3.2.2. 起始位

起始位：表示数据帧的开始，实际使用1位低电平表示起始位



1.3.2.3. 数据位



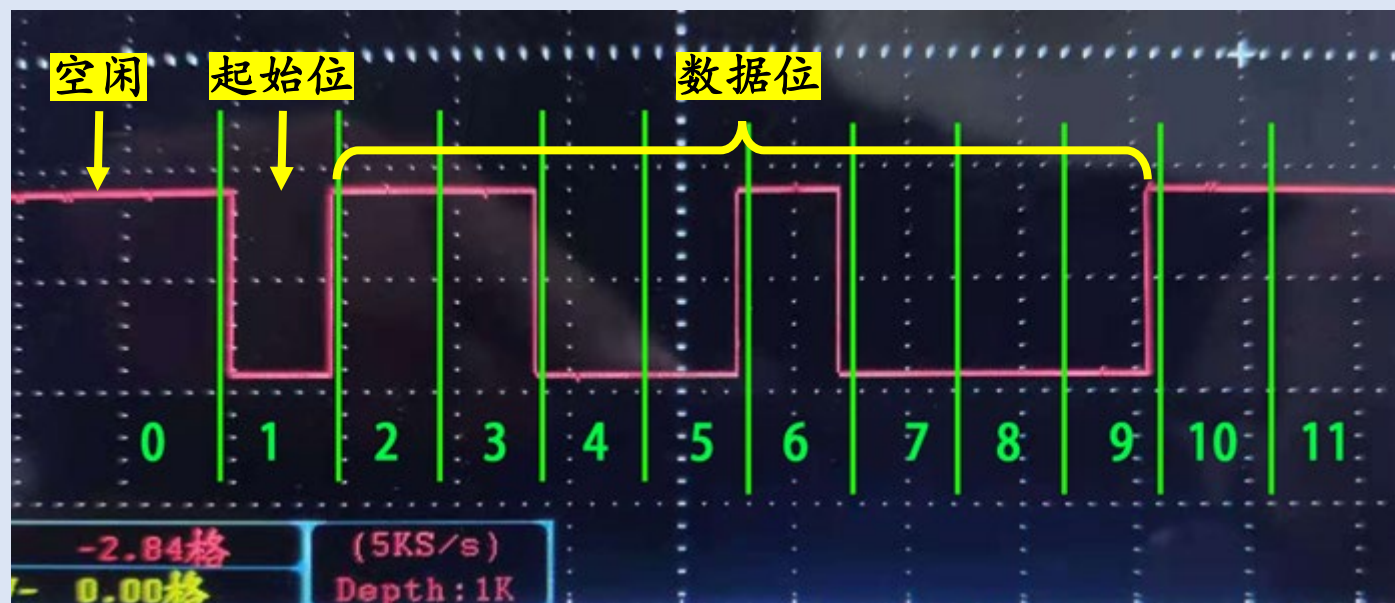
数据位:

用于传输数据

低电平表示0, 高电平表示1

8~9位

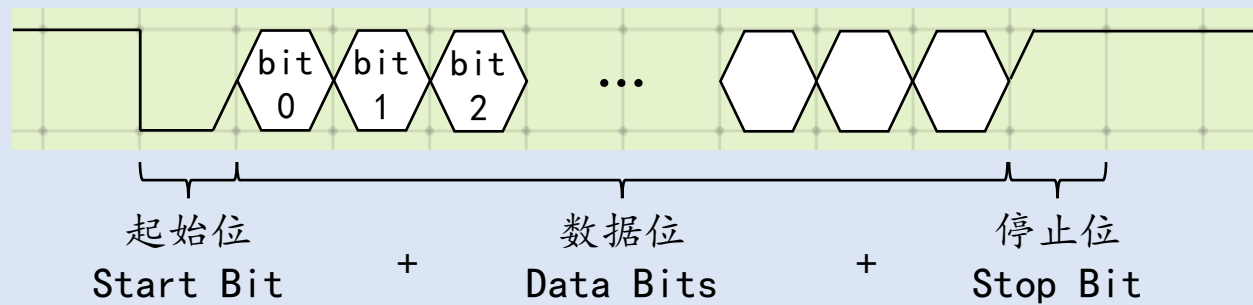
LSB First (Least Significant Bit First) 先传最低有效



MSB				LSB			
0	1	0	1	1	0	1	0
128	...	8	4	2	1		

1.3.2.4. 校验位

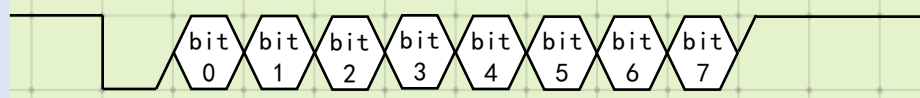
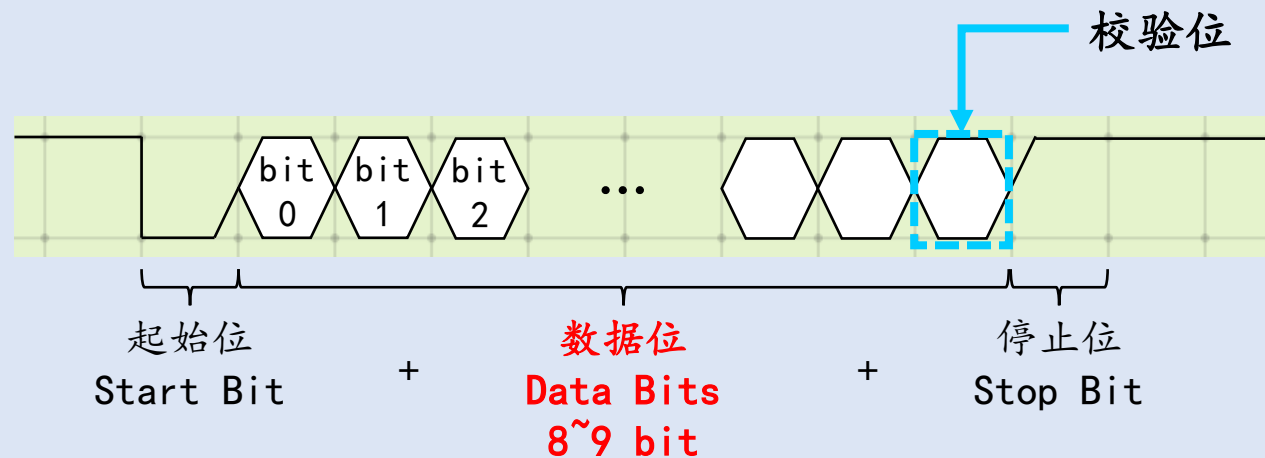
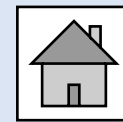
校验位：为了检验数据传输是否出错而设立的比特位



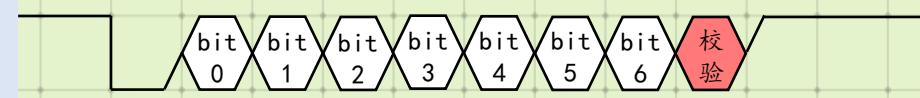
1.3.2.4.1. 校验位和数据位的关系

1.3.2.4.2. 校验规则

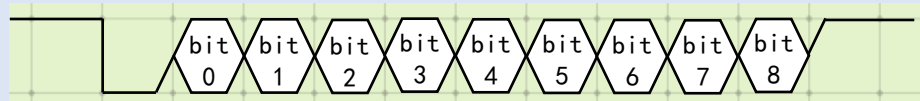
1.3.2.4.1. 校验位和数据位的关系



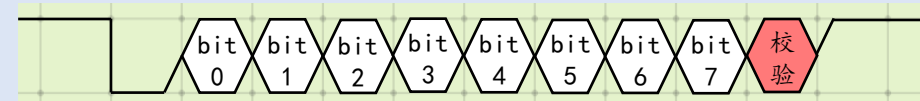
例子1: 8位数据位 = 8位有效载荷



例子2: 8位数据位 = 7位有效载荷+1位校验位



例子3: 9位数据位 = 9位有效载荷



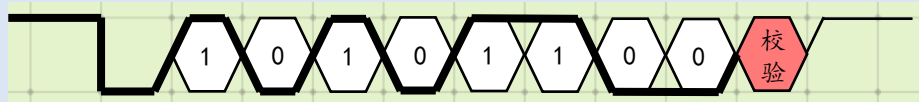
例子4: 9位数据位 = 8位有效载荷+1位校验位

1.3.2.4.2. 奇偶校验的规则

奇偶校验（查1的个数） $\left\{ \begin{array}{l} \text{奇校验：1的个数为奇数} \\ \text{偶校验：1的个数为偶数} \end{array} \right.$

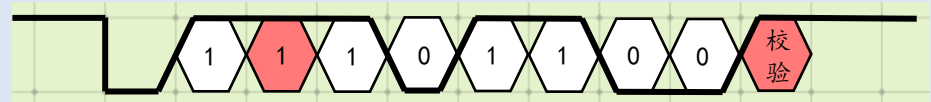
9位数据位 = 8位有效载荷+1位校验位

发送方：发送0x35 (00110101)，奇校验



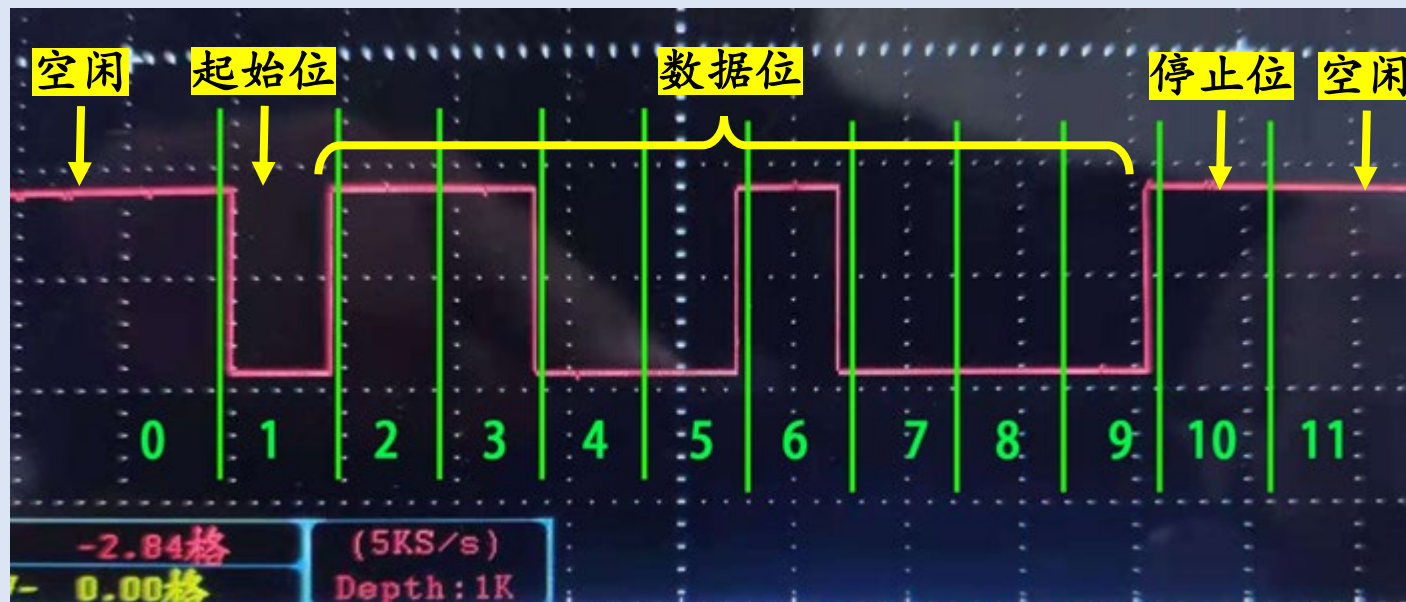
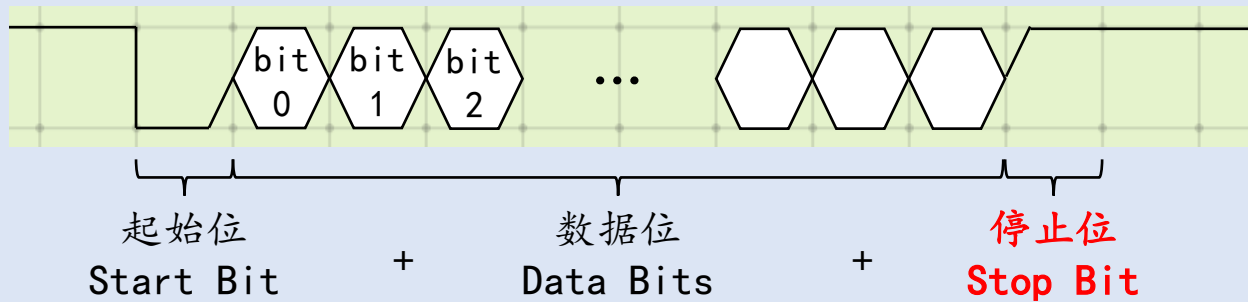
出错
→

接收方：奇校验



1.3.2.5. 停止位

停止位：表示数据帧的结束，高电平，长度可以是0.5，1，1.5，2

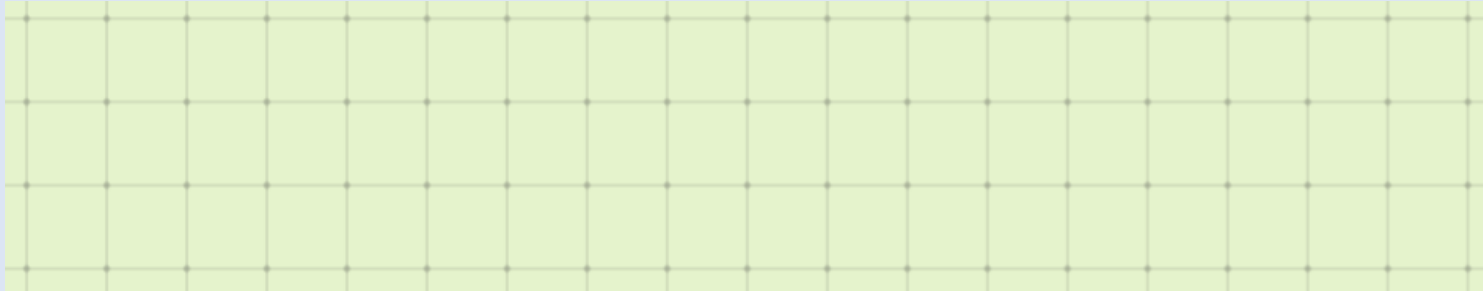


1.3.3. 单个数据发送



发送0x5b

9位数据位，偶校验，1位停止位

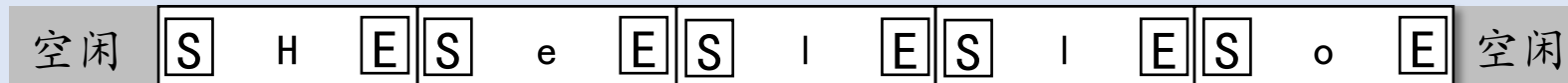
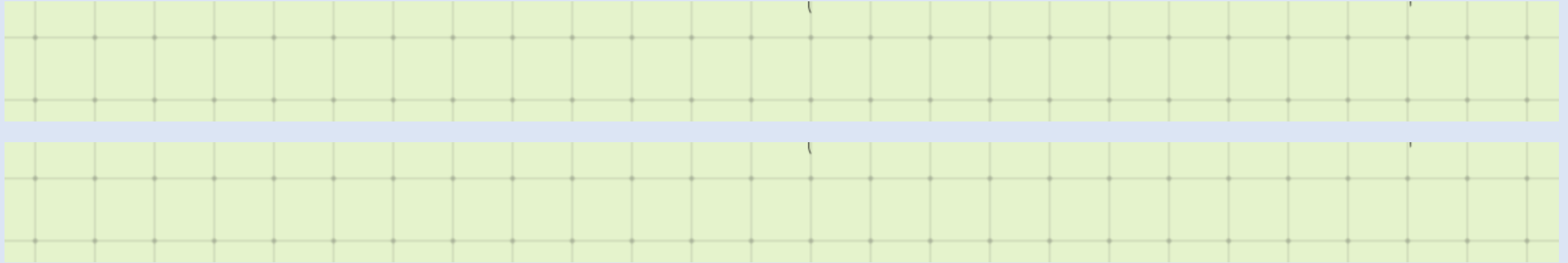


1.3.4. 连续发送

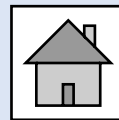
发送Hello

8位数据位，无校验，1位停止位

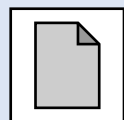
H - 0x48 (0100 1000) e - 0x65 (0110 0101) l - 0x6c (0110 1010) o - 0x6f (0110 1111)



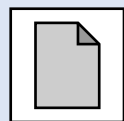
1.4. 异步通信和波特率



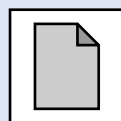
1.4.1. 同步通信



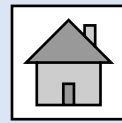
1.4.2. 异步通信



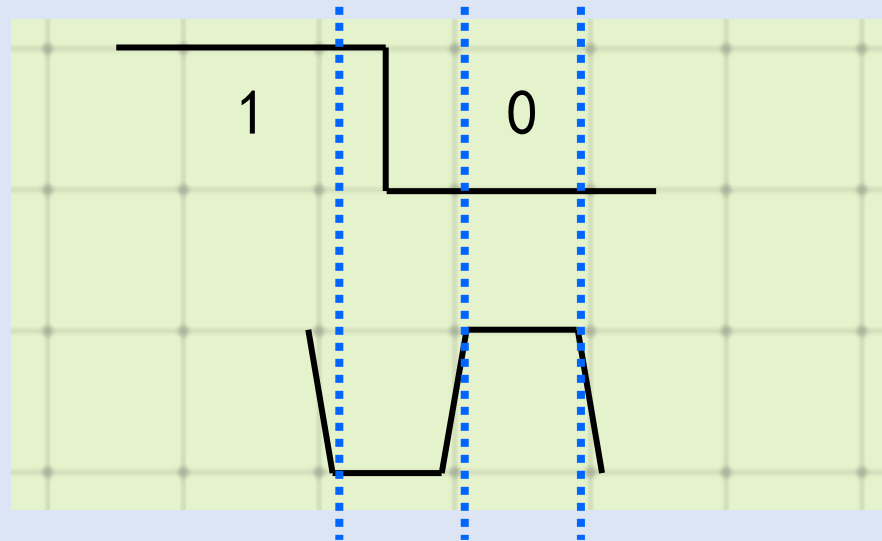
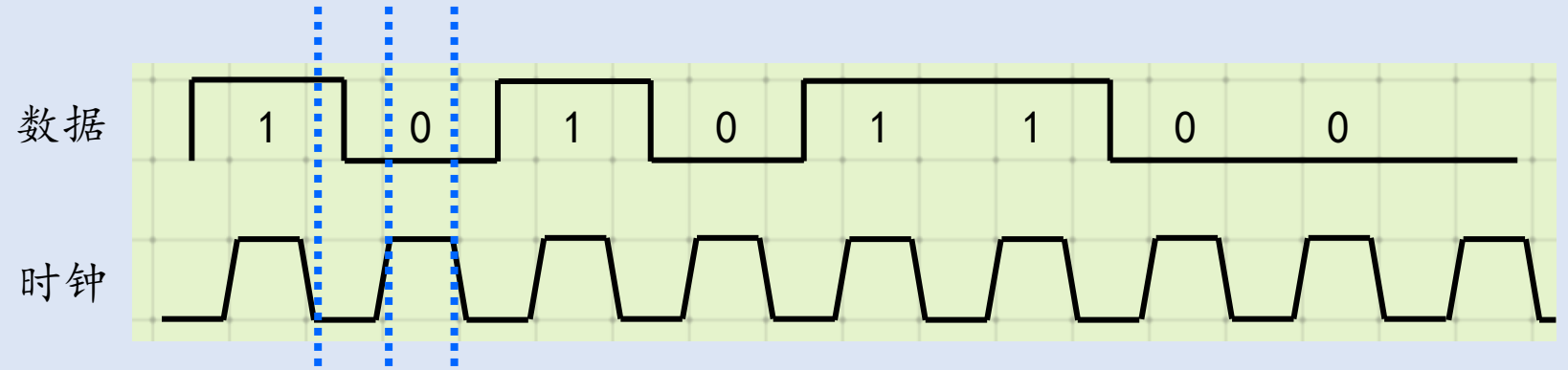
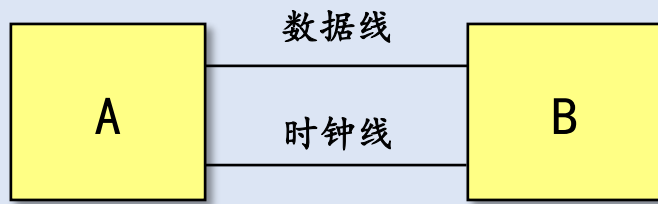
1.4.3. 波特率



1.4.1. 同步通信



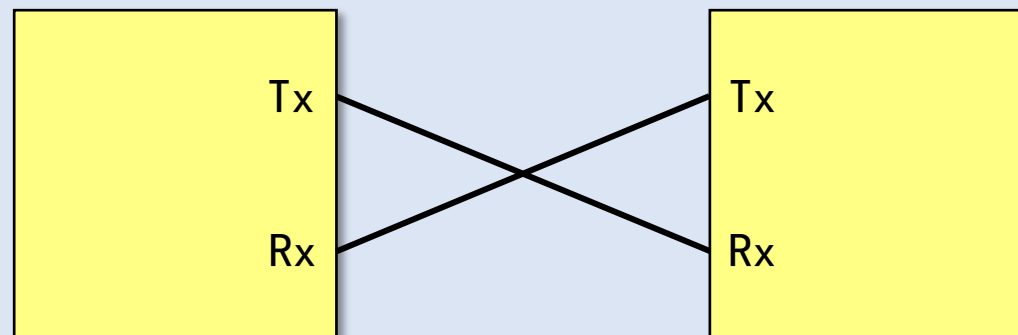
同步通信：收发双方共享时钟



1.4.2. 异步通信



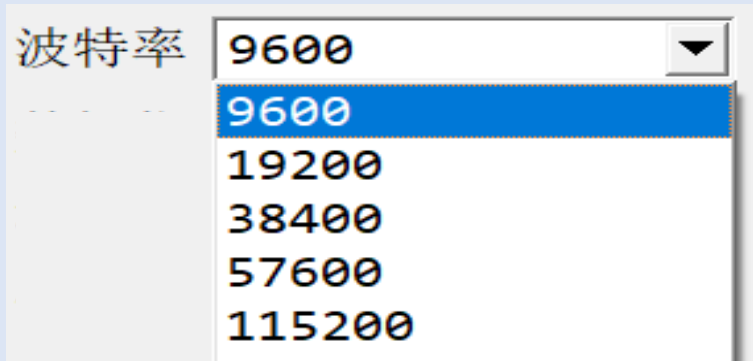
异步通信：收发双方不共享时钟



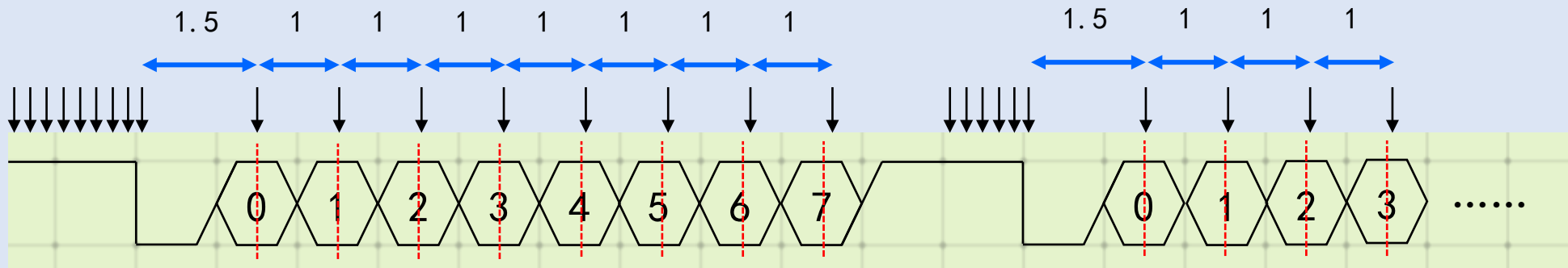
1.4.3. 波特率



波特率：每秒钟传输码元的个数



波特率	单个码元的宽度
9600	$1/9600 \text{ s} = 0.104 \text{ ms}$
19200	$1/19200 \text{ s} = 52 \text{ us}$
38400	$1/38400 \text{ s} = 26 \text{ us}$
57600	$1/57600 \text{ s} = 17.4 \text{ us}$
115200	$1/115200 \text{ s} = 8.7\text{us}$



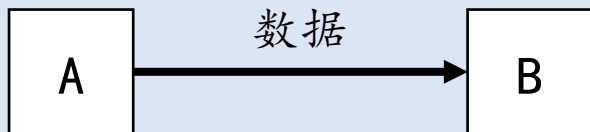
正常通信的条件：1. 采用相同的波特率 2. 采用相同的数据帧格式

1.5. 硬件流控

1.5.1. 流控的概念

1.5.2. 串口流控的工作原理

1.5.1. 流控的概念

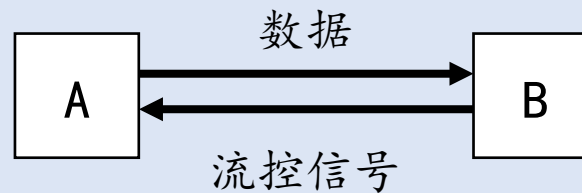


A: 人之初，性本善。

A: 性相近，习相远。

A: 苟不教，性乃迁。

A: 教之道，贵以专。



A: 人之初，性本善。

B: 下一句

A: 性相近，习相远。

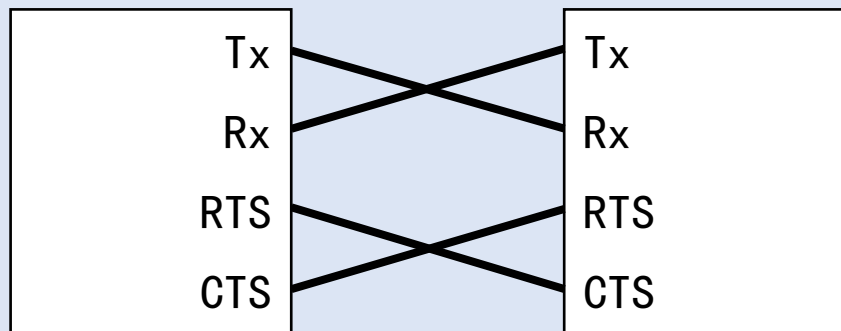
B: 下一句

A: 苟不教，性乃迁。

B: 下一句

A: 教之道，贵以专。

1.5.2. 串口流控的工作原理

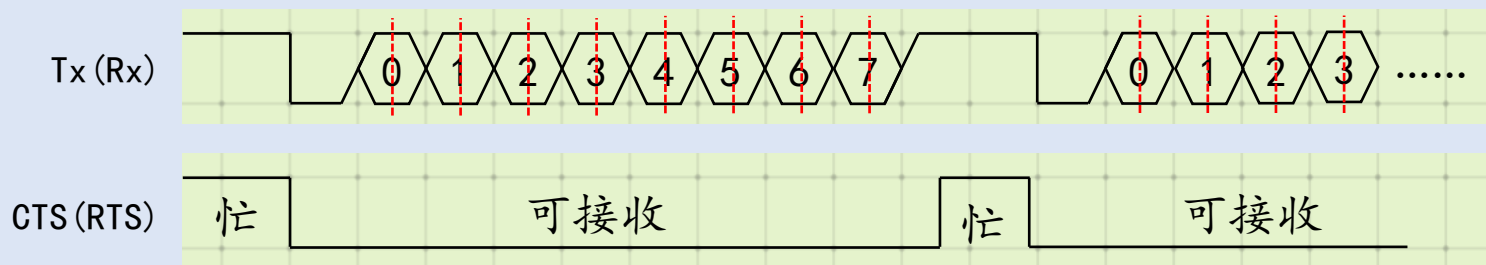
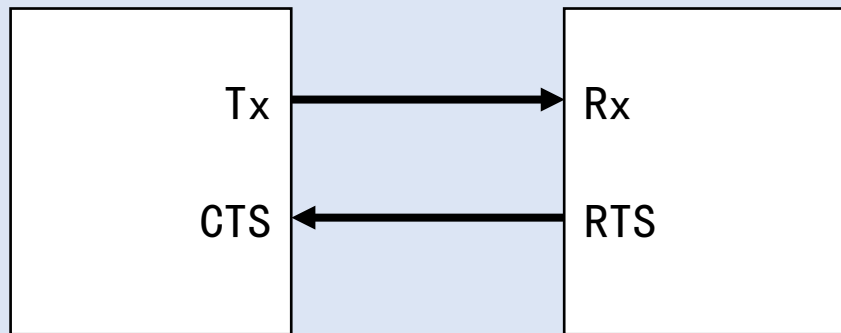


Tx: 数据发送

Rx: 数据接收

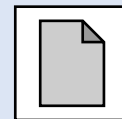
CTS (Clear To Send): 可以发送。低电平有效，发送方有效

RTS (Request To Send): 请求发送。低电平有效，接收方有效

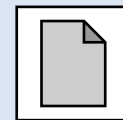


1.6. 随堂测验

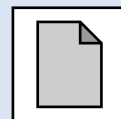
题目1：说出Tx、Rx、CTS、RTS、VCC、GND的作用



题目2：串口的数据帧最多包含多少个位？

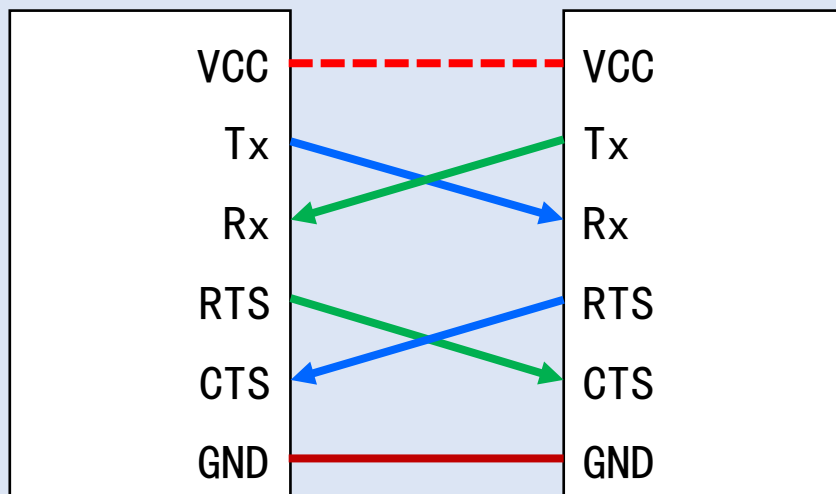


题目3：说出奇校验和偶校验的规则



题目4：通信双方应当如何选择数据帧格式和波特率？

题目1:



Tx - 发送数据

Rx - 接收数据

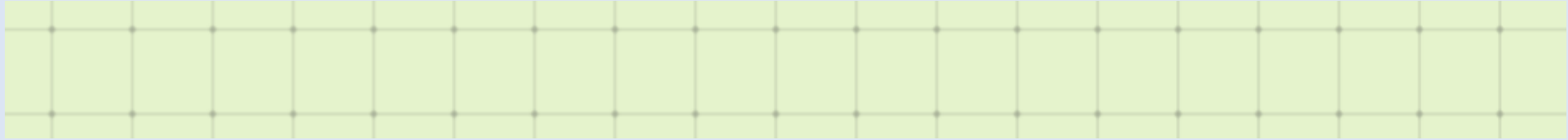
GND - 共地，提供参考电位

VCC - 一方给另一方供电

CTS - Clear To Send, 清空发送

RTS - Request To Send, 请求发送

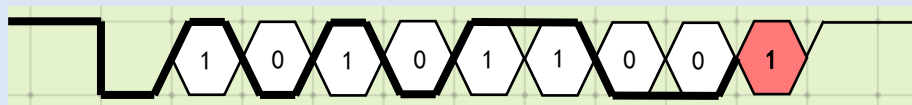
题目2:



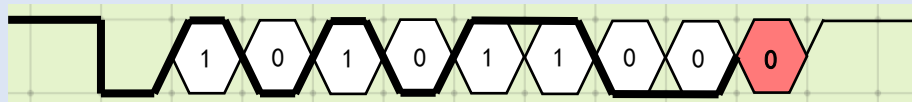
题目3:



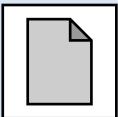
奇校验：要求数据位有奇数个1

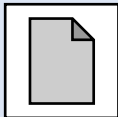


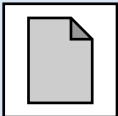
偶校验：要求数据位有偶数个1

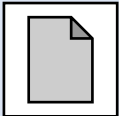


第二节、USART

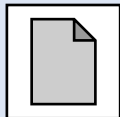
2.1. USART简介 

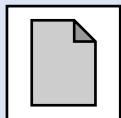
2.2. USART的工作原理 

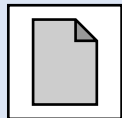
2.3. 错误标志位 

2.4. 随堂测试 

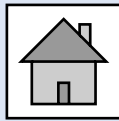
2.1. USART简介

问题1：什么是USART? 

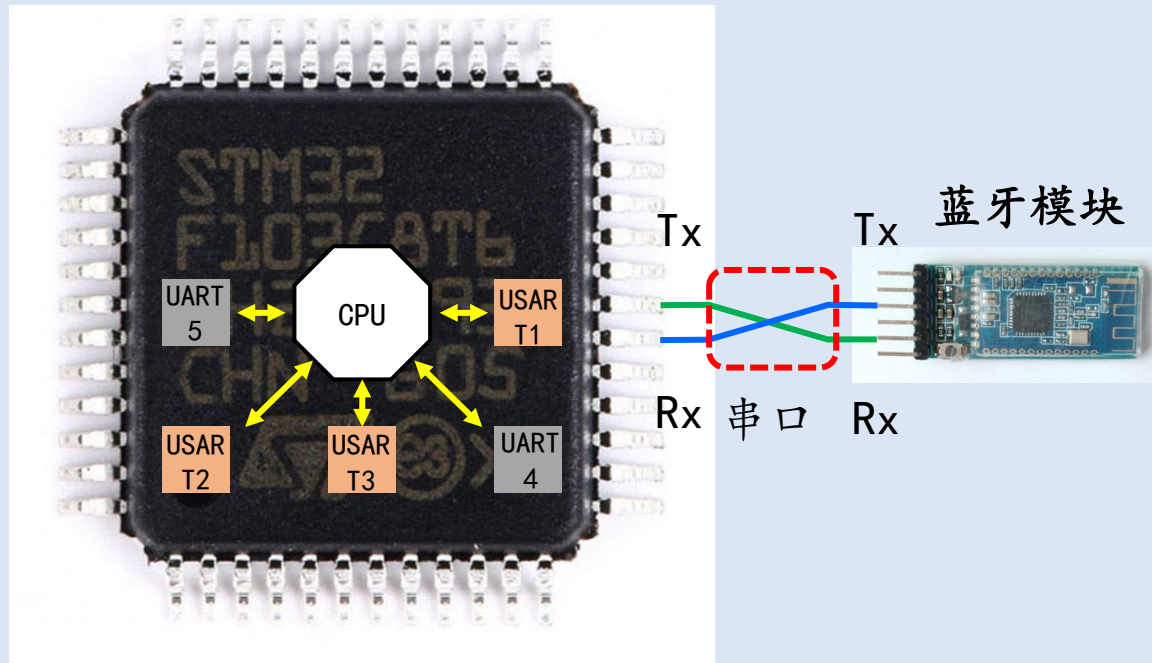
问题2：USART名字的含义? 

问题3：如何使用USART? 

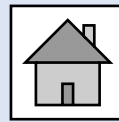
问题1：USART是什么？



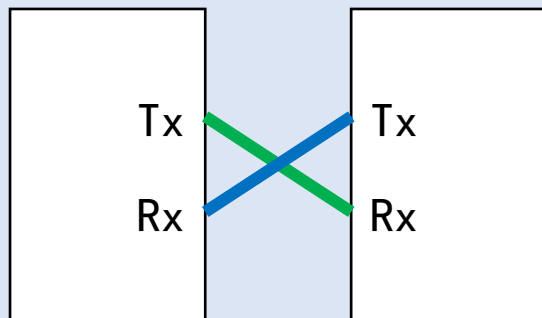
USART就是stm32内部的串口



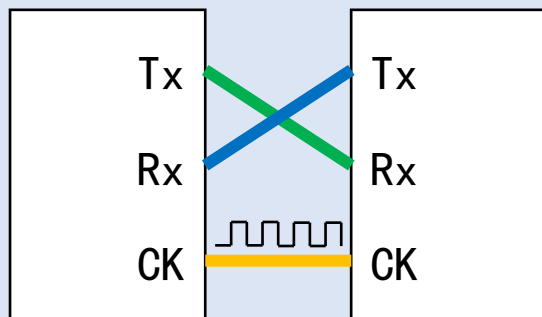
问题2: USART名字的含义?



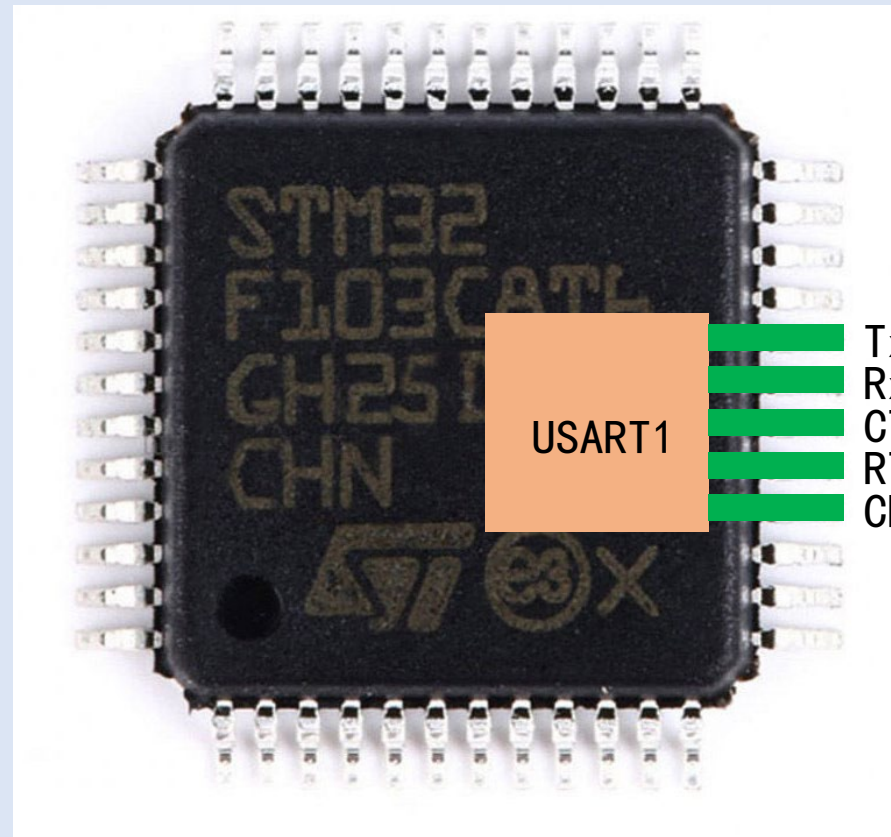
U	S	A	R	T
Universal	Synchronous	Asynchronous	Receiver	Transmitter
通用	同步 (共享时钟)	异步 (不共享时钟)	接收器	发射器



串口的异步模式



串口的同步模式(不常用)



USART1

USART2

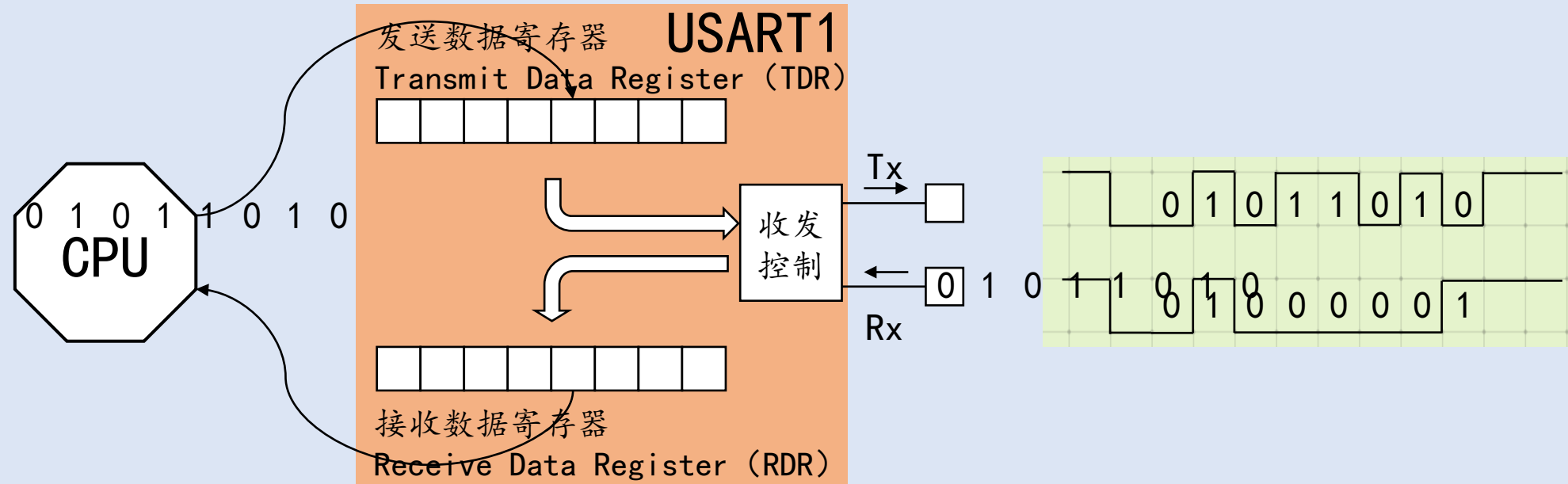
USART3

UART4

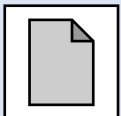
UART5

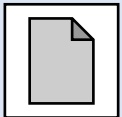
Tx ⇒ 发送
Rx ⇐ 接收
CTS
RTS
CK 时钟

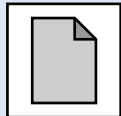
问题3：如何使用USART?

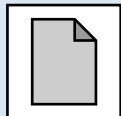


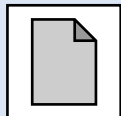
2.2. USART的工作原理

2.2.1. 串并转换电路 

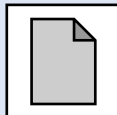
2.2.2. USART的完整框图 

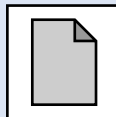
2.2.3. USART参数配置 

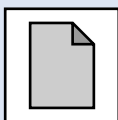
2.2.4. 数据发送过程 

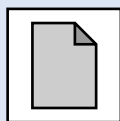
2.2.5. 数据接收过程 

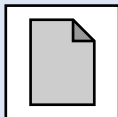
2.2.1. 串并转换电路

问题1：什么是串并转换？ 

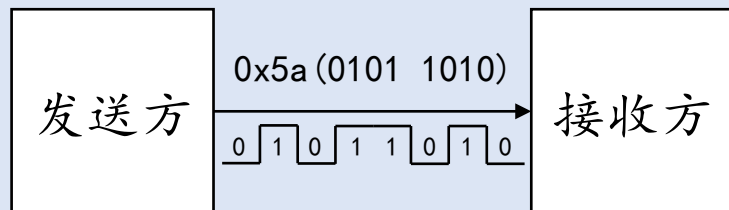
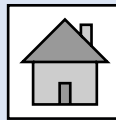
问题2：为什么要进行串并转换？ 

2.2.1.1. 移位寄存器 

2.2.1.2. 串并转换电路 

2.2.1.3. USART的基本模型 

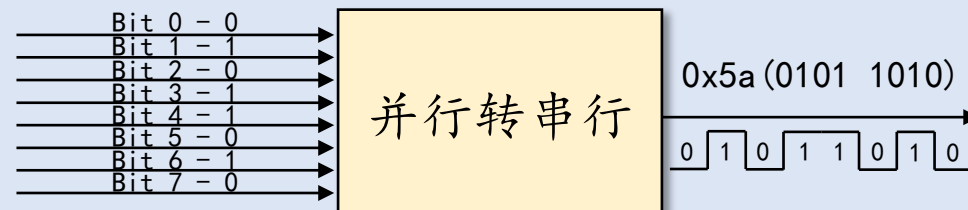
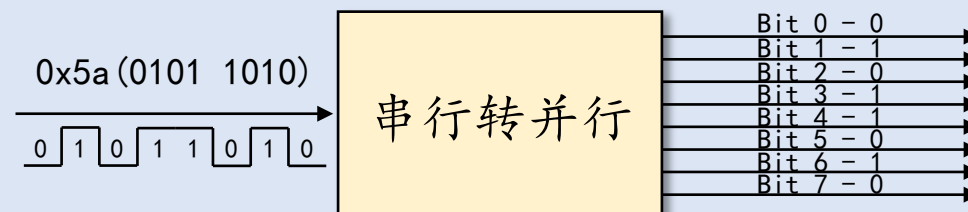
问题1：什么是串并转换？



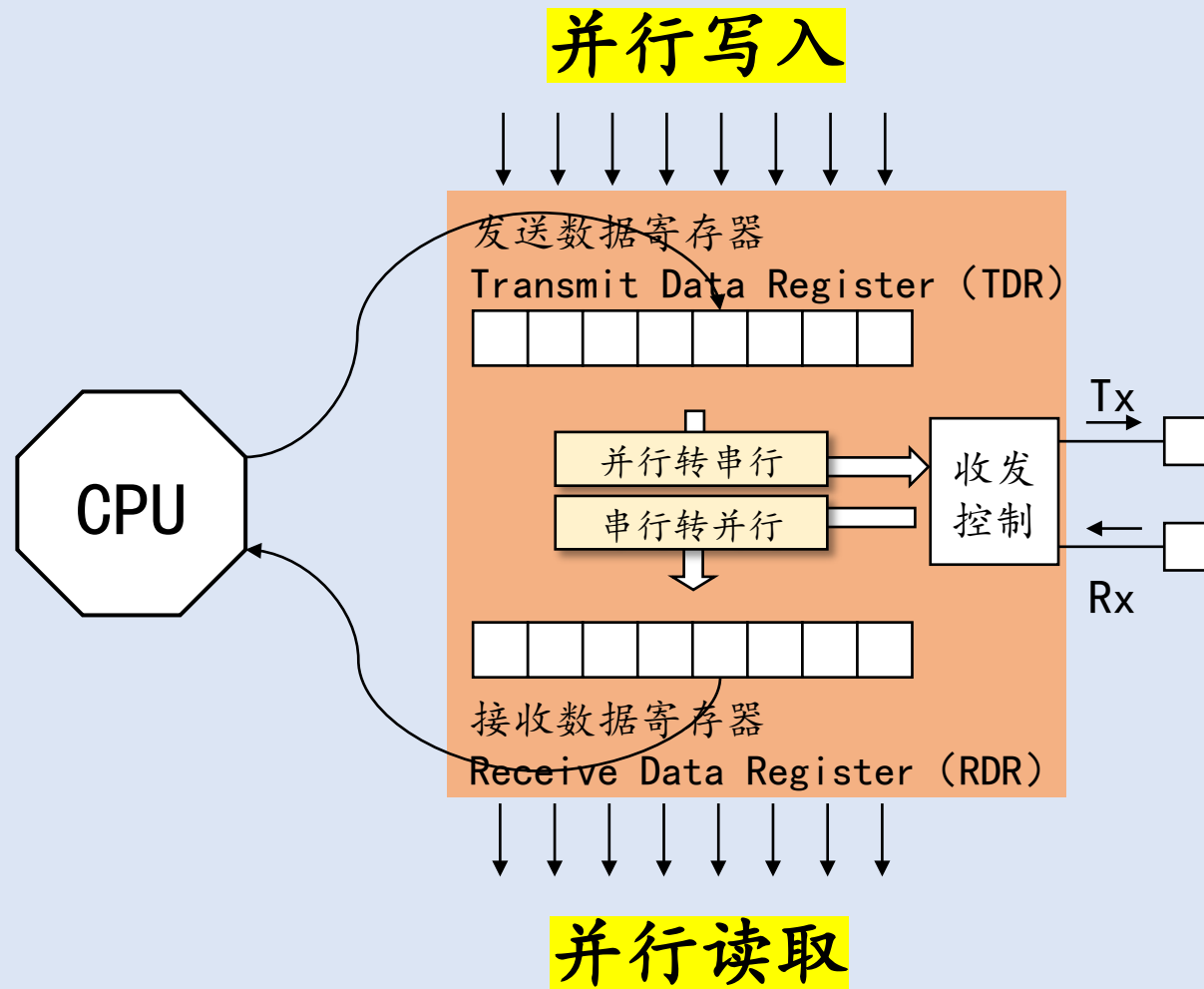
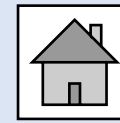
串行



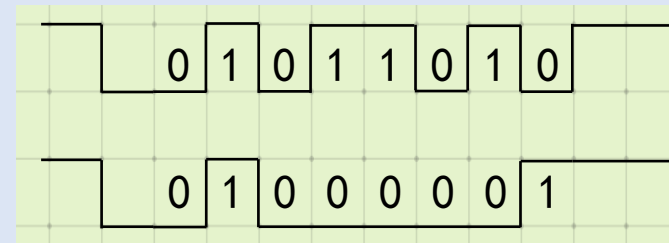
并行



问题2：为什么要进行串并转换？



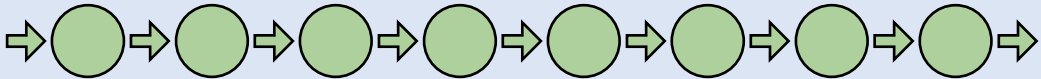
串行发送



串行接收

2.2.1.1. 移位寄存器

每动作一次所有比特位向右移动



初始状态 ⇒ 0 ⇒ 1 ⇒ 0 ⇒ 1 ⇒ 1 ⇒ 0 ⇒ 1 ⇒ 0 ⇒

第1次移动 ⇒ ⇒ 0 ⇒ 1 ⇒ 0 ⇒ 1 ⇒ 1 ⇒ 0 ⇒ 1 ⇒ 0

第2次移动 ⇒ ⇒ ⇒ 0 ⇒ 1 ⇒ 0 ⇒ 1 ⇒ 1 ⇒ 0 ⇒ 1

第3次移动 ⇒ ⇒ ⇒ ⇒ 0 ⇒ 1 ⇒ 0 ⇒ 1 ⇒ 1 ⇒ 0

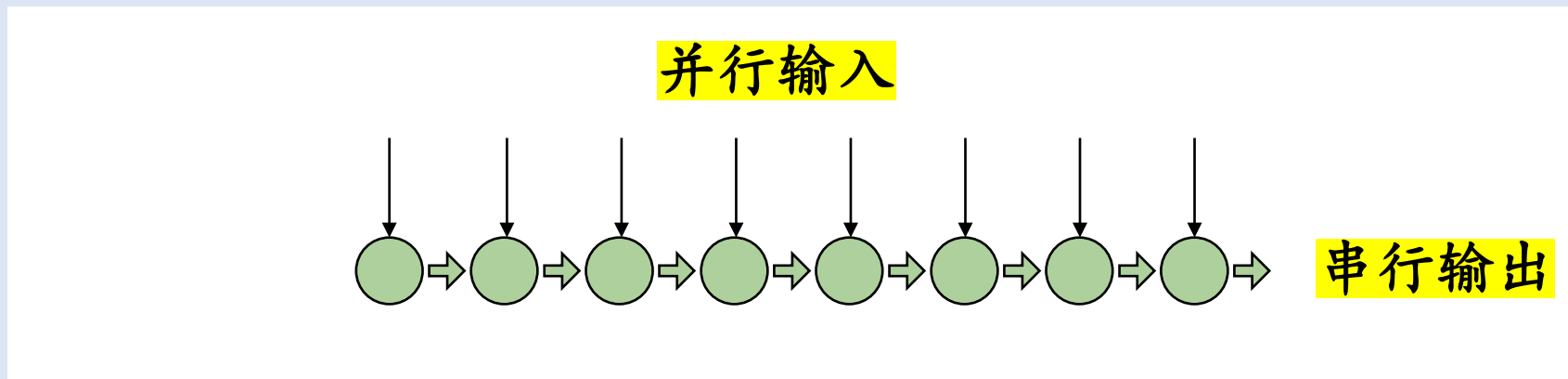
...

...

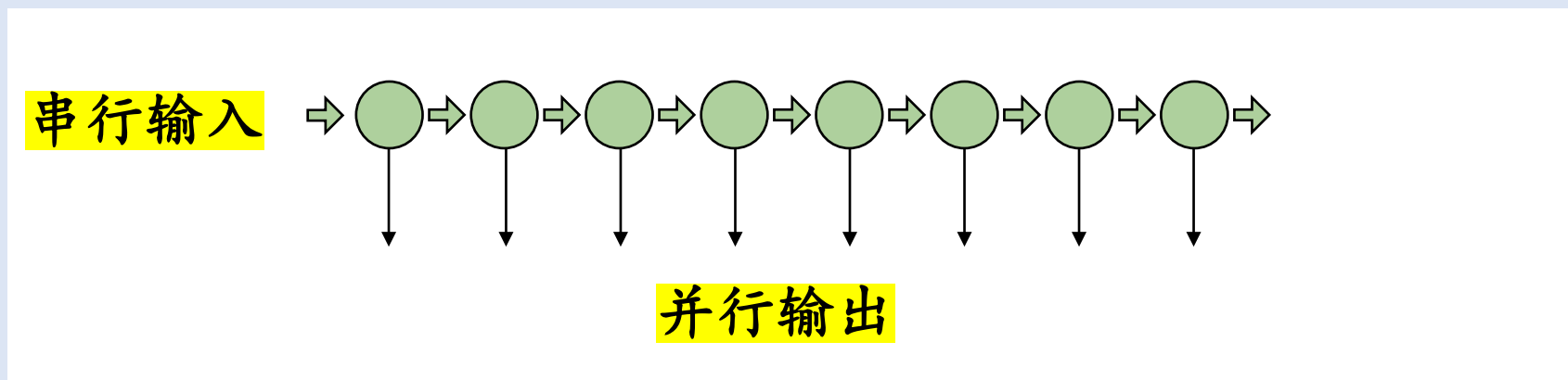
第8次移动 ⇒ ⇒ ⇒ ⇒ ⇒ ⇒ ⇒ ⇒ ⇒ 0

2.2.1.2. 串并转换电路

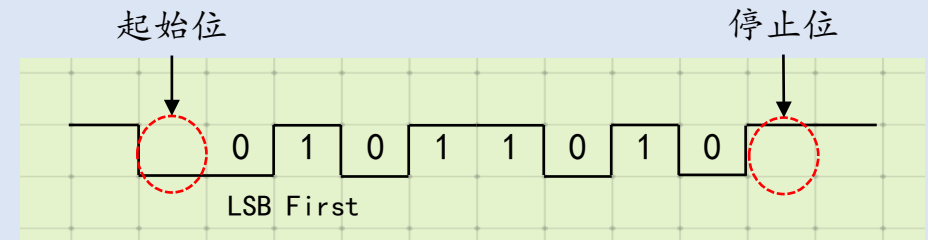
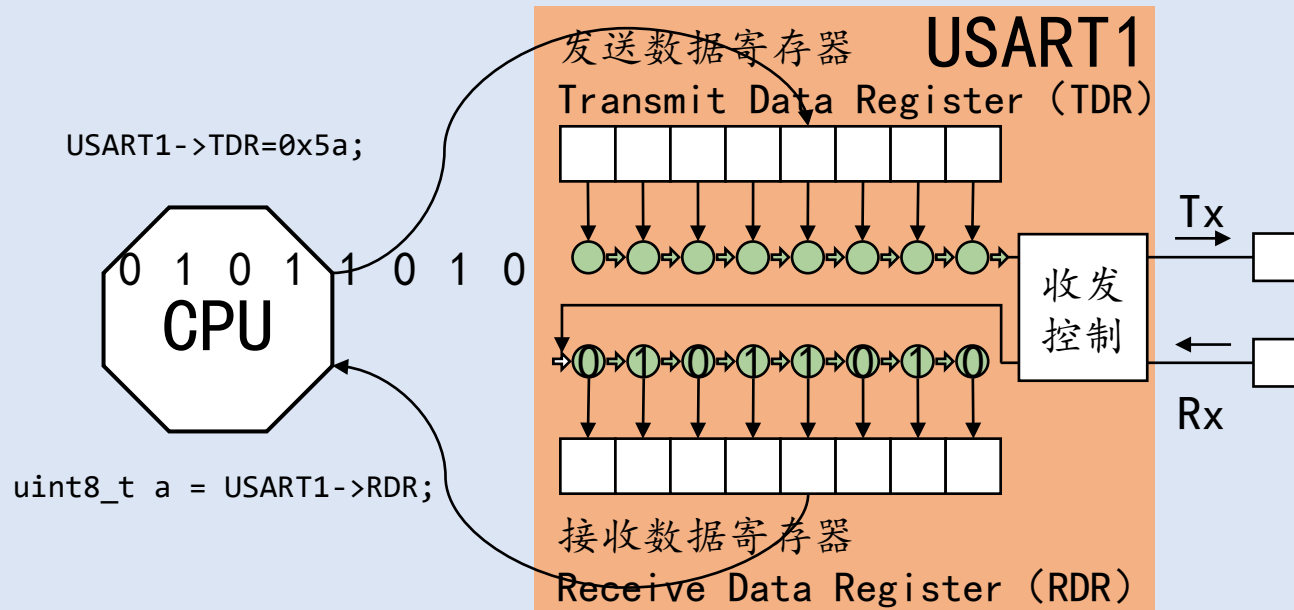
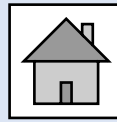
并转串



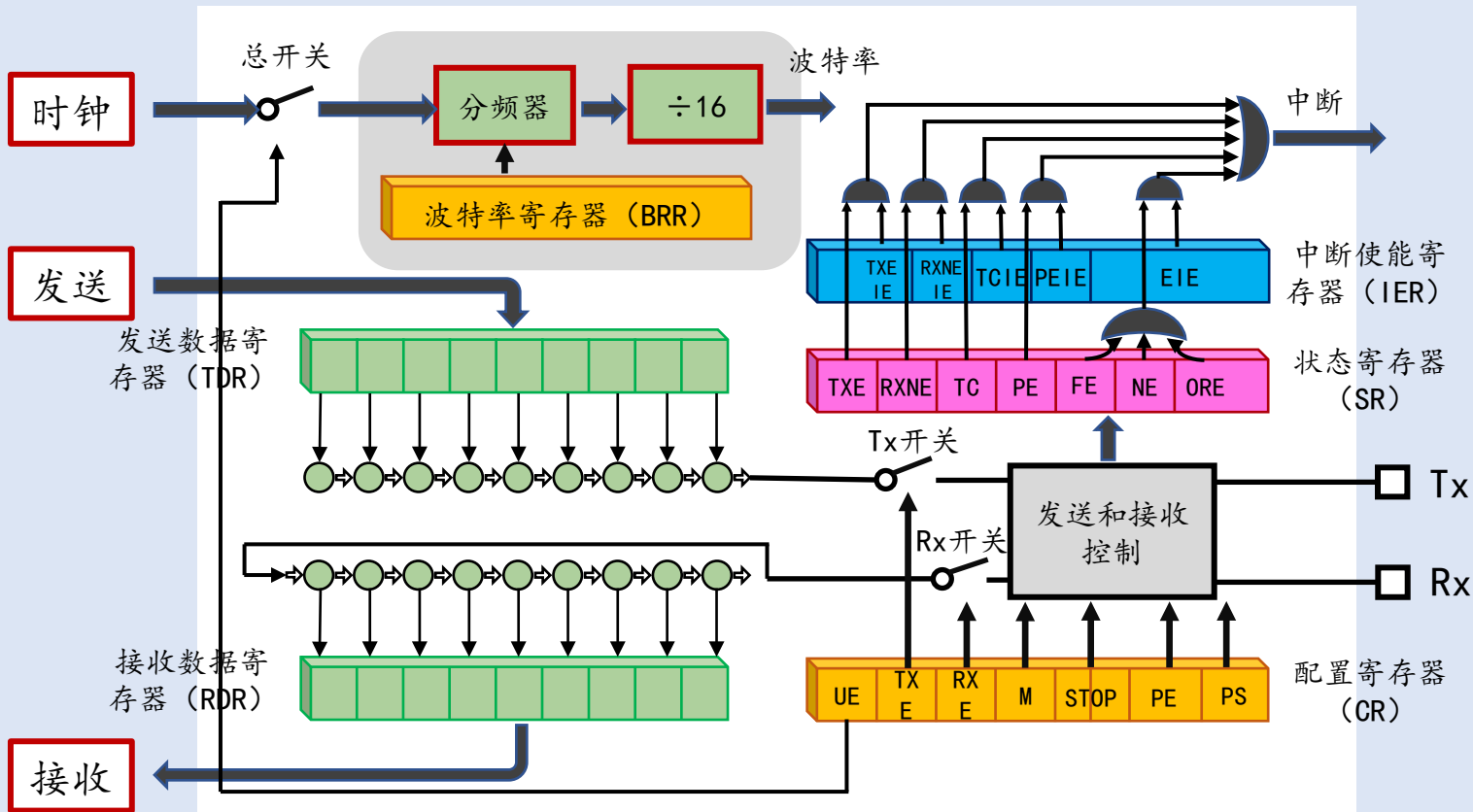
串转并



2.2.1.3. USART的基本模型

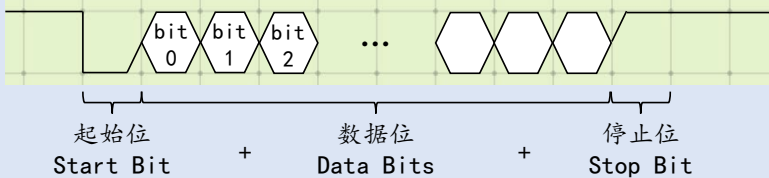


2.2.2. USART的完整框图

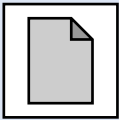


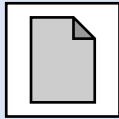
寄存器组

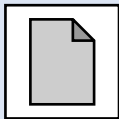
- TDR - 发送数据寄存器
- RDR - 接收数据寄存器
- CR - 配置寄存器
- SR - 状态寄存器
- BRR - 波特率寄存器
- IER - 中断使能寄存器

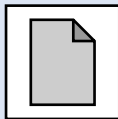


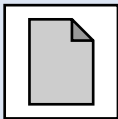
2.2.3. USART参数配置

2.2.3.1. 数据帧格式设置 

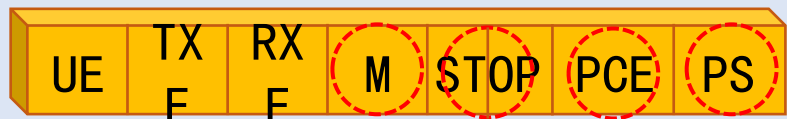
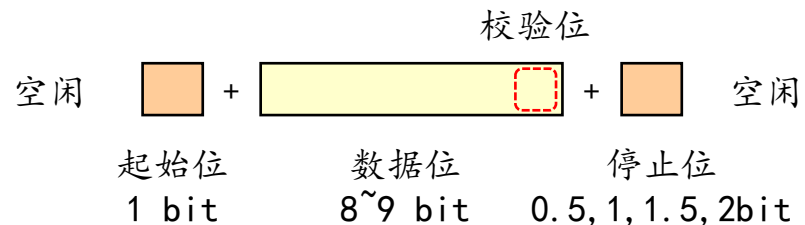
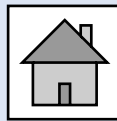
2.2.3.2. 数据传输的方向选择 

2.2.3.3. 波特率设置 

2.2.3.4. USART的总开关 

2.2.3.5. 练习 

2.3.2.1. 数据帧格式设置



M - (Word Length) 数据位长度。

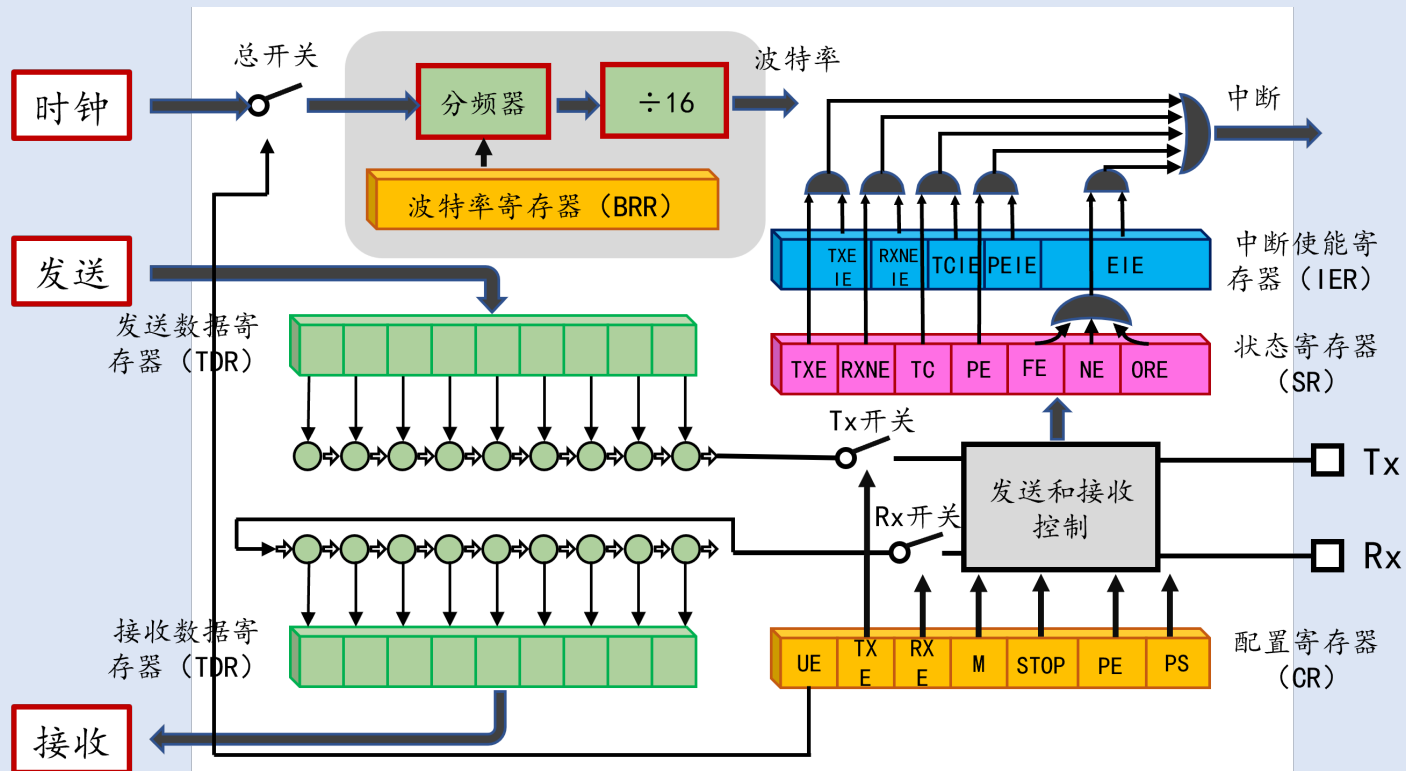
M=0, 8位; M=1, 9位

PCE - (Parity Control Enable) 奇偶校验使能

PCE=0, 禁止; PCE=1, 使能

PS - (Parity Selection) 奇偶选择

PS=0, 偶校验; PS=1, 奇校验

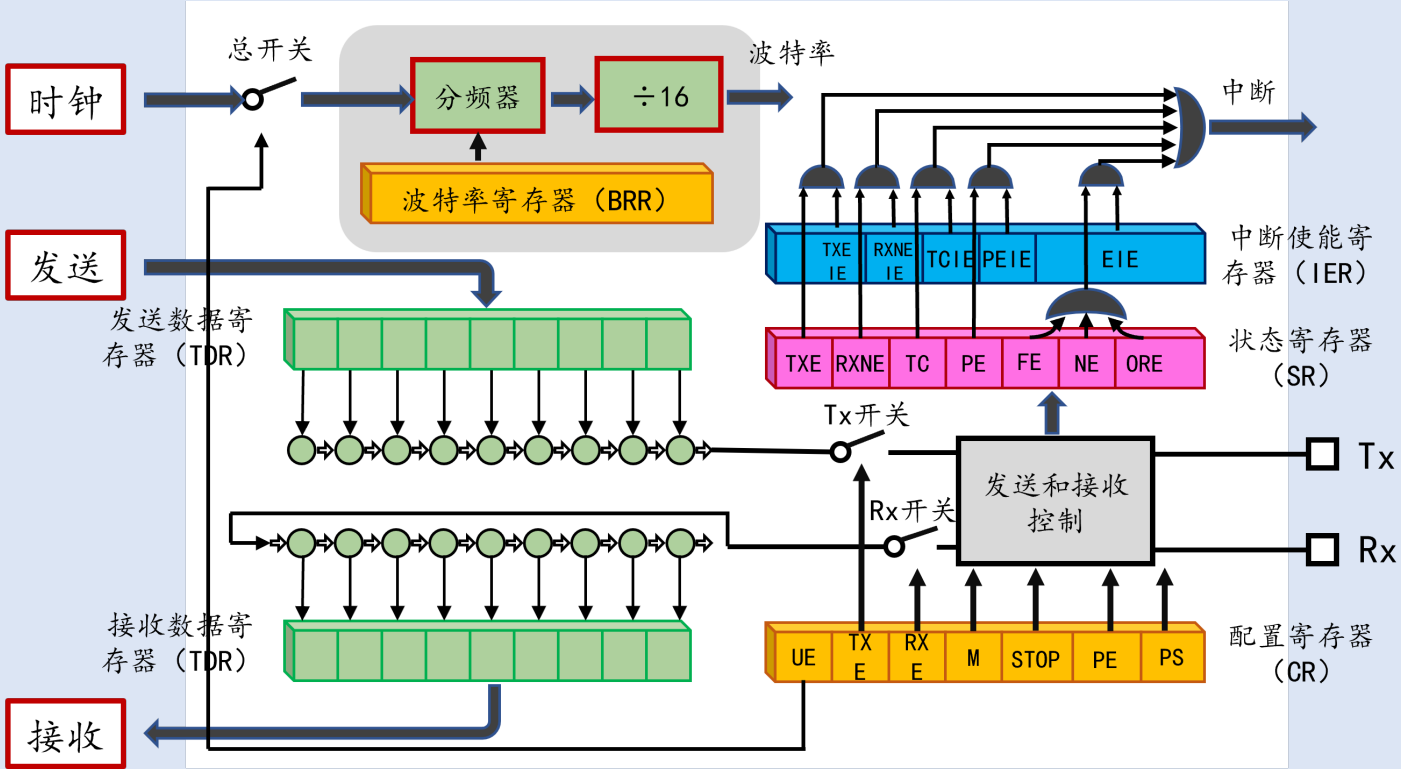
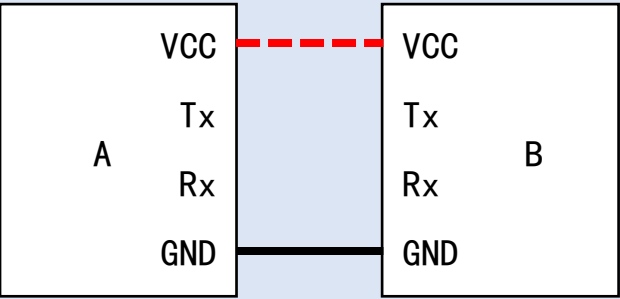
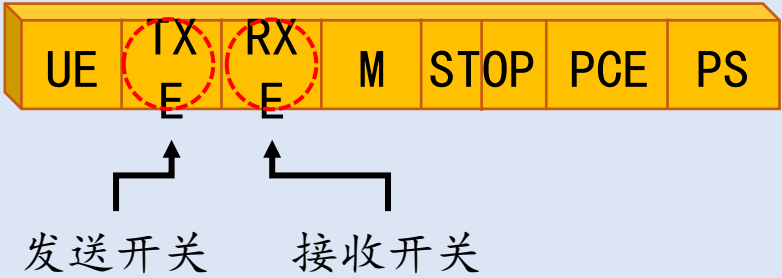


STOP - (Stop Bits) 停止位长度

STOP=00 - 1位; STOP=01 - 0.5位;

STOP=10 - 2位; STOP=11 - 1.5位

2.3.2.2. 数据传输的方向选择



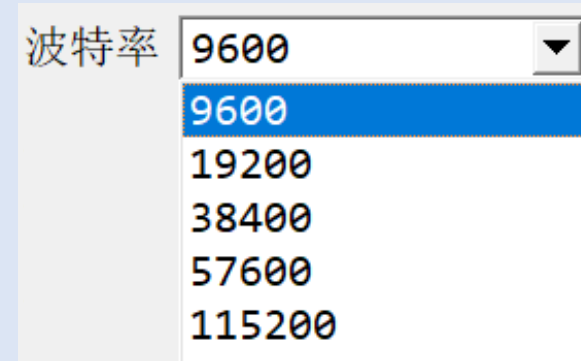
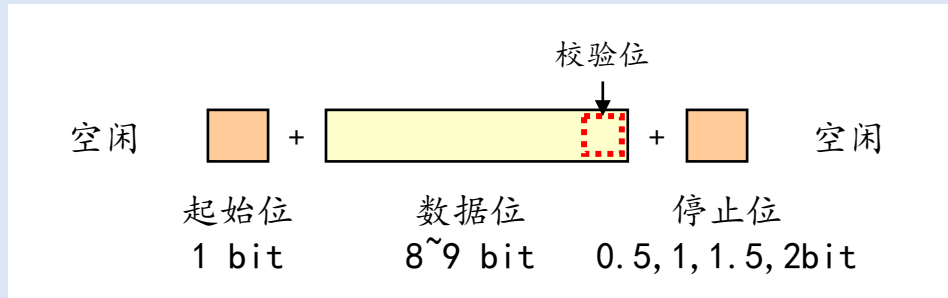
2.3.2.3. 波特率设置

2.3.2.3.1. 波特率的概念

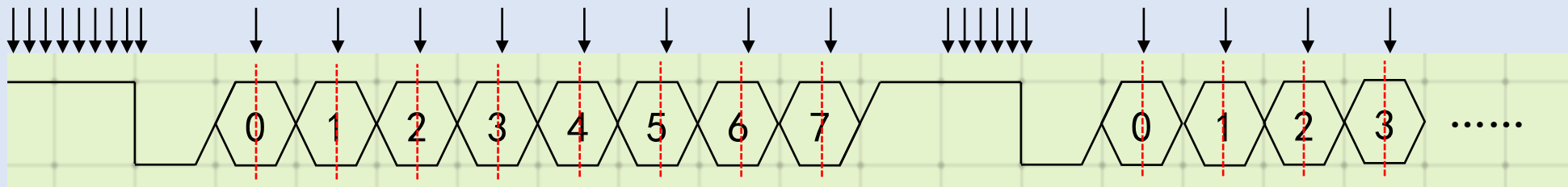
2.3.2.3.2. 波特率产生的原理

2.3.2.3.3. 具体电路

2.3.2.3.1. 波特率的概念

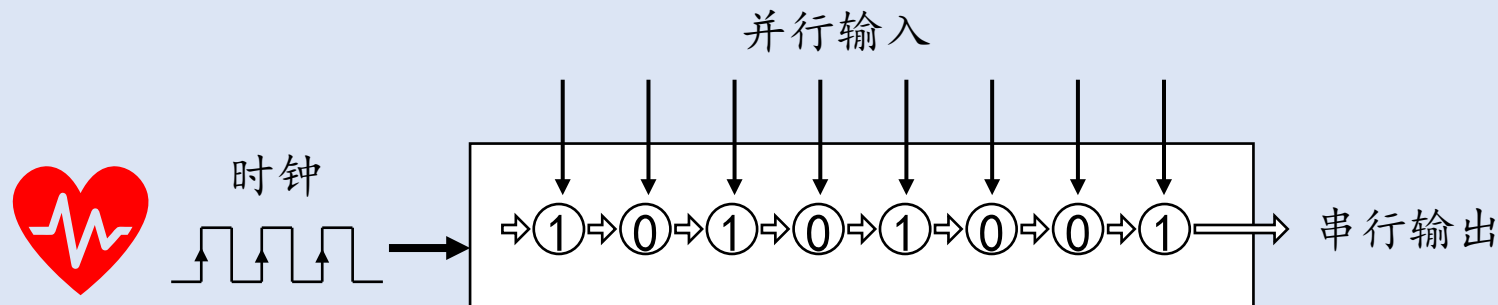
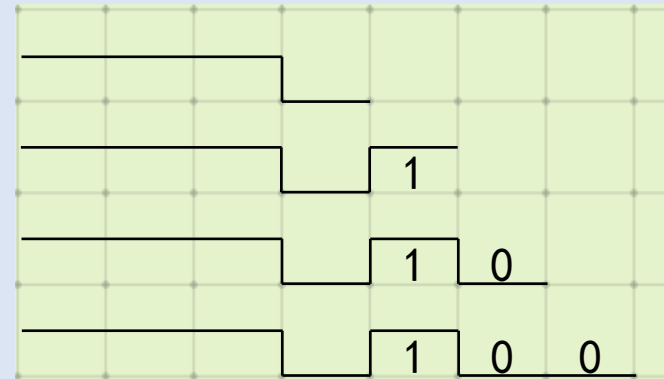


波特率：每秒钟传输码元的个数

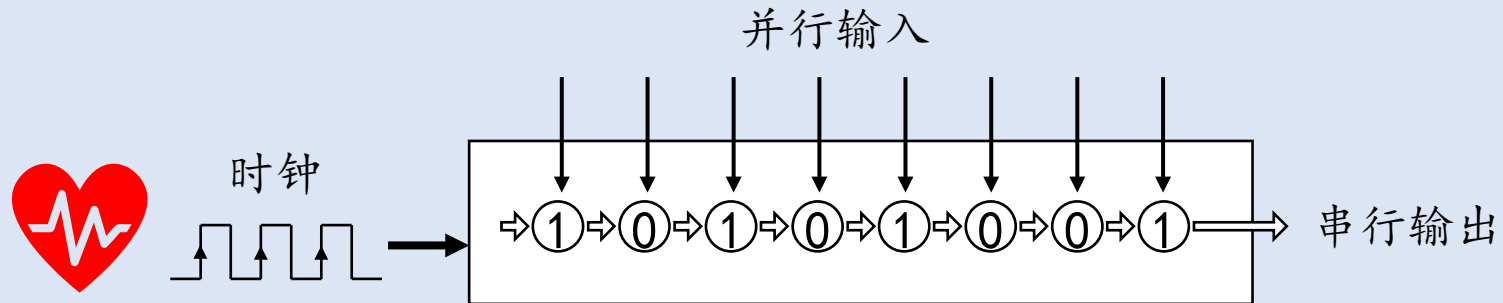


2.3.2.3.2. 波特率产生的原理

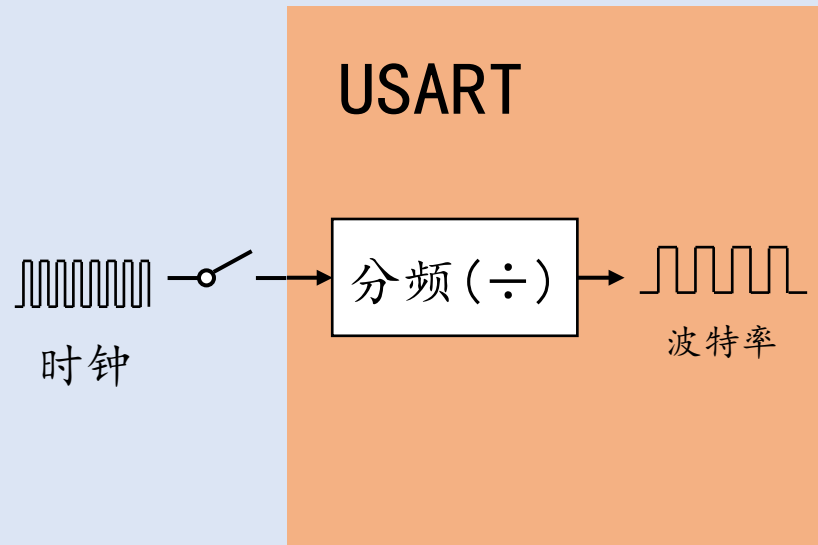
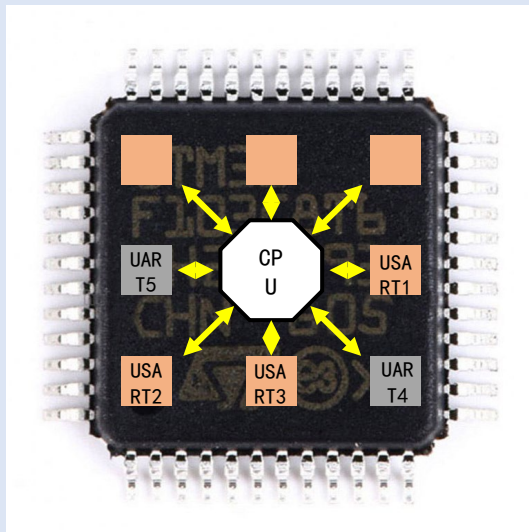
⇒ 1 ⇒ 0 ⇒ 1 ⇒ 0 ⇒ 1 ⇒ 0 ⇒ 0 ⇒ 1 ⇒
⇒ 1 ⇒ 0 ⇒ 1 ⇒ 0 ⇒ 1 ⇒ 0 ⇒ 0 ⇒ 1 ⇒ 1
⇒ 1 ⇒ 0 ⇒ 1 ⇒ 0 ⇒ 1 ⇒ 0 ⇒ 1 ⇒ 0 ⇒ 0
⇒ 1 ⇒ 0 ⇒ 1 ⇒ 0 ⇒ 1 ⇒ 0 ⇒ 1 ⇒ 0 ⇒ 0



2.3.2.3.2. 波特率产生的原理

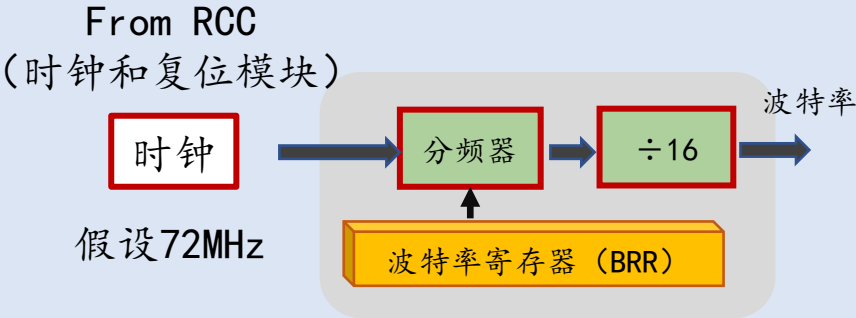
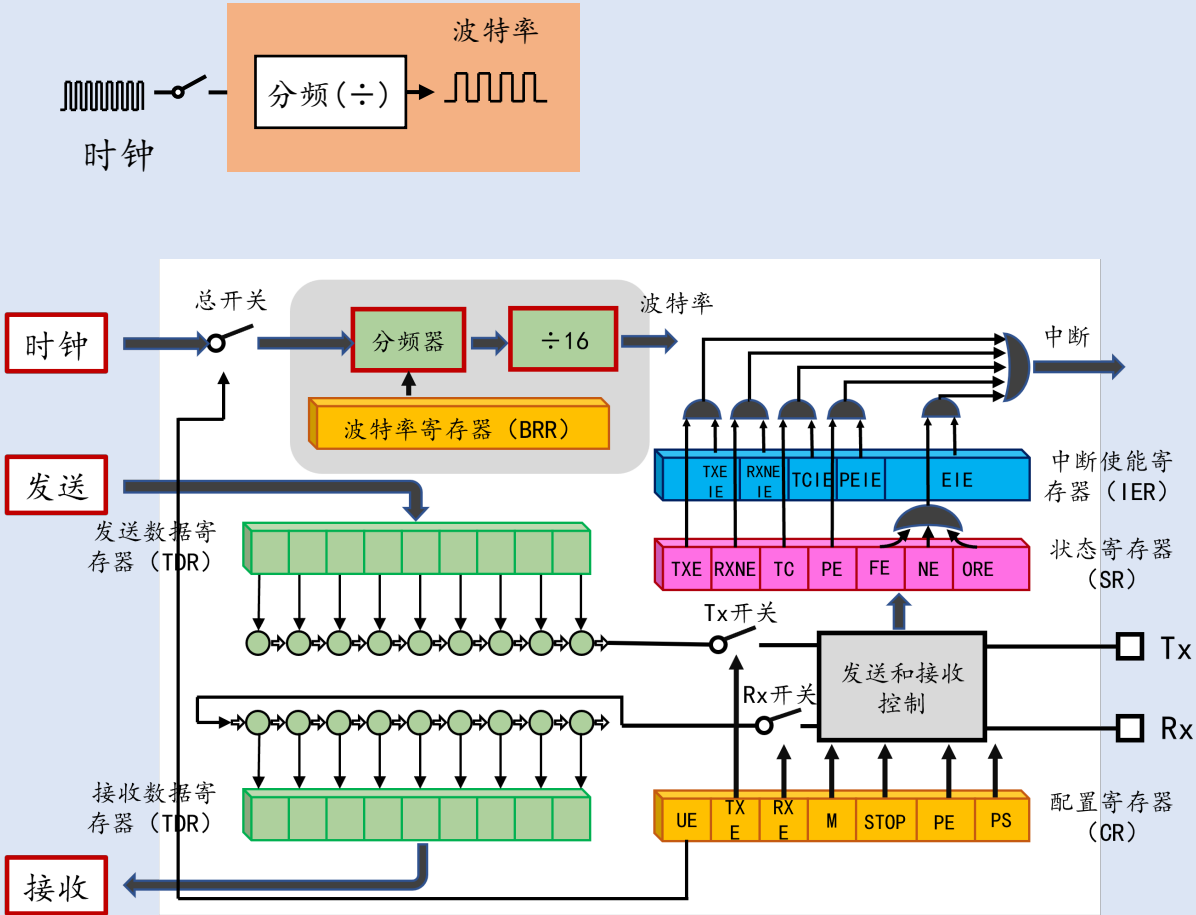


```
RCC_APB2PeriphClockCmd(..., ...); // 开启/关闭片上外设的时钟
```



2.3.2.3.3. 具体电路

USART

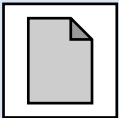


Mantissa整数部分											Fraction小数部分				
M11	M10	M9	M8	M7	M6	M5	M4	M3	M2	M1	M0	F3	F2	F1	F0
2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}
2048	1024	512	256	128	64	32	16	8	4	2	1	0.5	0.25	0.125	0.0625

$$\text{波特率} = \frac{\text{RCC时钟}}{\text{分频系数} * 16}$$

$$\text{分频系数} = \frac{\text{RCC时钟}}{\text{波特率} * 16}$$

习题:

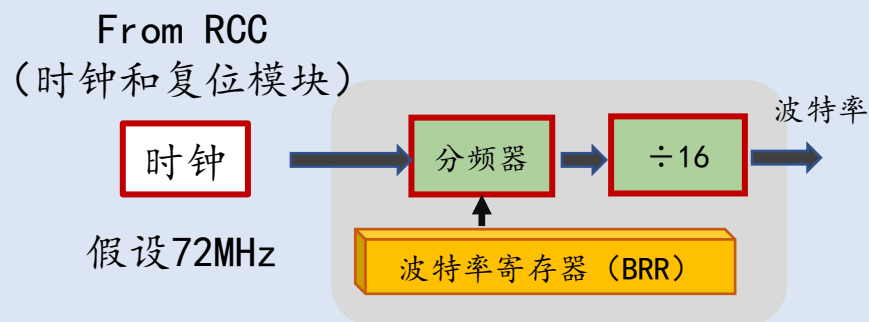


习题



要求:

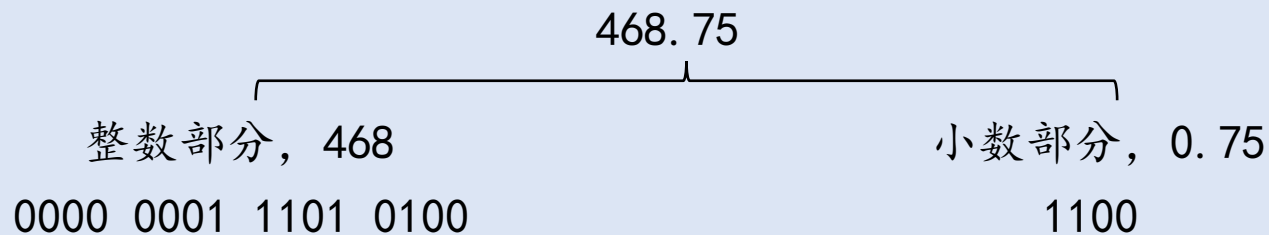
在输入时钟是72MHz的情况下产生9600波特率



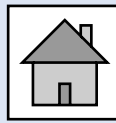
$$\text{分频系数} = \frac{72000000}{9600 * 16} = 468.75$$



$$\text{波特率} = \frac{RCC\text{时钟}}{\text{分频系数} * 16} \quad \text{分频系数} = \frac{RCC\text{时钟}}{\text{波特率} * 16}$$



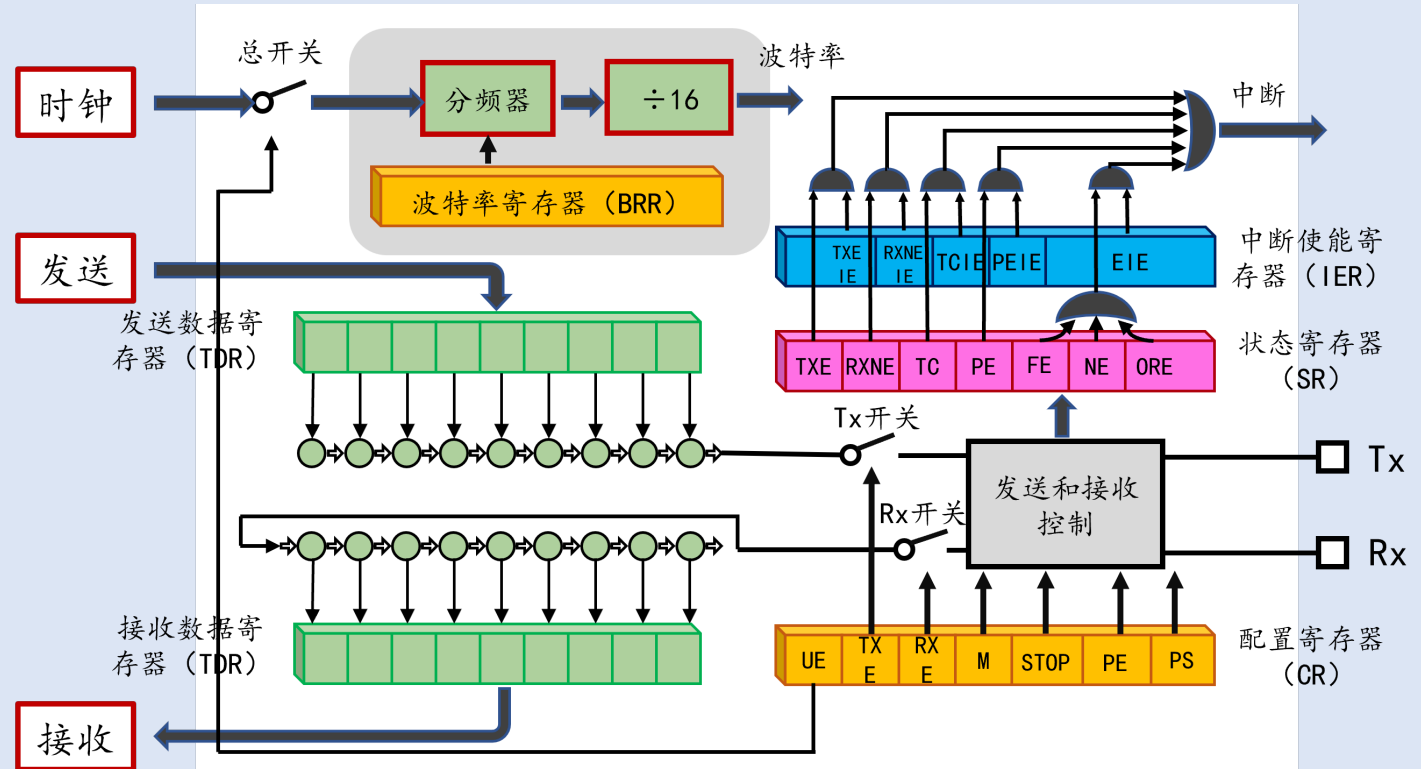
2.3.2.4. USART的总开关



UE - (USART Enable) USART使能

UE=0, USART禁止

UE=1, USART使能



2.3.2.5. 练习

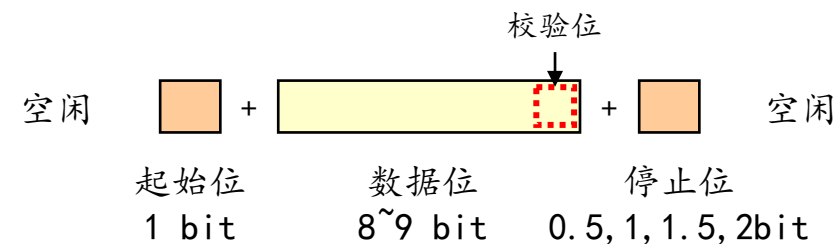
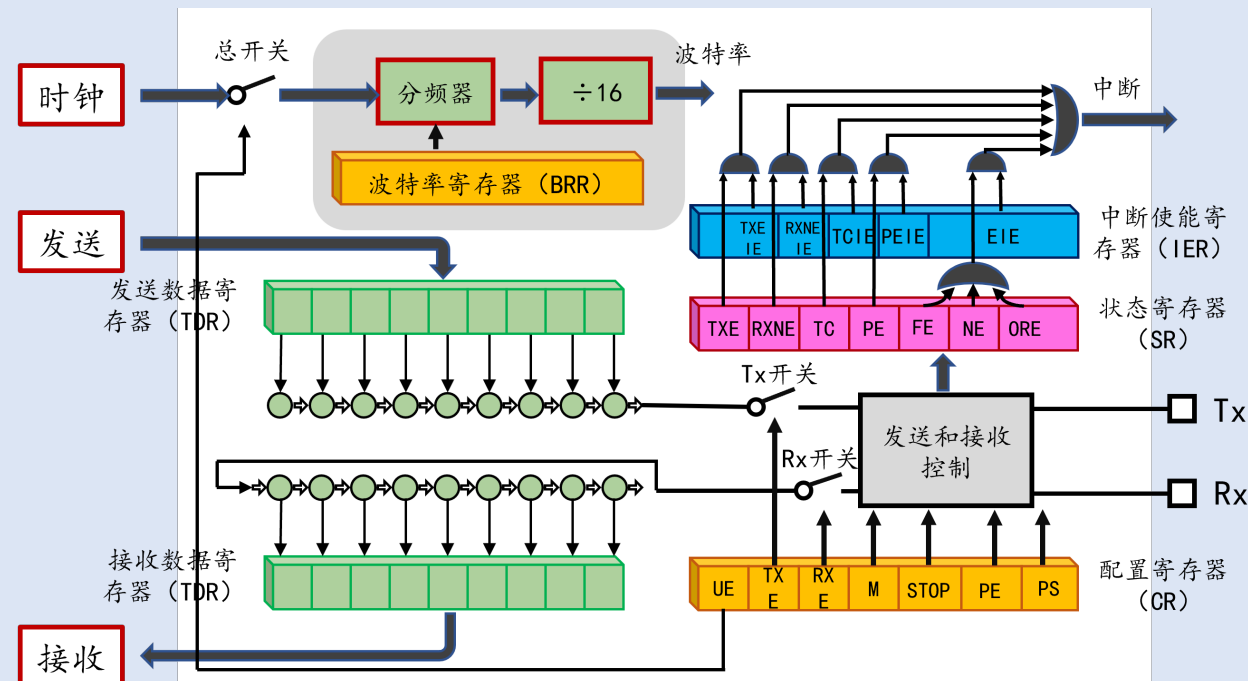


要求:

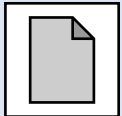
通过串口发送数据, 每次发送一个字节, 使用偶校验, 1位停止位, 波特率115200

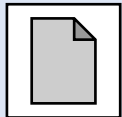


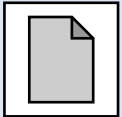
$$\text{分频系数} = \frac{\text{RCC时钟}}{\text{波特率} * 16}$$

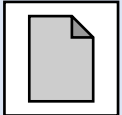


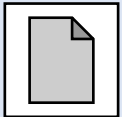
2.2.4. 数据发送过程

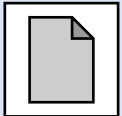
2.2.4.1. 双缓冲与连续发送 

2.2.4.2. 数据发送过程中的问题 

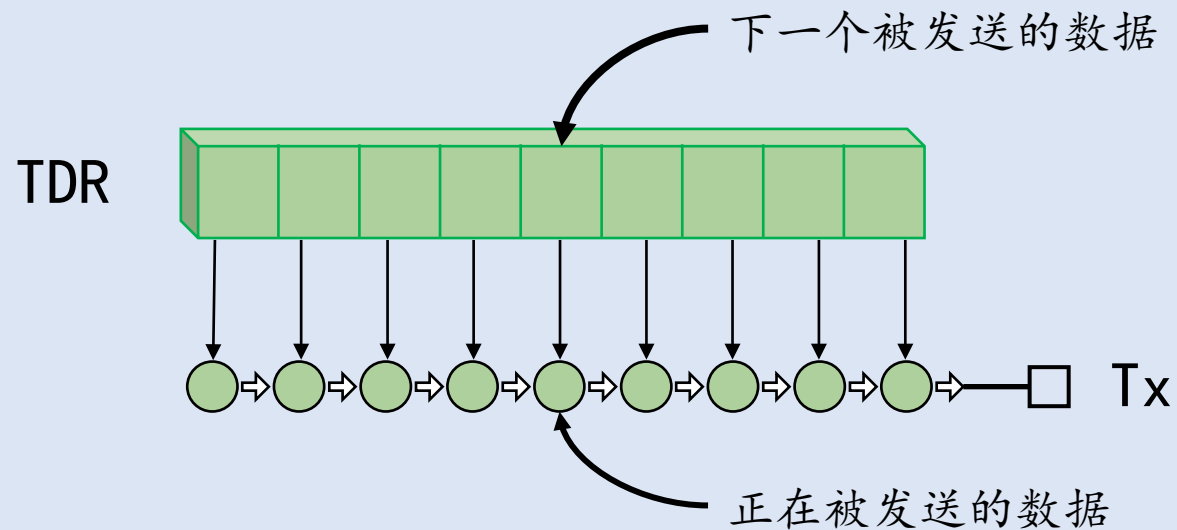
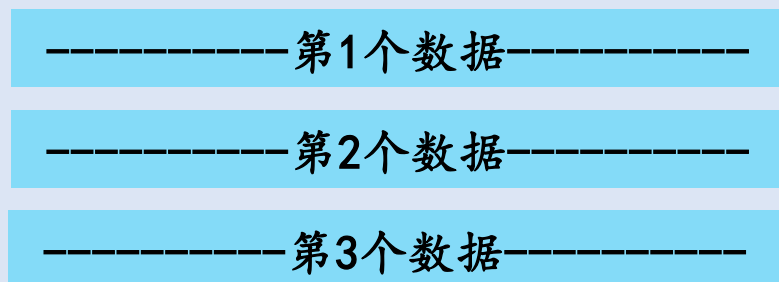
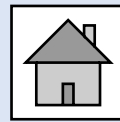
2.2.4.3. TXE标志位 

2.2.4.4. TC标志位 

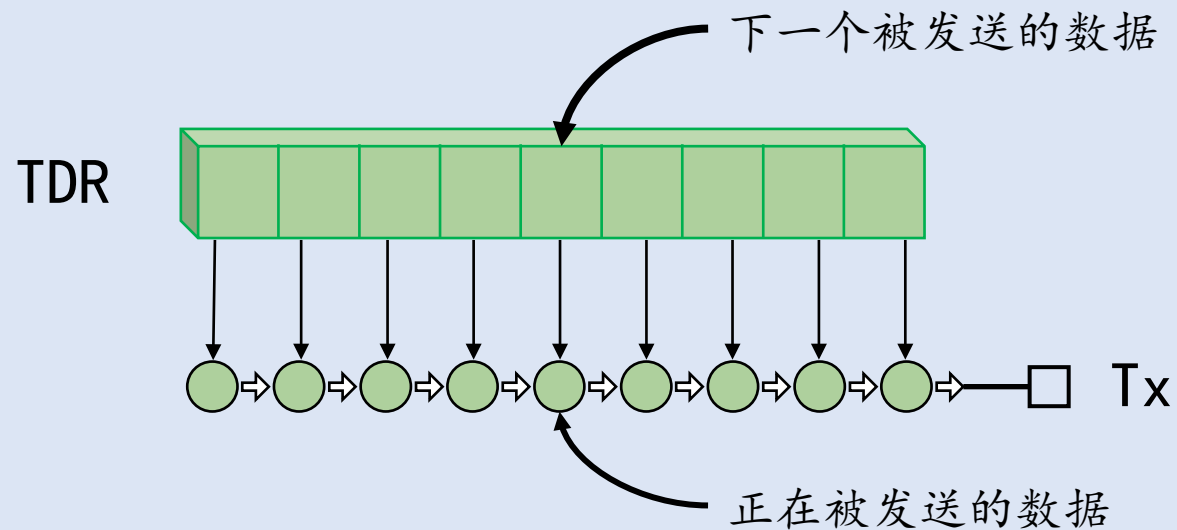
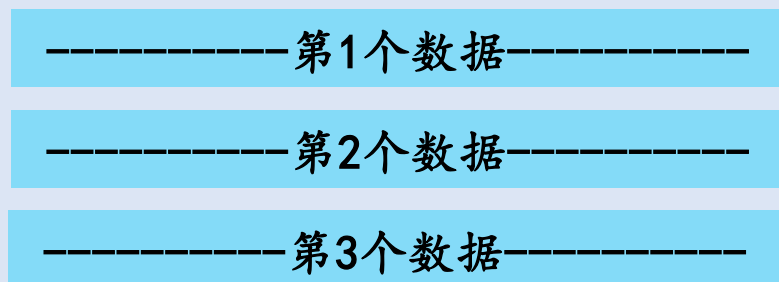
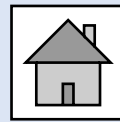
2.2.4.5. 单个数据的发送 

2.2.4.6. 数据的连续发送 

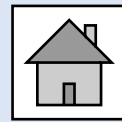
2.2.4.1. 双缓冲与连续发送



2.2.4.1. 双缓冲与连续发送



2.2.4.2. 数据发送过程中的问题

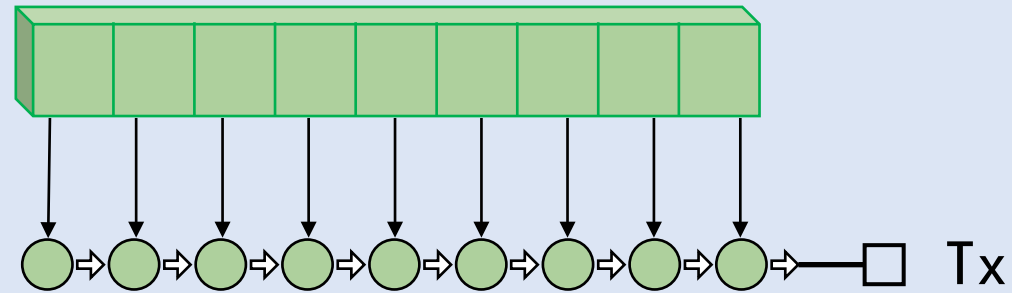


问题1：数据发送过快导致的覆盖问题

问题2：数据什么时候发送完成的问题



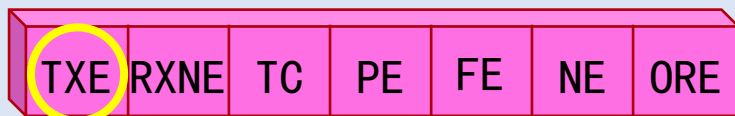
TDR



2.2.4.3. TXE标志位



x



TXE (Transmit Data Register Empty)

发送数据寄存器空

-----第1个数据-----

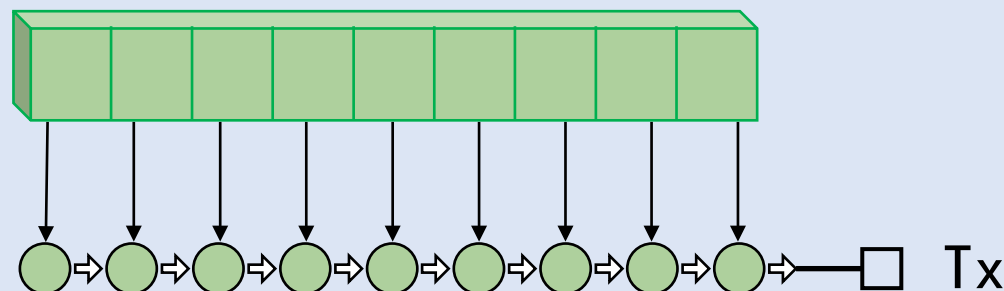
-----第2个数据-----

-----第3个数据-----

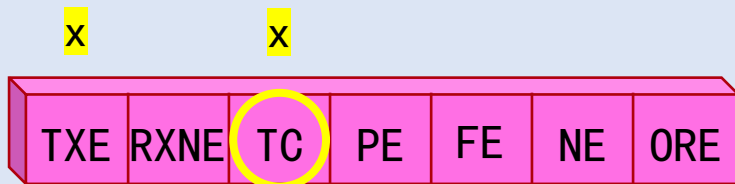
```
while (TXE == 0); // 等待发送数据寄存器空
```

```
TDR = xxx; // 将要发送的数据写入TDR
```

TDR



2.2.4.4. TC标志位



TC(Transmit Complete) 发送完成
TDR空&&移位寄存器空

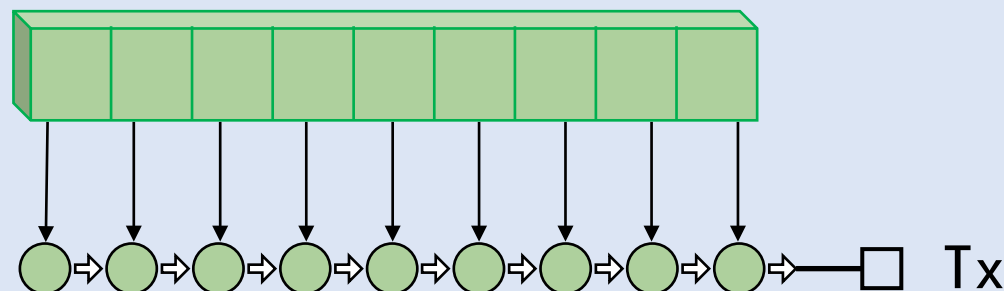
-----第1个数据-----

-----第2个数据-----

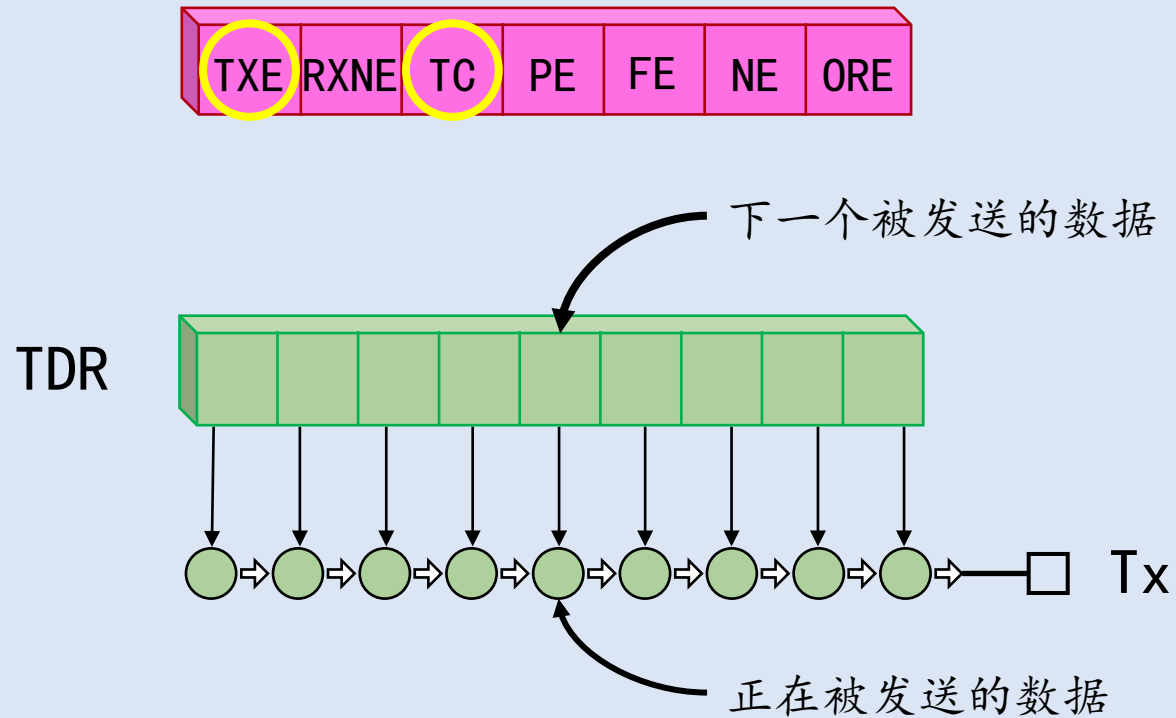
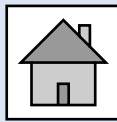
-----第3个数据-----

```
while(TXE==0); // 等待发送数据寄存器空  
TDR = xxx; // 将要发送的数据写入TDR  
while(TC==0); //等待数据发送完成
```

TDR

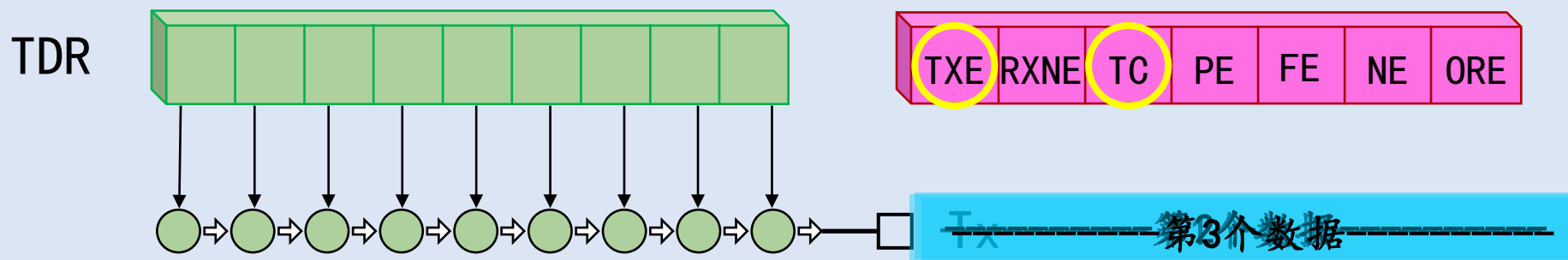


2.2.4.5. 单个数据的发送



```
// 1. 等待TDR清空  
while(TXE == 0);  
  
// 2. 写入要发送的数据  
TDR = 0x5a;  
  
// 3. 等待数据发送完成  
while(TC == 0);
```

2.2.4.6. 数据的连续发送



```
for (i=0; i<n; i++) {
```

```
    // 1. 等待TDR清空
```

```
    while(TXE == 0);
```

```
    // 2. 写入要发送的数据
```

```
    TDR = 0x5a;
```

```
    // 3. 等待数据发送完成
```

```
    while(TC == 0);
```

```
}
```



```
for (i=0; i<n; i++) {
```

```
    // 1. 等待TDR清空
```

```
    while(TXE == 0);
```

```
    // 2. 写入要发送的数据
```

```
    TDR = 0x5a;
```

```
}
```

```
    // 等待数据发送完成
```

```
    while(TC == 0);
```

2.2.5. 数据接收过程

2.2.5.1. 数据接收过程中的问题

2.2.5.2. RXNE标志位

2.2.5.3. 接收单个数据

2.2.5.4. 连续接收多个数据

2.2.5.1. 数据接收过程中的问题

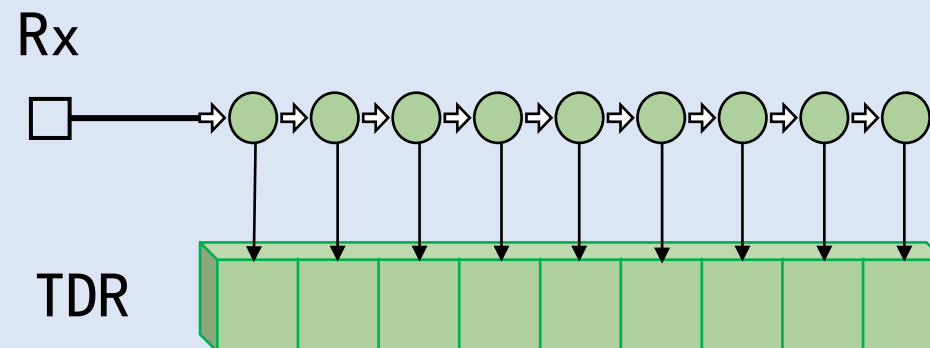


我们不知道什么时候去读取数据？

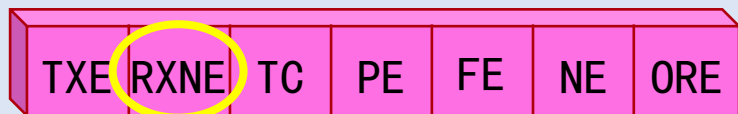
-----第1个数据-----

-----第2个数据-----

-----第3个数据-----



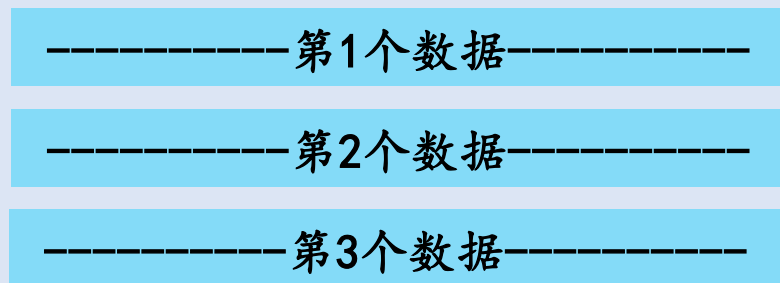
2.2.5.2. RXNE标志位



RXNE (Receive data register Not Empty)

接收数据寄存器非空

RXNE=0, 无数据; RXNE=1, 有数据



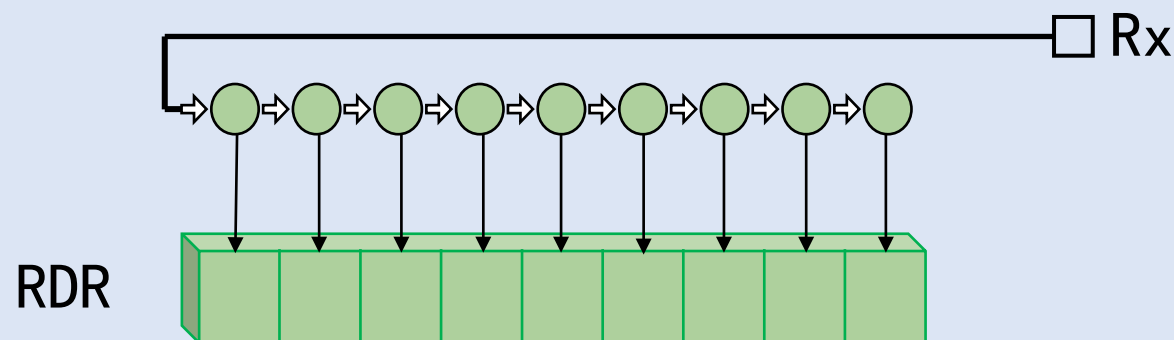
```
uint8_t a;
```

```
// 1. 等待RDR有数据需要读取
```

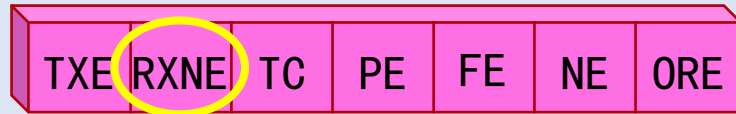
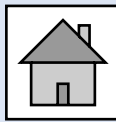
```
while (RXNE==0);
```

```
// 2. 立即把数据读取出来
```

```
a = RDR;
```



2.2.5.3. 接收单个数据



RXNE (Receive data register Not Empty)

接收数据寄存器非空

RXNE=0, 无数据; RXNE=1, 有数据

```
uint8_t a;
```

```
// 1. 等待RDR有数据需要读取
```

```
while (RXNE==0);
```

```
// 2. 立即把数据读取出来
```

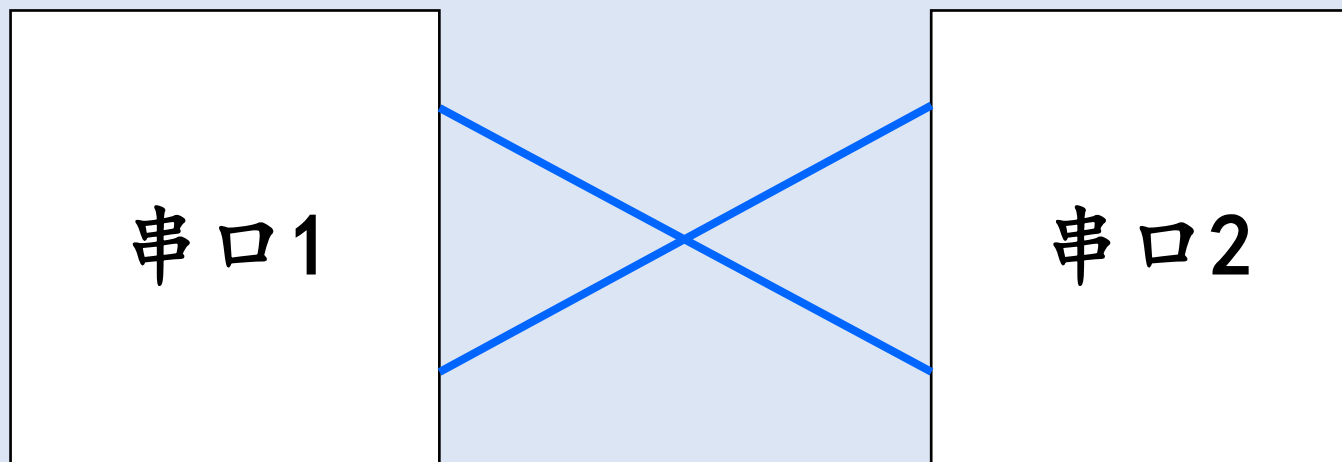
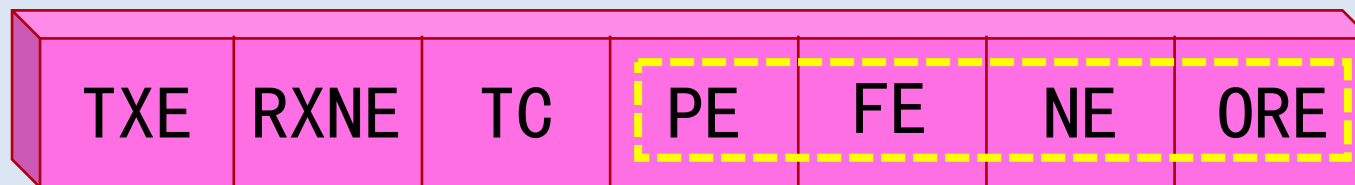
```
a = RDR;
```

2.2.5.4. 连续接收多个数据

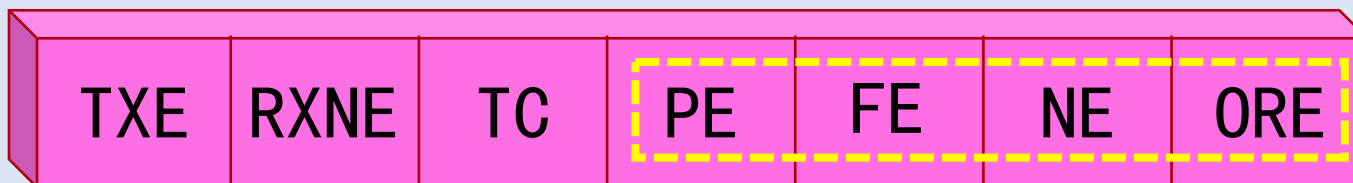


```
uint8_t a[10];  
  
for (i=0; i<10; i++) {  
    // 1. 等待RDR有数据需要读取  
    while (RXNE==0);  
    // 2. 立即把数据读取出来  
    a[i] = RDR;  
}
```

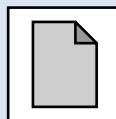
错误标志位



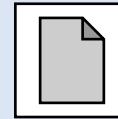
2.3. 错误标志位



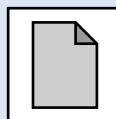
2.3.1. PE奇偶校验错



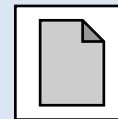
2.3.2. FE帧格式错



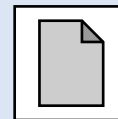
2.3.3. NE噪声错



2.3.4. ORE过载错

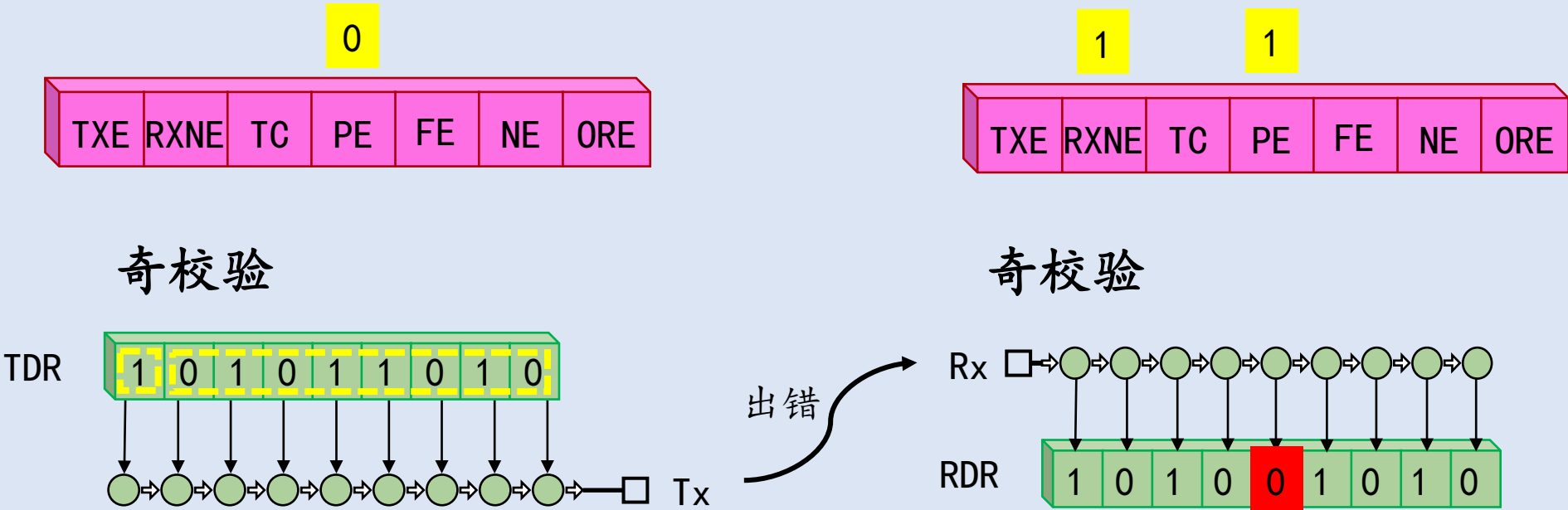


2.3.5. 错误标志位的使用方法



2.3.1. PE奇偶校验错

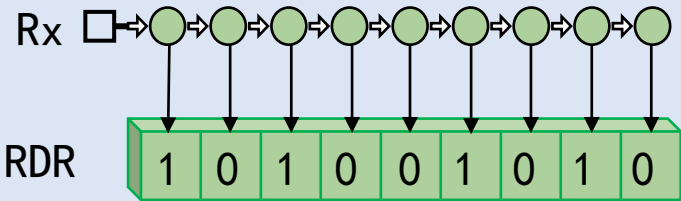
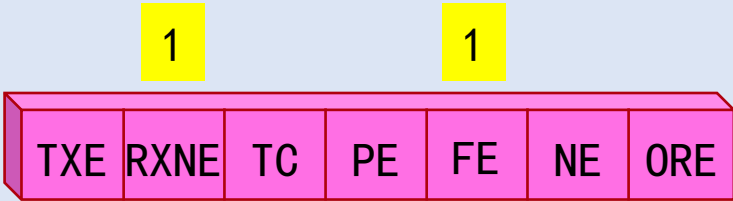
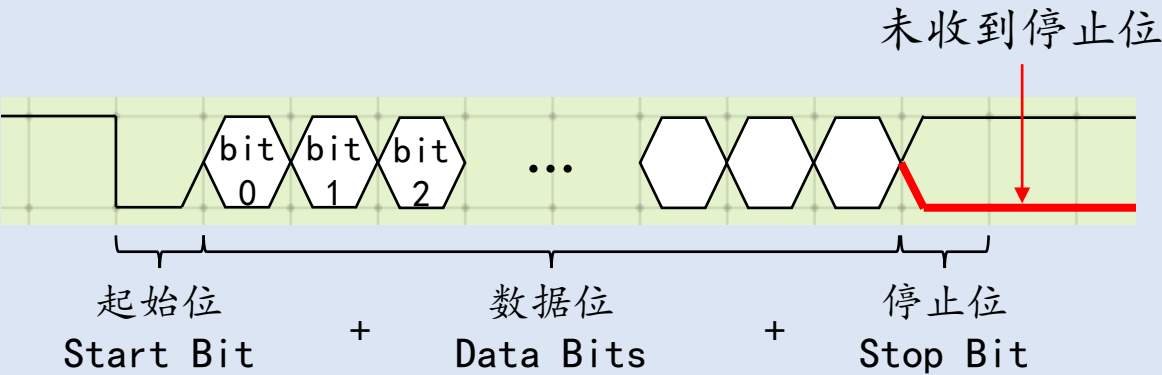
PE (Parity Error) 奇偶校验错



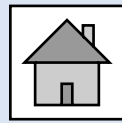
2.3.2. FE帧格式错



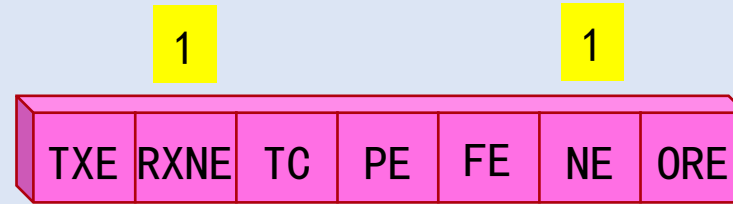
FE (Frame Error) 帧格式错



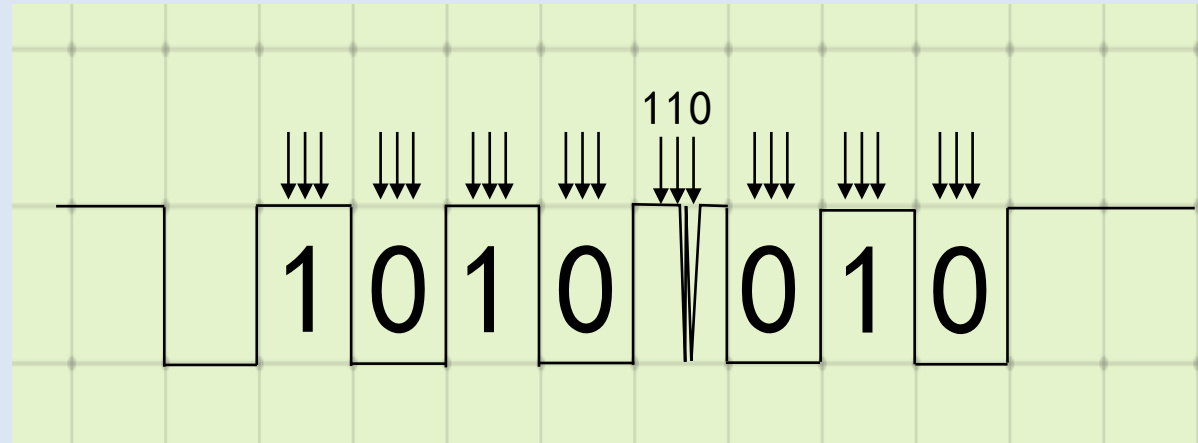
2.3.3. NE噪声错



NE (Noise Error) 噪声错

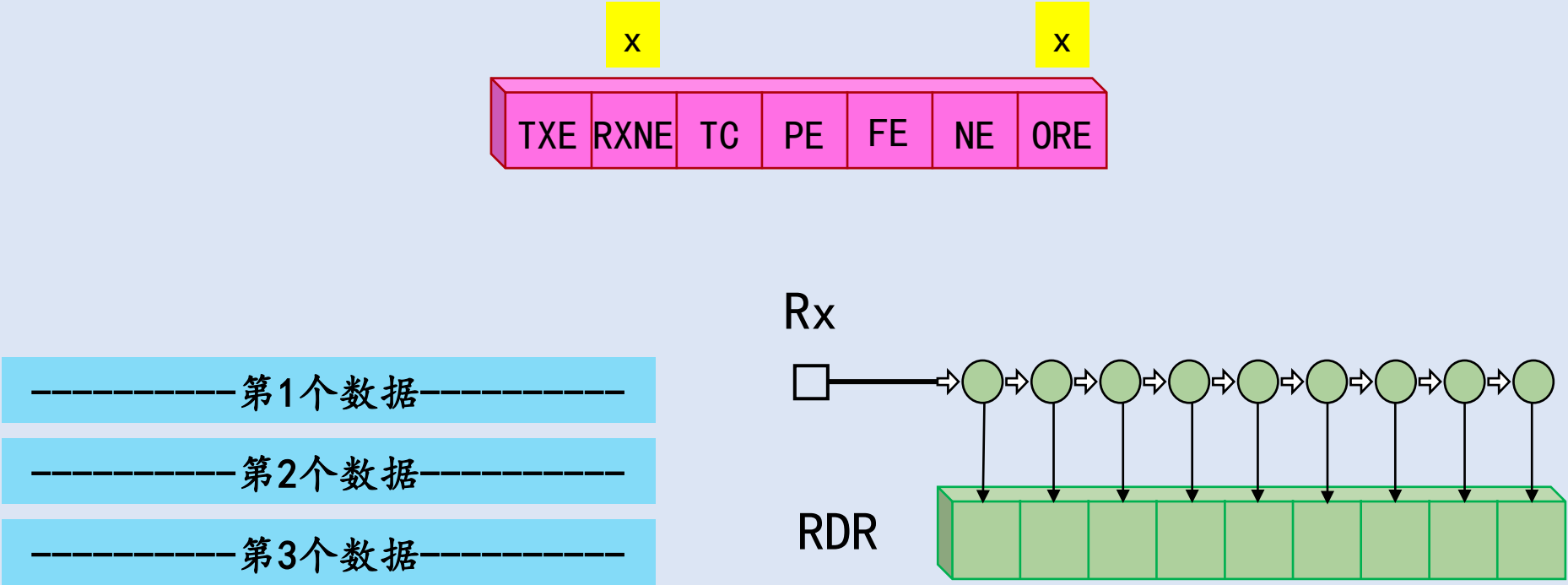


过采样

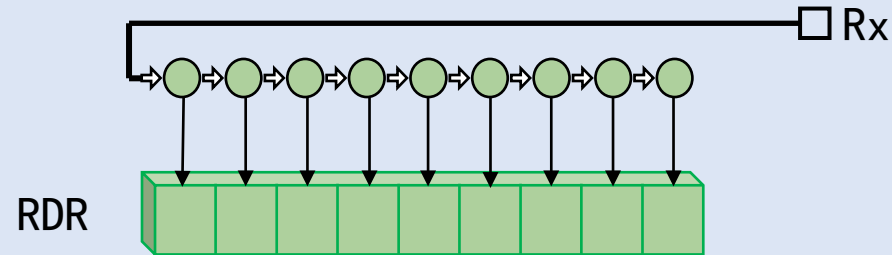
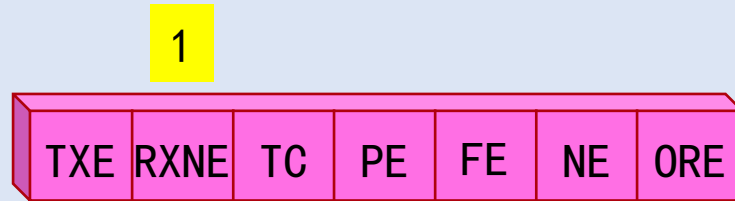
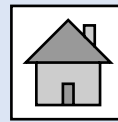


2. 3. 4. ORE过载错

ORE (Overrun Error) 过载错



2.3.5. 错误标志位的使用方法

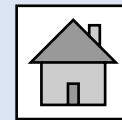


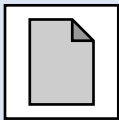
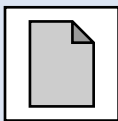
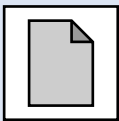
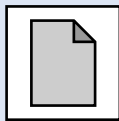
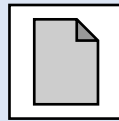
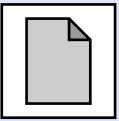
```
uint8_t a;  
// 1. 等待RDR有数据需要读取  
while (RXNE==0);  
// 2. 先备份SR寄存器  
uint32_t srcpy = SR;  
// 3. 把数据读取出来  
a = USARTx->RDR;
```

```
// 4. 处理错误  
if(srcpy & PE != 0) {} //校验错  
if(srcpy & FE != 0) {} //帧格式错  
if(srcpy & NE != 0) {} //噪声错  
if(srcpy & ORE != 0) {} //过载错
```

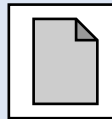

题目7：请写出连续发送多个数据的伪代码

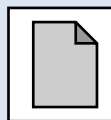
第三节、USART的标准库编程

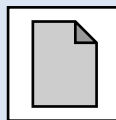


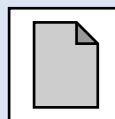
- 3. 1. 实验介绍 
- 3. 2. 标准库编程接口 
- 3. 3. USART的初始化 
- 3. 4. 使用串口发送数据 
- 3. 5. 使用串口接收数据 
- 3. 6. 随堂测试 

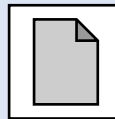
3.1. 实验介绍

3.1.1. 使用USART与计算机通信 

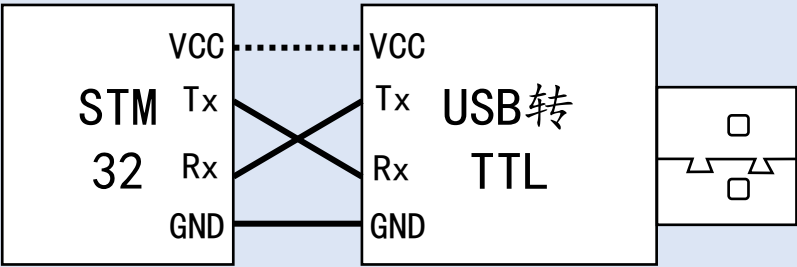
3.1.2. 定位USART的引脚 

3.1.3. 完整接线图 

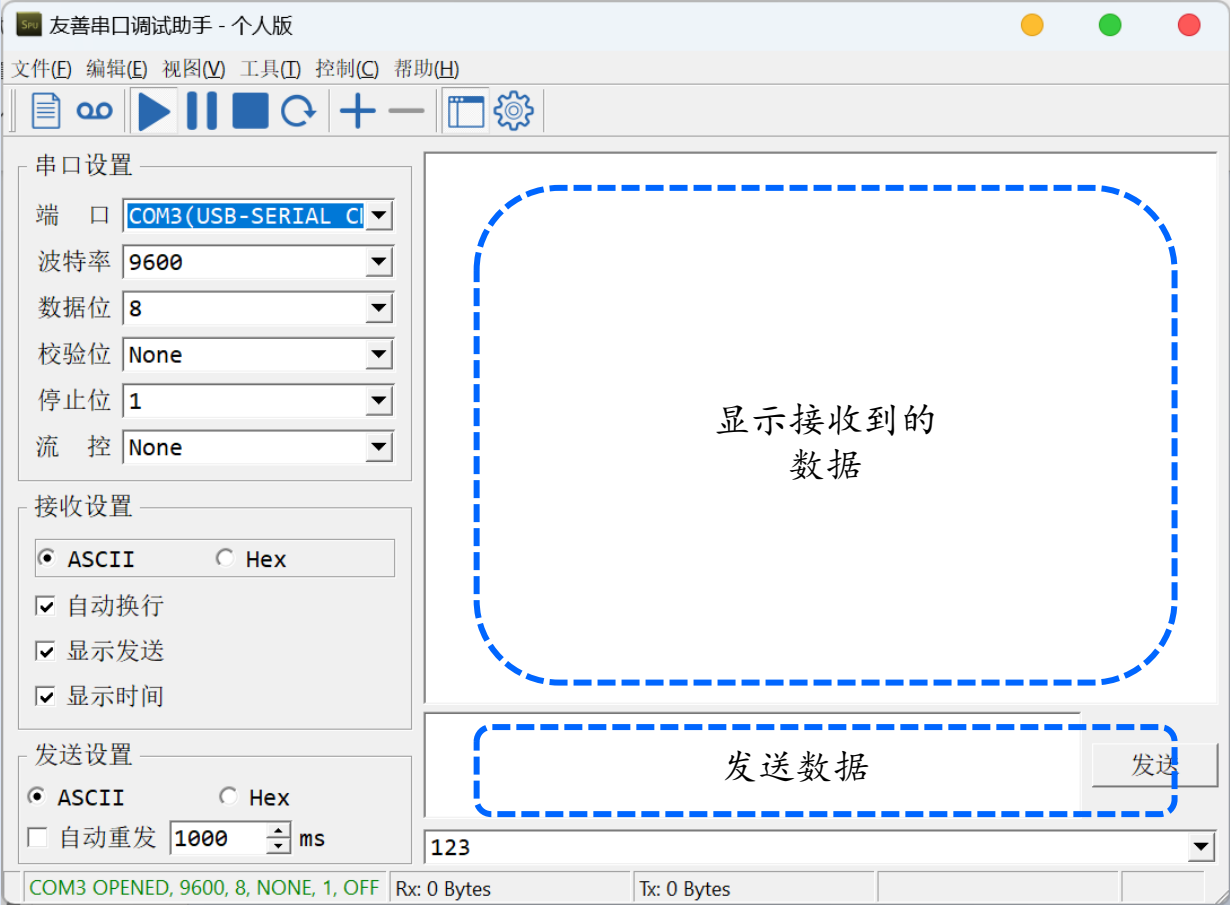
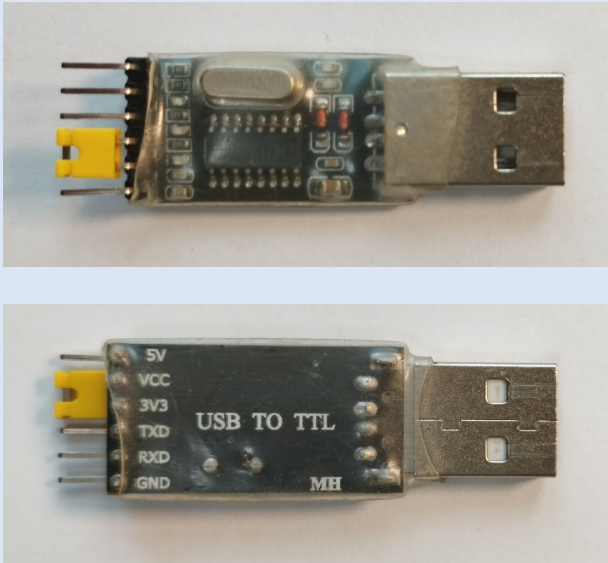
3.1.4. 安装USB转TTL驱动 

3.1.5. 安装串口调试助手 

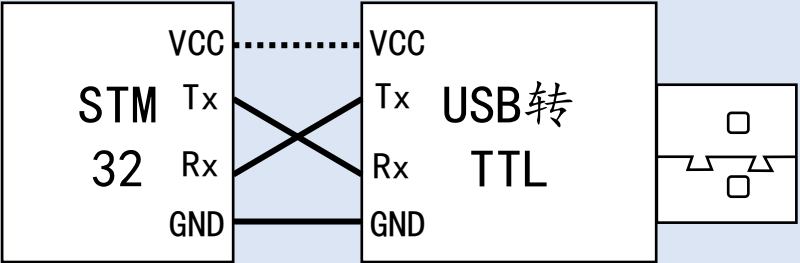
3.1.1. 使用USART与计算机通信



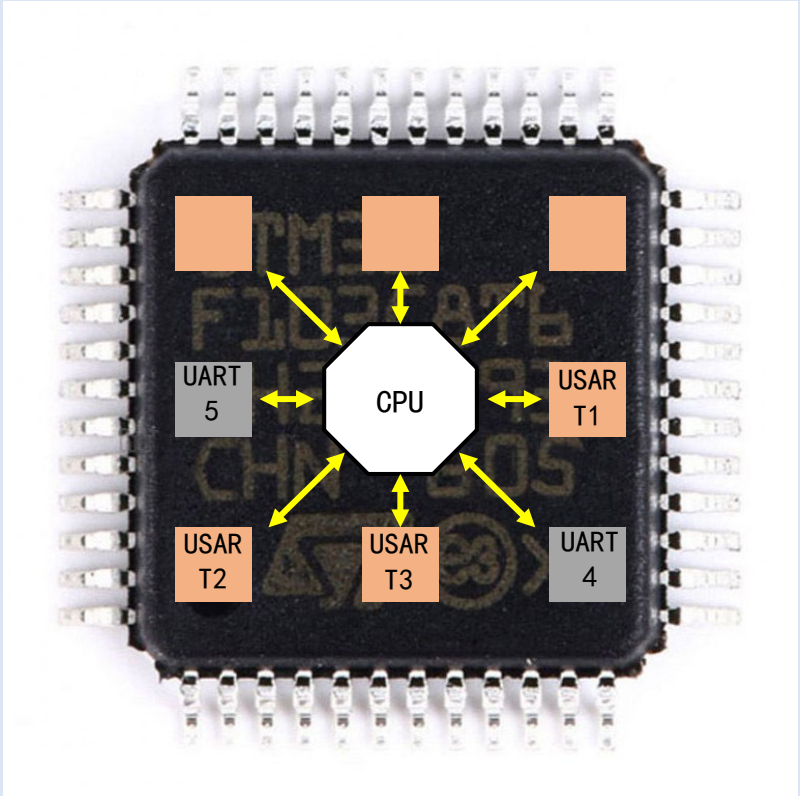
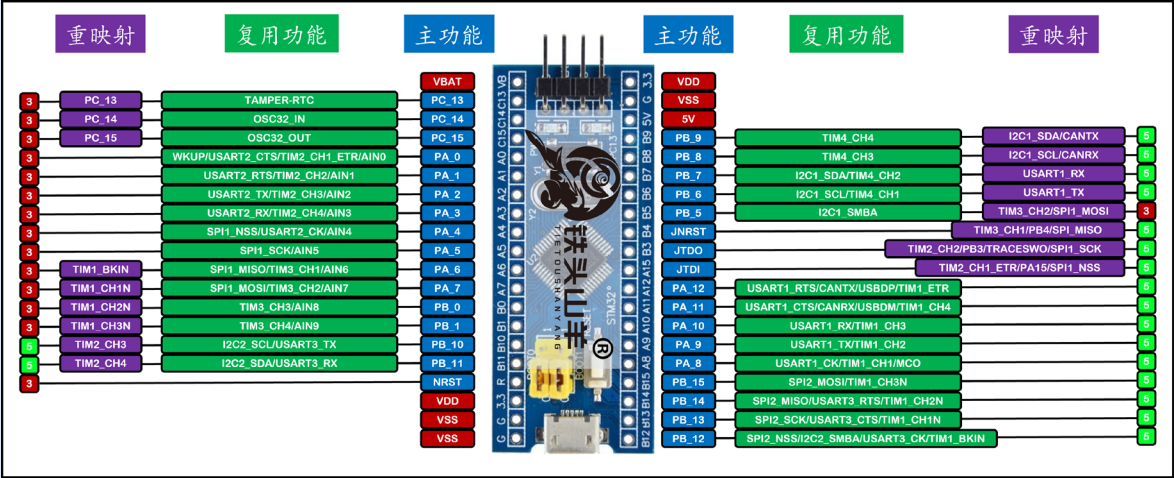
USB转TTL模块



3.1.2. 定位USART的引脚



方式1：查找“最小系统板引脚分布图”

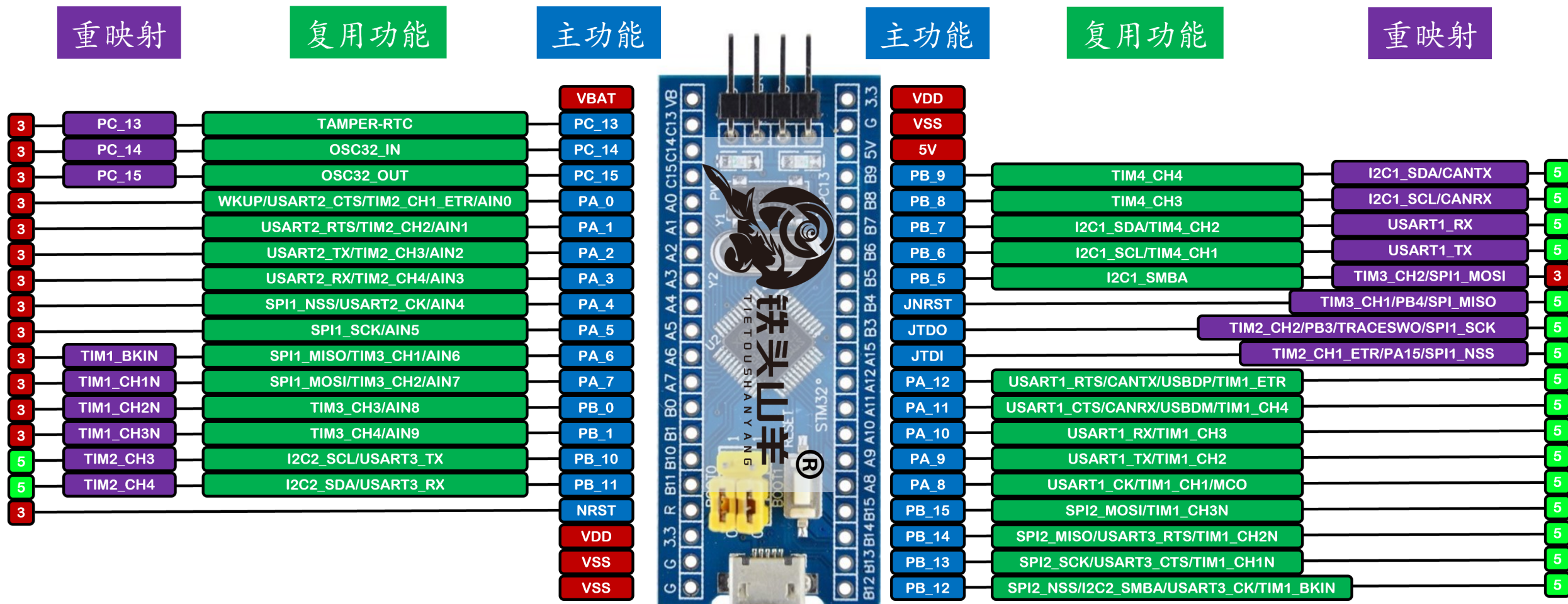


方式2：查找“复用功能重映射表”

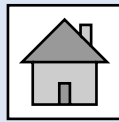
表47 USART1重映像

复用功能	USART1_REMAP = 0	USART1_REMAP = 1
USART1_TX	PA9	PB6
USART1_RX	PA10	PB7

最小系统板引脚分布图

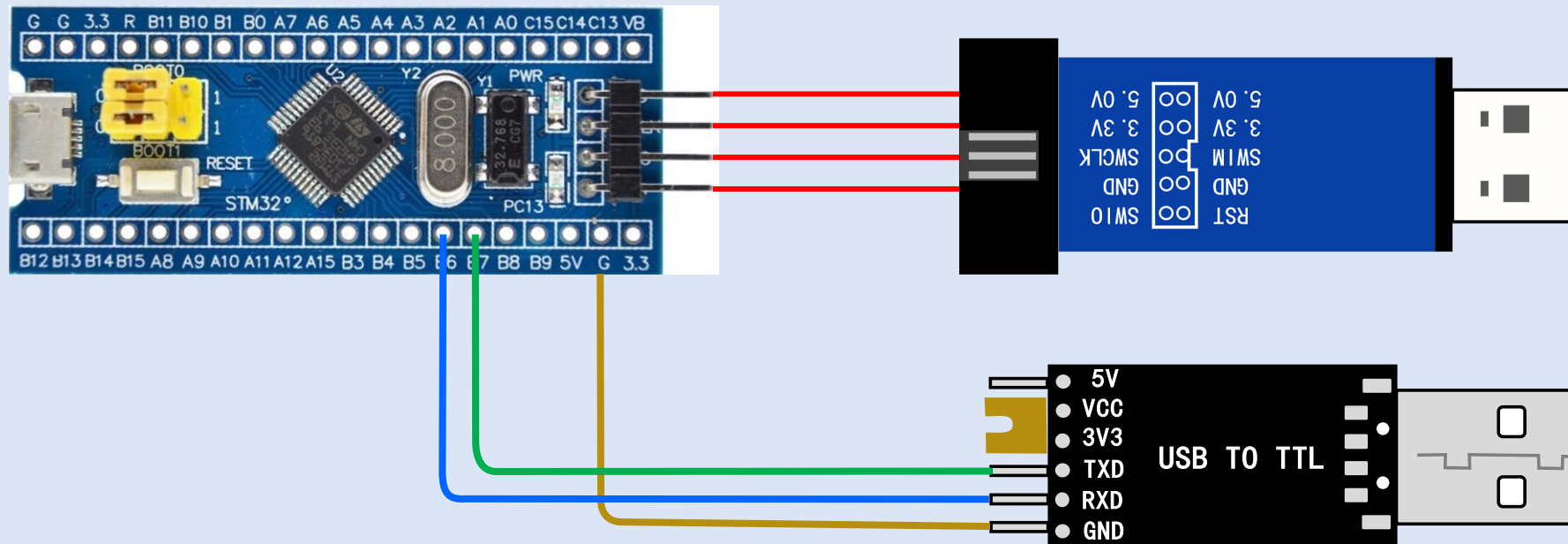
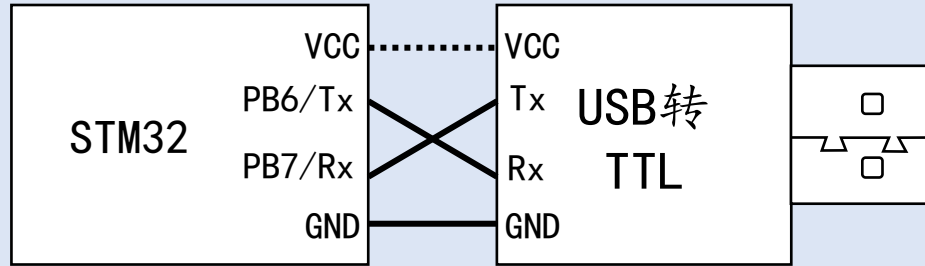


3.1.3. 完整接线图

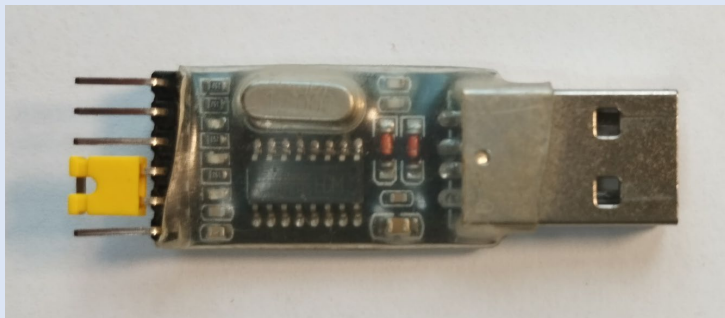


实验材料

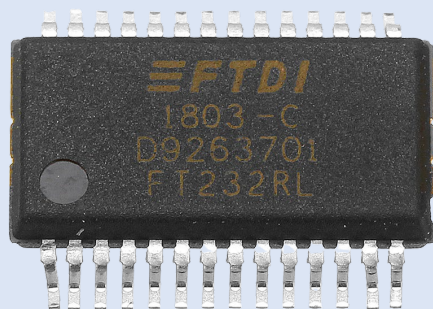
1. STM32最小系统板
2. ST-Link调试器
3. USB转TTL模块
4. 母对母杜邦线 * 7



3.1.4. 安装USB转TTL驱动



FT232RL

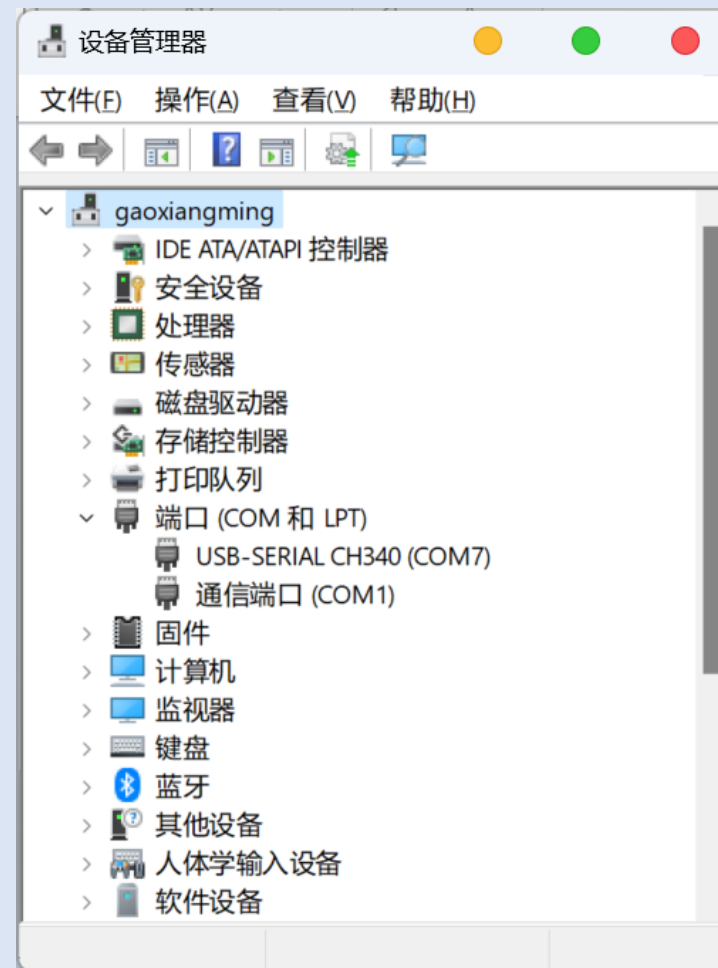


 CDM212364_Setup.exe

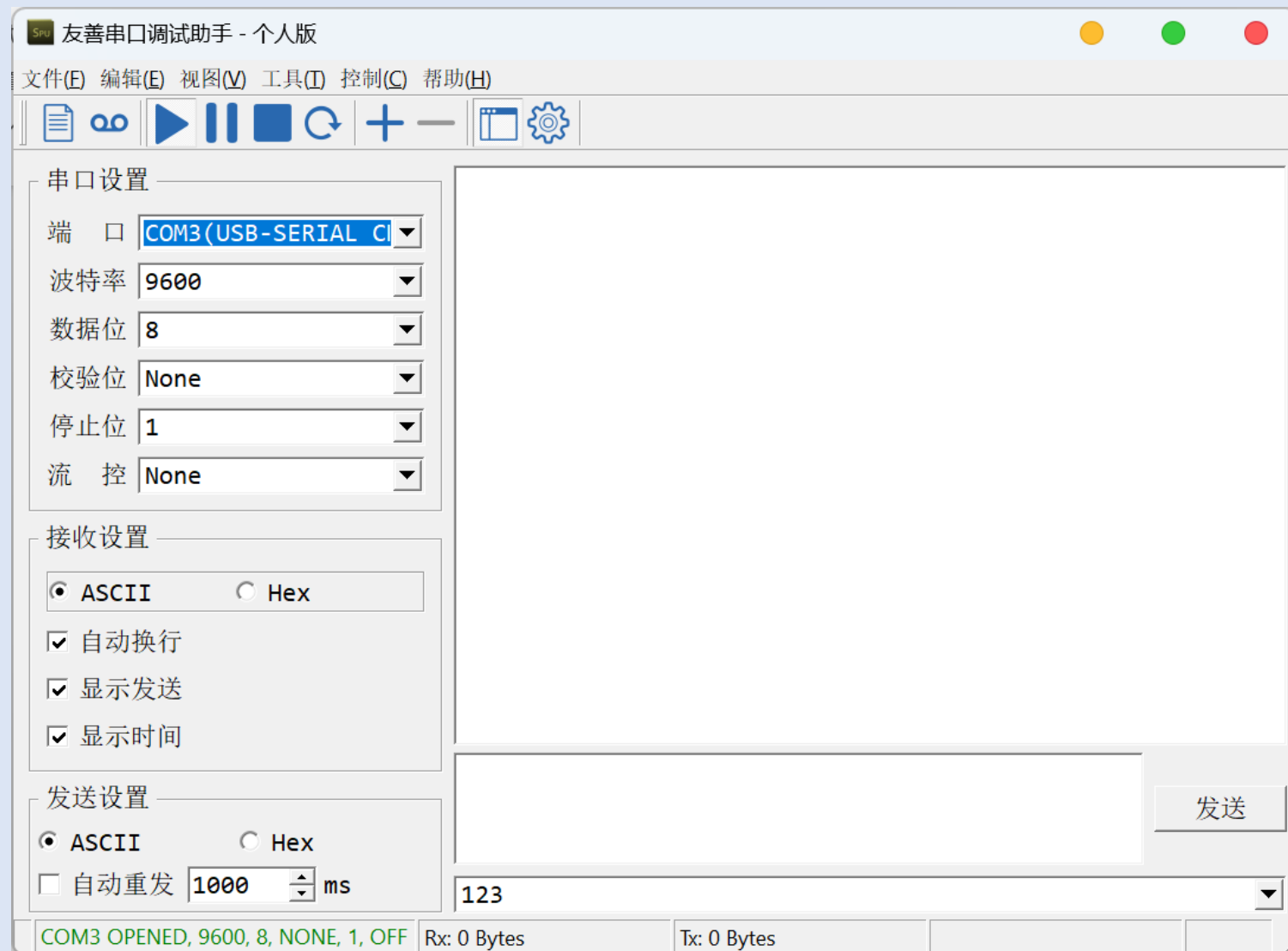
CH340



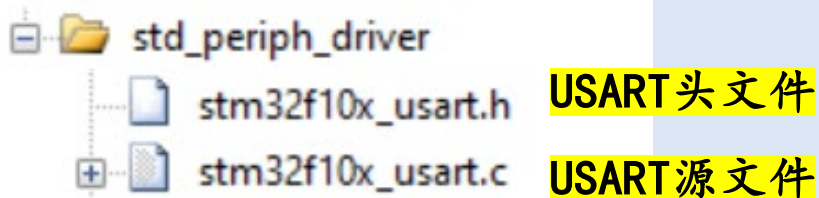
 CH341SER.EXE



3.1.5. 安装串口调试助手



3.2. 标准库编程接口



`void USART_Init(...);` // 初始化(帧格式、波特率)



`void USART_Cmd(...);` // 总开关



`void USART_SendData(...);` // 写TDR寄存器



`uint16_t USART_ReceiveData(...);` // 读RDR寄存器



`FlagStatus USART_GetFlagStatus(...);` // 读标志位

USART_Init



编程接口说明

// @简介: 串口初始化

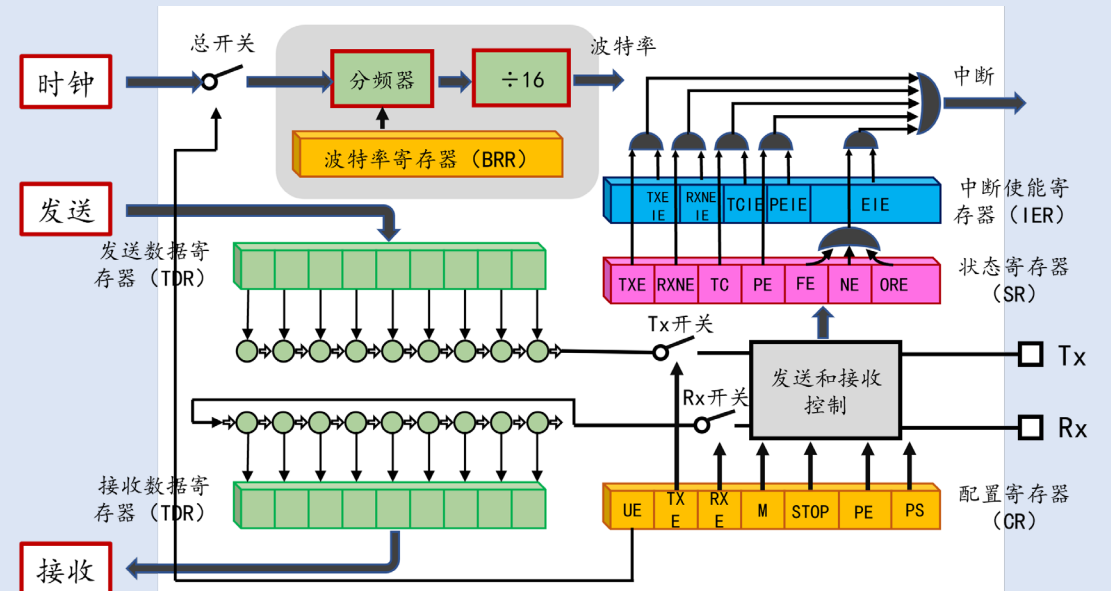
// @参数: USARTx - 选择要操作的串口, 可以是USART1, USART2, USART3, UART4或USART5

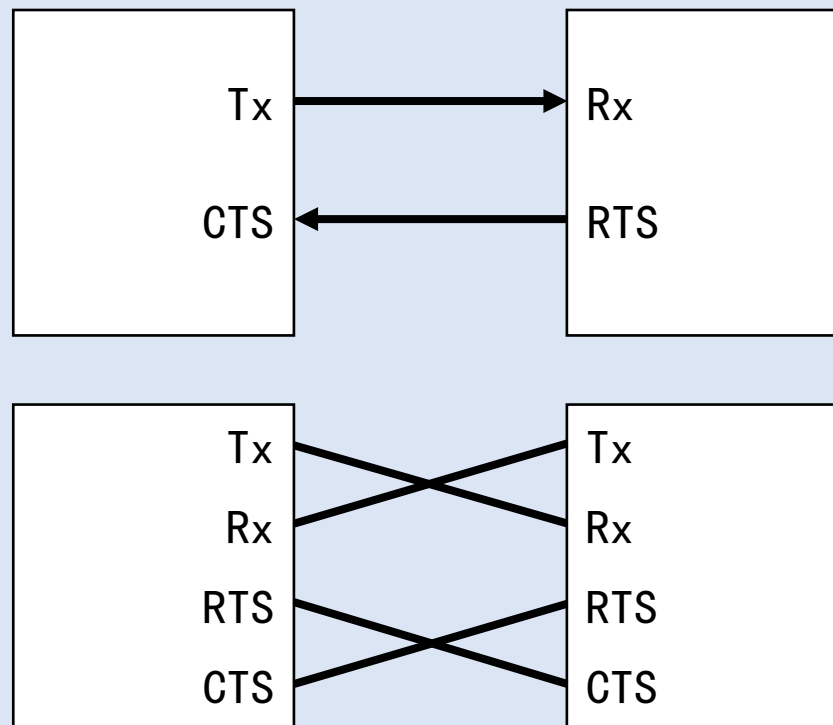
// @参数: USART_InitStruct - 初始化参数, USART_InitTypeDef结构体指针类型

void USART_Init(USART_TypeDef* USARTx, USART_InitTypeDef* USART_InitStruct)

示例代码

```
USART_InitTypeDef USARTInitStruct;  
USARTInitStruct.USART_WordLength = xxx;  
USARTInitStruct.USART_Parity=xxx;  
USARTInitStruct.USART_StopBits=xxx;  
USARTInitStruct.USART_Mode=xxx;  
USARTInitStruct.USART_BaudRate=xxx;  
USARTInitStruct.USART_HardwareFlowControl=xxx;  
USART_Init(USARTx, &USARTInitStruct);
```





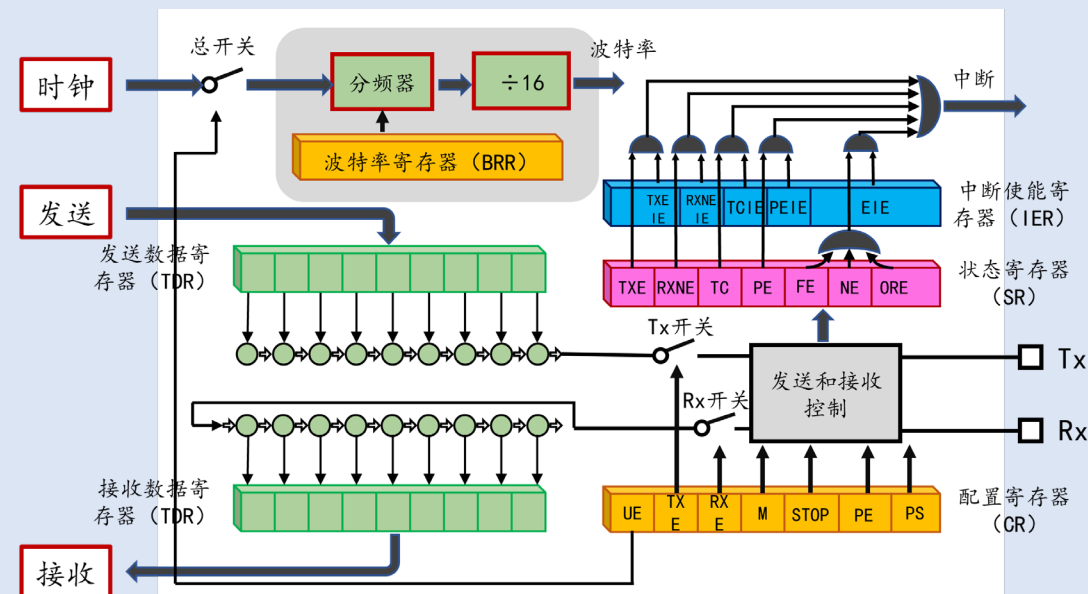


编程接口说明

```
// @简介: 控制USART总开关的断开和闭合
// @参数: USARTx - 选择要操作的串口, 可以是USART1, USART2, USART3, UART4或USART5
// @参数: NewState - 开关状态, ENABLE - 闭合, DISABLE - 断开
void USART_Cmd(USART_TypeDef* USARTx, FunctionalState NewState)
```

示例代码

```
USART_Cmd(USART1, ENABLE); // 闭合USART1开关
USART_Cmd(USART1, DISABLE); // 断开USART1开关
```



USART_SendData



编程接口说明

// @简介: 向TDR写数据

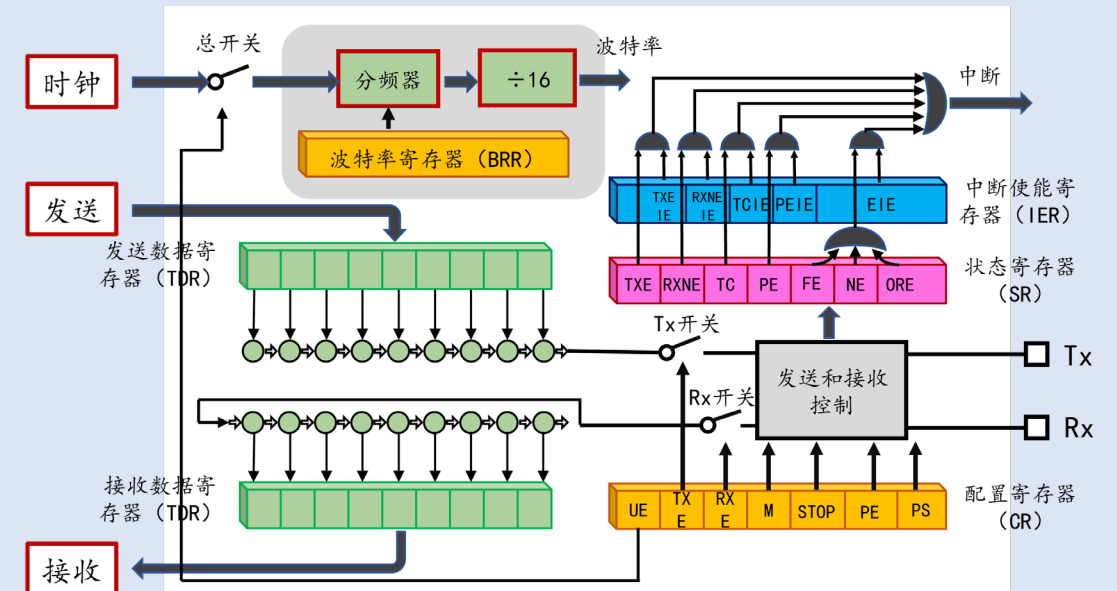
// @参数: USARTx - 选择要操作的串口, 可以是USART1, USART2, USART3, UART4或USART5

// @参数: Data - 要写入的数据

```
void USART_SendData(USART_TypeDef* USARTx, uint16_t Data)
```

示例代码

```
USART_SendData(USART1, 0x5a); // 发送0x5a
```



USART_ReceiveData



编程接口说明

// @简介: 从RDR读取数据

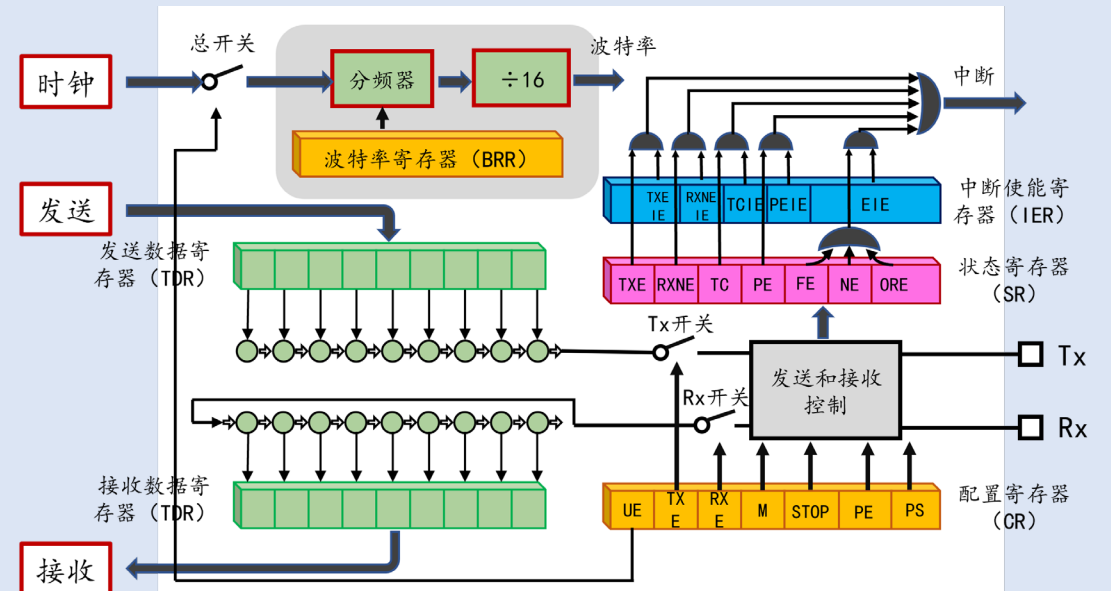
// @参数: USARTx - 选择要操作的串口, 可以是USART1, USART2, USART3, UART4或USART5

// @返回值: 读取到的数据

uint16_t USART_ReceiveData(USART_TypeDef* USARTx)

示例代码

```
uint8_t a = USART_ReceiveData(USART1);
```



USART_GetFlagStatus

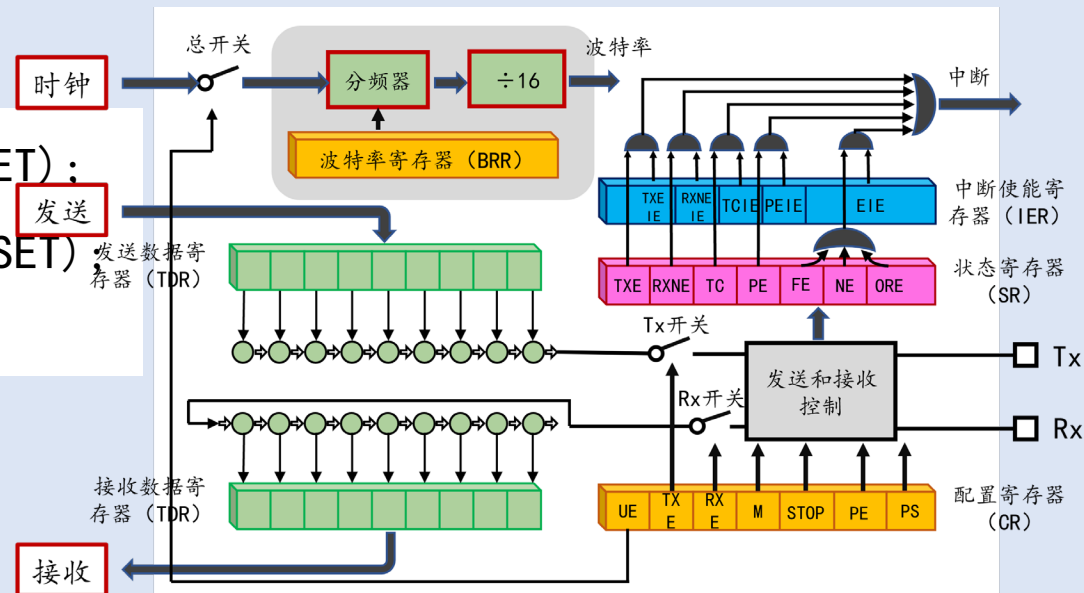


编程接口说明

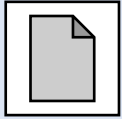
```
// @简介: 向TDR写数据
// @参数: USARTx - 选择要操作的串口, 可以是USART1, USART2, USART3, UART4或USART5
// @参数: USART_Flag - 标志位名称。
//          可以是USART_FLAG_TXE, USART_FLAG_RXNE, USART_FLAG_ORE, USART_FLAG_TC,
//          USART_FLAG_PE, USART_FLAG_FE, USART_FLAG_NE
// @返回值: 标志位状态, SET - 1, RESET - 0
FlagStatus USART_GetFlagStatus(USART_TypeDef* USARTx, uint16_t USART_Flag)
```

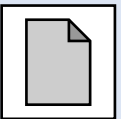
示例代码

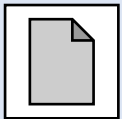
```
while(USART_GetFlagStatus(USART1, USART_FLAG_TXE) == RESET) :
while(USART_GetFlagStatus(USART1, USART_FLAG_RXNE) == RESET)
if(USART_GetFlagStatus(USART1, USART_FLAG_PE) == SET) {}
```

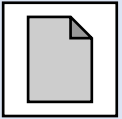


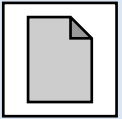
3.3. USART的初始化

3.3.1. USART初始化流程介绍 

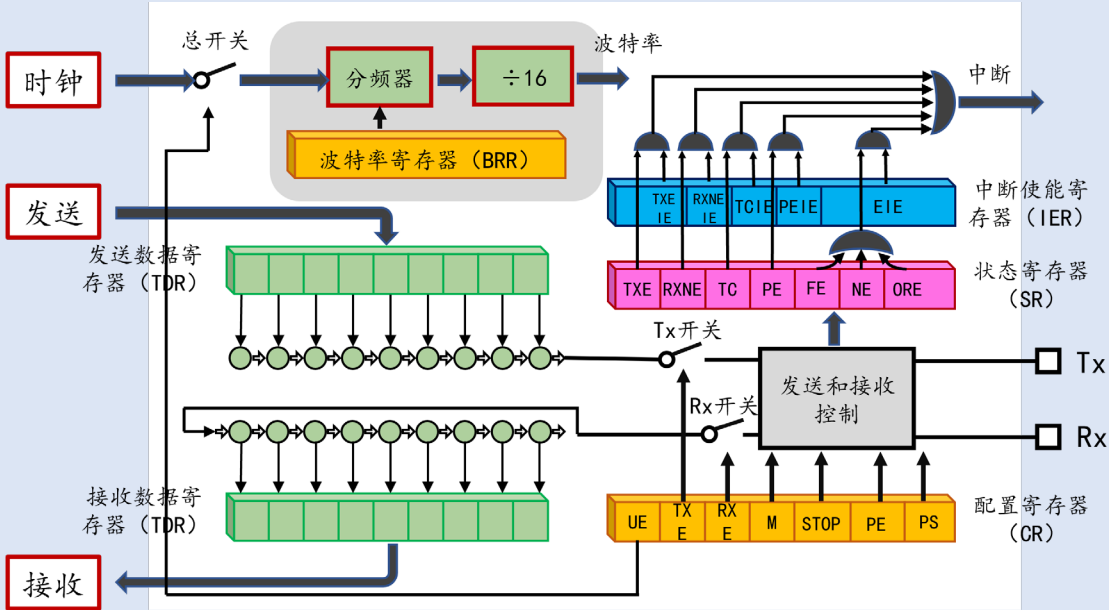
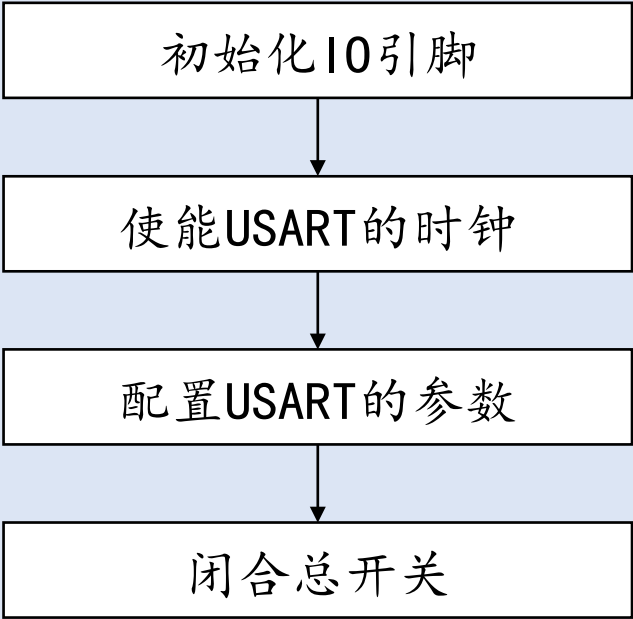
3.3.2. 初始化IO引脚 

3.3.3. 使能USART的时钟 

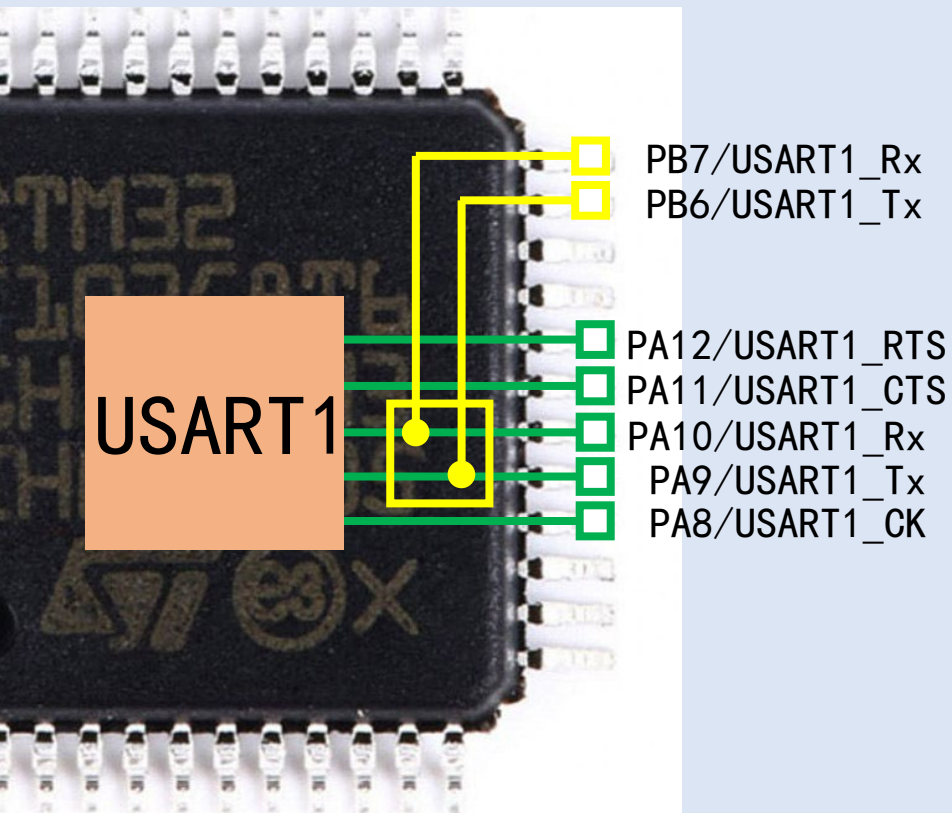
3.3.4. 设置USART的参数 

3.3.5. 闭合总开关 

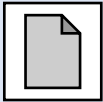
3. 3. 1. USART初始化流程介绍

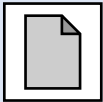


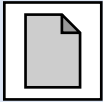
3.3.2. 10引脚初始化



3.3.2.1. 猜测10引脚的参数 

3.3.2.2. 外设IO配置表 

3.3.2.3. 完整的参数列表 

3.3.2.4. 复用功能重映射 

3.3.1.1. 猜测IO引脚的参数



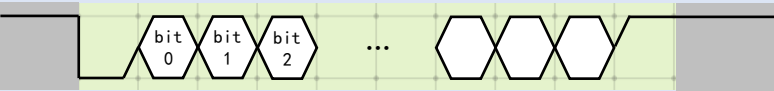
输入？输出？

模拟模式？

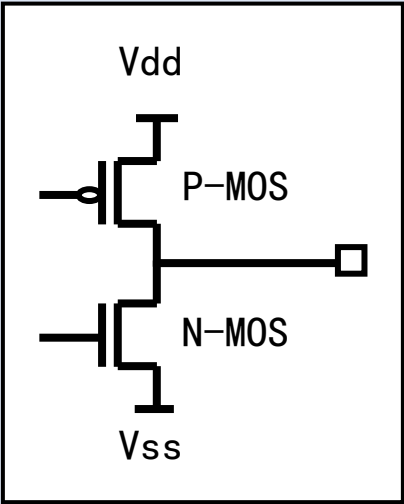
通用？复用？

上拉？下拉？浮空？

推挽？开漏？



	Tx	Rx	CTS	RTS	CK
模式	复用推挽	输入上拉	输入上拉	复用推挽	复用推挽
速度	10MHz			10MHz	10MHz



3.3.1.2. 外设配置表

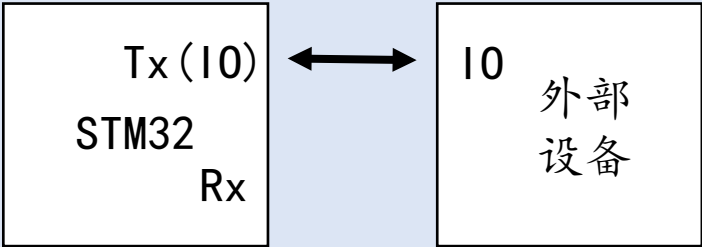


《参考手册》8.1.11 外设的GPIO配置

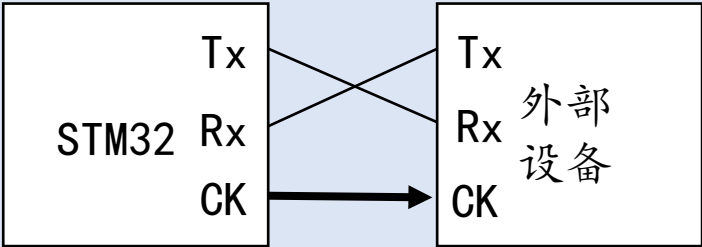
表21 USART

USART引脚	配置	GPIO配置
USARTx_TX	全双工模式	推挽复用输出
	半双工 模式	推挽复用输出
USARTx_RX	全双工模式	浮空输入或带上拉输入
	半双工 模式	未用，可作为通用I/O
USARTx_CK	同步模式	推挽复用输出
USARTx_RTS	硬件流量控制	推挽复用输出
USARTx_CTS	硬件流量控制	浮空输入或带上拉输入

全双工和半双工



同步模式和异步模式



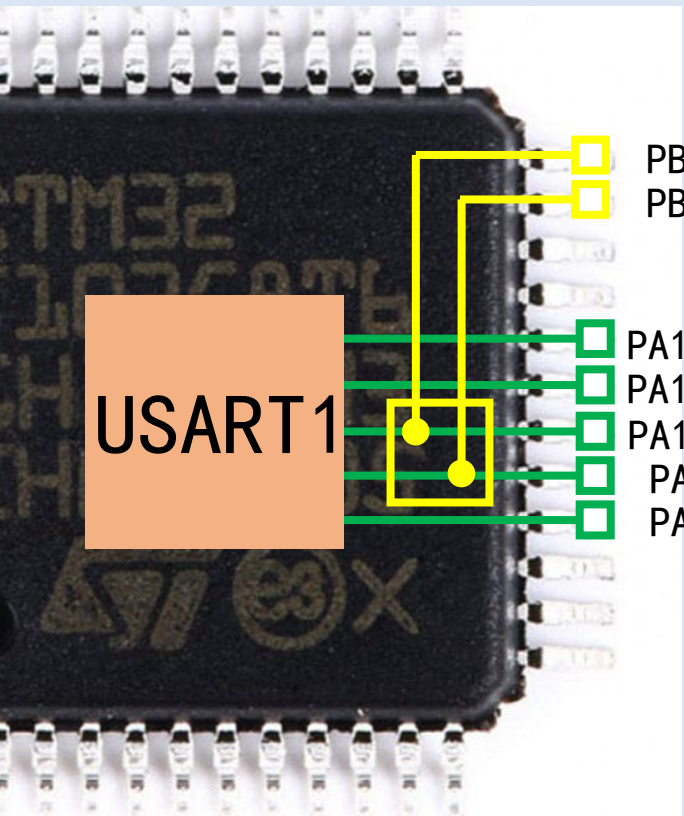
3.3.1.3. 完整的参数列表



	Tx	Rx	CTS	RTS	CK
引脚编号	PB6	PB7	PA11	PA12	PA8
模式	复用推挽	输入浮空/ 输入上拉	输入浮空/ 输入上拉	复用推挽	复用推挽
速度	10MHz	/	/	10MHz	10MHz

未使用的引脚无需配置

3.3.1.4. 复用功能重映射



PB7/USART1_Rx
PB6/USART1_Tx

PA12/USART1_RTS
PA11/USART1_CTS
PA10/USART1_Rx
PA9/USART1_Tx
PA8/USART1_CK

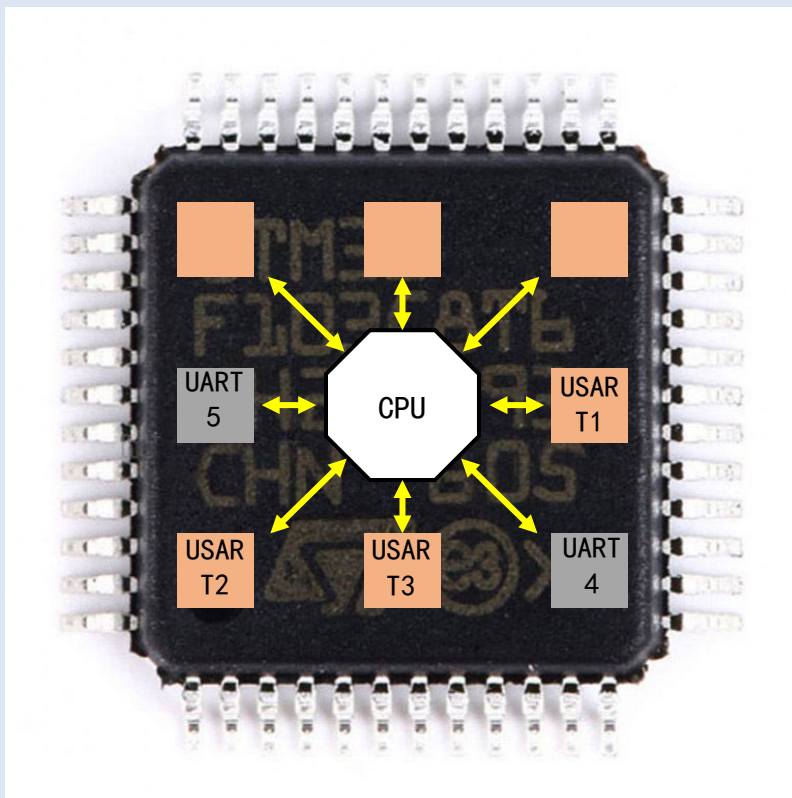
```
// 使能AFIO的时钟
RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO, ENABLE);

// 重映射 = 1
GPIO_PinRemapConfig(GPIO_Remap_USART1, ENABLE);
```

表47 USART1重映像

复用功能	USART1_REMAP = 0	USART1_REMAP = 1
USART1_TX	PA9	PB6
USART1_RX	PA10	PB7

3.3.2. 使能USART的时钟



USART1

```
RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1, ENABLE);
```

USART2..5

```
RCC_APB1PeriphClockCmd(RCC_APB1Periph_USARTx, ENABLE);
```

3.3.3. 设置USART的参数

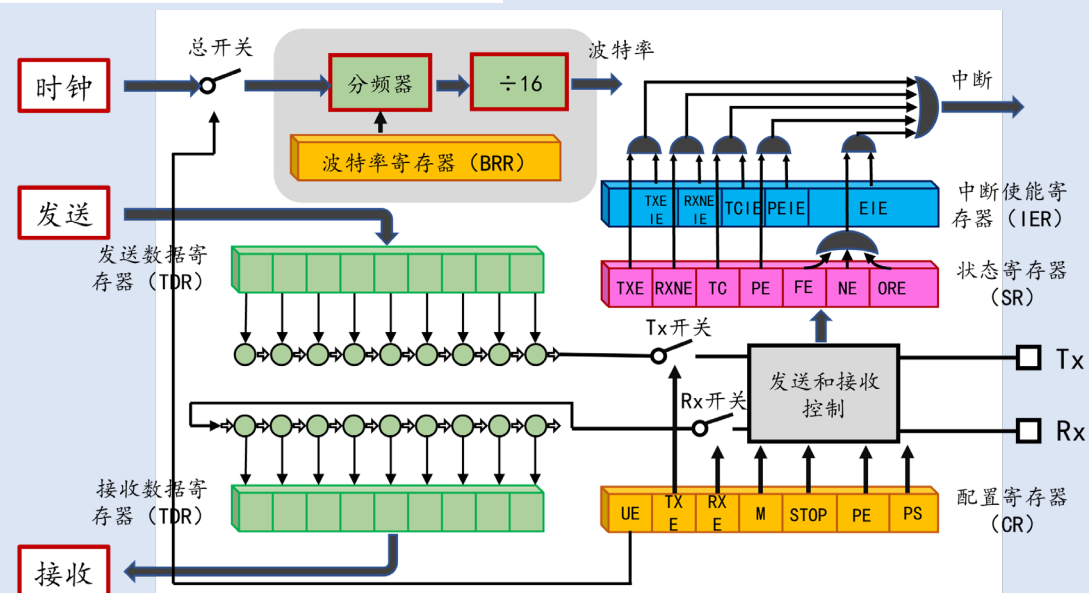
要求：波特率9600，8位数据位，无奇偶校验，1位停止位，收发双向

编程接口

```
1 // 简介：USART初始化(帧格式、波特率)
2 // 参数：·USARTx·-·选择USART片上外设，可以填写USART1,USART2,USART3,UART4,UART5
3 // 参数：·USART_InitStruct·-·初始化结构体
4 void USART_Init(USART_TypeDef·*USARTx, USART_InitTypeDef·*USART_InitStruct);
```

例子

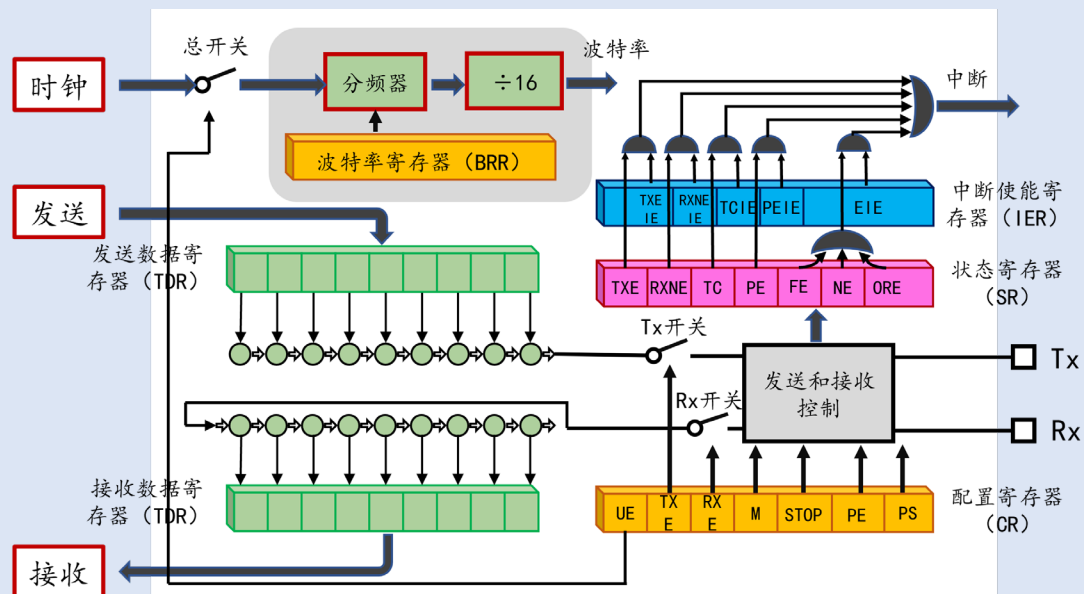
```
2 USART_InitTypeDef USARTInitStruct;
3
4 USARTInitStruct.Baudrate = xxx; // 波特率
5 USARTInitStruct.USART_WordLength = xxx; // 数据位长度
6 USARTInitStruct.USART_StopBits = xxx; // 停止位长度
7 USARTInitStruct.USART_Parity = xxx; // 校验方式
8 USARTInitStruct.USART_Mode = xxx; // 数据方向
9
10 USART_Init(USARTx, &USARTInitStruct);
```



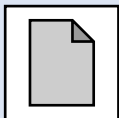
3.3.4. 闭合总开关

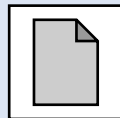
编程接口

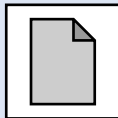
```
1 // 简介: 使能/禁止USART
2 // 参数: USARTx -- 选择USART片上外设, 可以填写USART1, USART2, USART3, UART4, UART5
3 // 参数: NewState -- ENABLE 使能, DISABLE 禁止
4 void USART_Cmd(USART_TypeDef* USARTx, FunctionalState NewState);
```

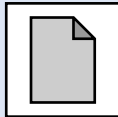


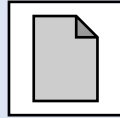
3.4. 使用串口发送数据

3.4.1. 实验介绍 

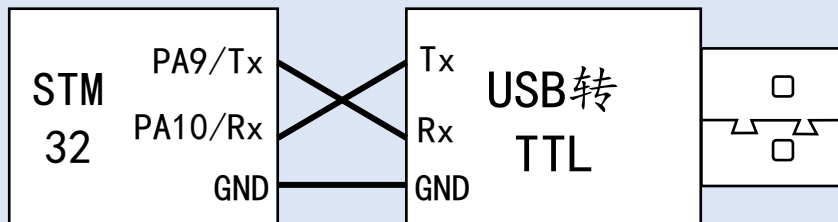
3.4.2. 串口调试助手参数设置 

3.4.3. 发送单个字节 

3.4.4. 发送字节数组 

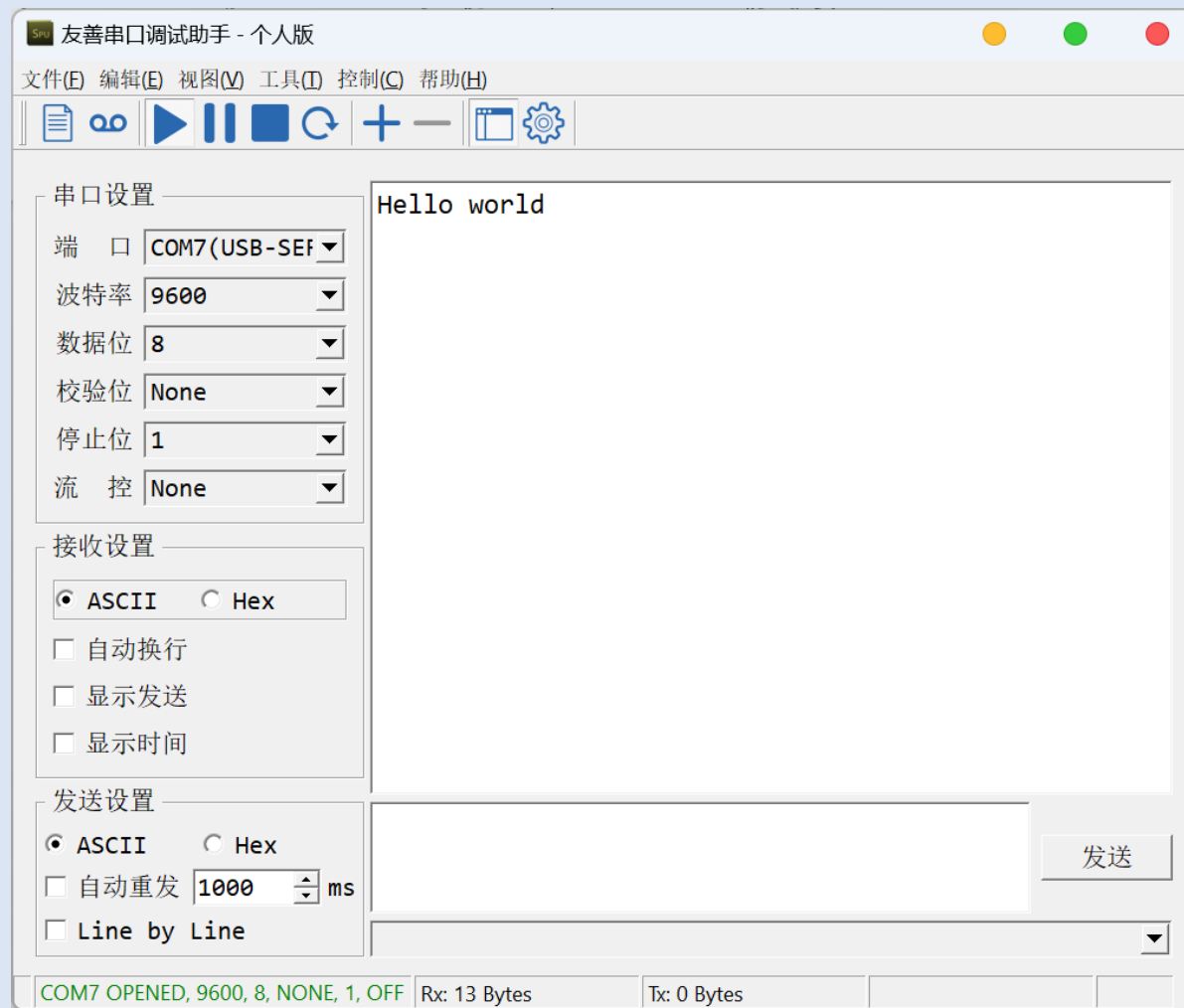
3.4.5. 发送字符和字符串 

3.4.1. 实验介绍

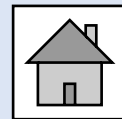


要求:

1. 发送单个字节
2. 发送字节数组
3. 发送字符
4. 发送字符串



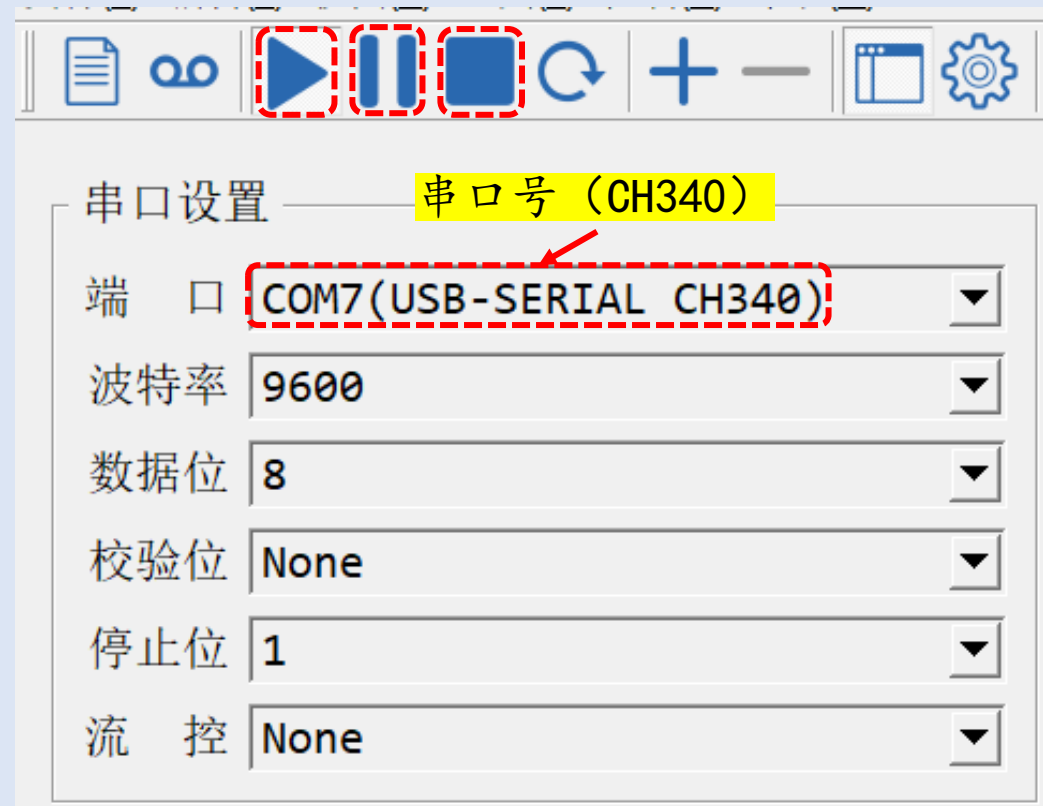
3.4.2. 串口调试助手参数设置



通信双方参数一致

```
29 // 波特率9600
30 USARTInitStruct.USART_BaudRate = 9600;
31 // 8位数据位
32 USARTInitStruct.USART_WordLength = USART_WordLength_8b;
33 // 无奇偶校验
34 USARTInitStruct.USART_Parity = USART_Parity_No;
35 // 1位停止位
36 USARTInitStruct.USART_StopBits = USART_StopBits_1;
37 // 无硬件流控
38 USARTInitStruct.USART_HardwareFlowControl = USART_HardwareFlowControl_None;
39 // 收发双向
40 USARTInitStruct.USART_Mode = USART_Mode_Tx | USART_Mode_Rx;
41 USART_Init(USART1, &USARTInitStruct);
```

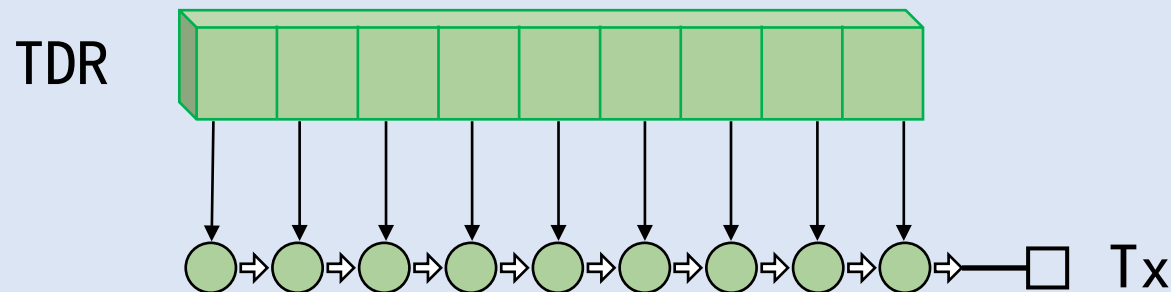
打开 暂停 关闭



串口设置 — 串口号 (CH340)

端 口	COM7(USB-SERIAL CH340)
波特率	9600
数据位	8
校验位	None
停止位	1
流 控	None

3.4.3. 发送单个字节



// 1. 等待TDR清空

```
while(USARTx->SR^TXE == 0);
```

// 2. 写入要发送的数据

```
USARTx->TDR = 0x5a;
```

// 3. 等待数据发送完成

```
while(USARTx->SR^TC == 0);
```

// 1. 等待TDR清空

```
while(USART_GetFlagStatus(USART1, USART_FLAG_TXE) == RESET);
```

// 2. 写入要发送的数据

```
USART_SendData(USART1, 0x5a);
```

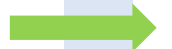
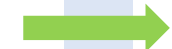
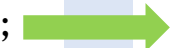
// 3. 等待数据发送完成

```
while(USART_GetFlagStatus(USART1, USART_FLAG_TC) == RESET);
```

3.4.4. 发送字节数组



```
for (i=0; i<n; i++)  
{  
    // 1. 等待TDR清空  
    while (USARTx->SR^TXE == 0);  
    // 2. 写入要发送的数据  
    USARTx->TDR = a[i];  
}  
// 3. 等待数据发送完成  
while (USARTx->SR^TC == 0);
```



```
for (i=0; i<n; i++)  
{  
    // 1. 等待TDR清空  
    while (USART_GetFlagStatus (USART1, USART_FLAG_TXE) == RESET);  
    // 2. 写入要发送的数据  
    USART_SendData (USART1, a[i]);  
}  
// 3. 等待数据发送完成  
while (USART_GetFlagStatus (USART1, USART_FLAG_TC) == RESET);
```

3.4.5. 发送字符和字符串



字符 ASCII码

a -> 0x61

b -> 0x62

c -> 0x63

1 -> 0x31

2 -> 0x32

3 -> 0x33

Hello world -> 0x48, 0x65, ...

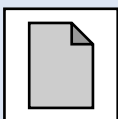
```
USART_SendData(USART1, 0x61); // 发送字符a
```

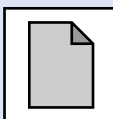


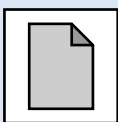
```
USART_SendData(USART1, 'a'); // 发送字符a
```

```
const char *str = "Hello world";  
for(i=0; i<strlen(str); i++)  
{  
    //每次发送一个字符(str[i])  
}
```

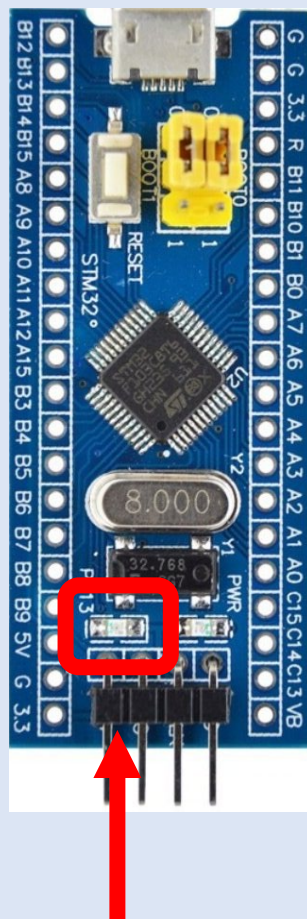
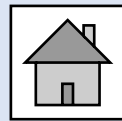
3.5. 使用串口接收数据

3.5.1. 实验介绍 

3.5.2. 接收数据原理回顾 

3.5.3. 思路梳理 

3.5.1. 实验介绍



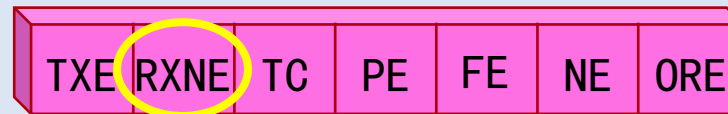
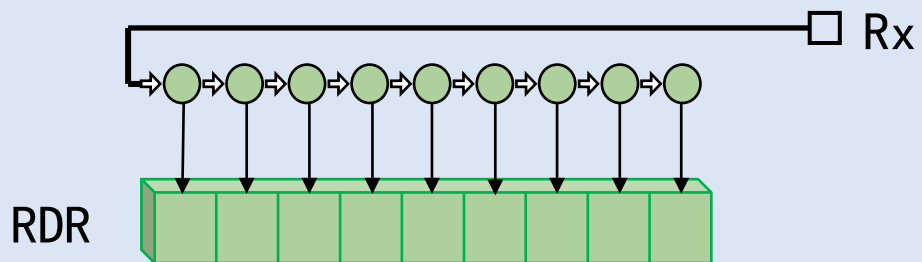
通过串口发指令



发1 - 点亮

发0 - 熄灭

3.5.2. 接收数据原理回顾



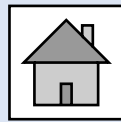
```
uint8_t a;  
while (RXNE==0);  
a = RDR;
```



```
uint8_t a;  
while (USART_GetFlagStatus (USART1, USART_FLAG_RXNE) ==RESET);  
a = USART_ReceiveData (USART1);
```



3.5.3. 思路梳理



```
char c;  
while(1)  
{  
    c = 读取一个字节();  
    if(c== '0') {熄灭LED;}  
    if(c== '1') {点亮LED;}  
}
```

3.6. 随堂测试



题目1：如何使用单片机的UART接口与电脑实现数据通信？

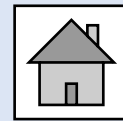
题目2：串口的IO引脚参数应该如何配置？

题目3：分别说出以下功能对应的编程接口

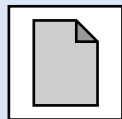
1. 向TDR写入数据
2. 从RDR读取数据
3. 闭合总开关
4. 获取标志位的状态
5. 设置串口数据帧格式

题目4：简要回顾数据发送和接收的代码

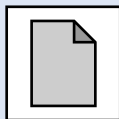
第四节、中断方式的数据接收



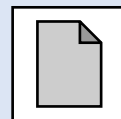
4.1. 中断接收简介



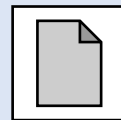
4.2. 配置中断源



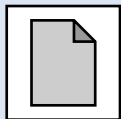
4.3. 中断接收的编程思路



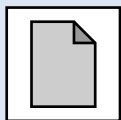
4.4. 改进的串口接收实验



4.5. Echo实验



4.6. 随堂测试





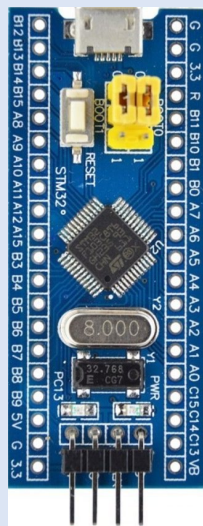
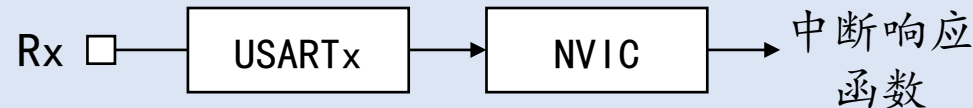
中断方式

```
uint8_t c;

while(1)
{
    // 1. 等待RXNE标志位
    while(RXNE==0);

    // 2. 从RDR读取数据
    c = RDR;

    // 3. 数据处理
    if(c== '0' ) {} // 熄灭
    else if(c== '1' ) {} // 点亮
}
```



‘0’ 灭

‘1’亮

```
void 中断响应函数(void)
{
    uint8_t c;

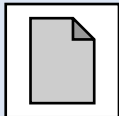
    // 1. 从RDR读取数据

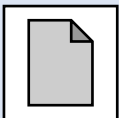
    c = RDR;

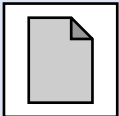
    // 2. 数据处理

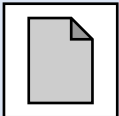
    if(c== '0' ) {} //熄灭
    else if(c== '1' ) {} //点亮
}
```

4.2. 配置中断源

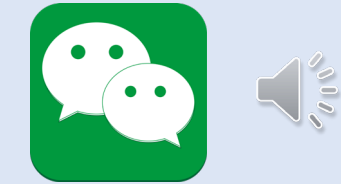
4.2.1. 产生电平型的中断源 

4.2.2. 中断共用 

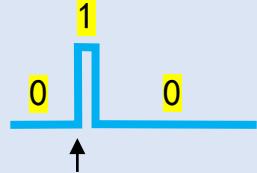
4.2.3. 中断屏蔽 

4.2.4. 编程接口 

4.2.1. 产生电平型的中断源



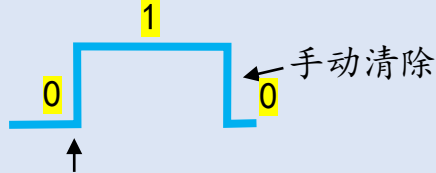
微信提示音
(脉冲型)



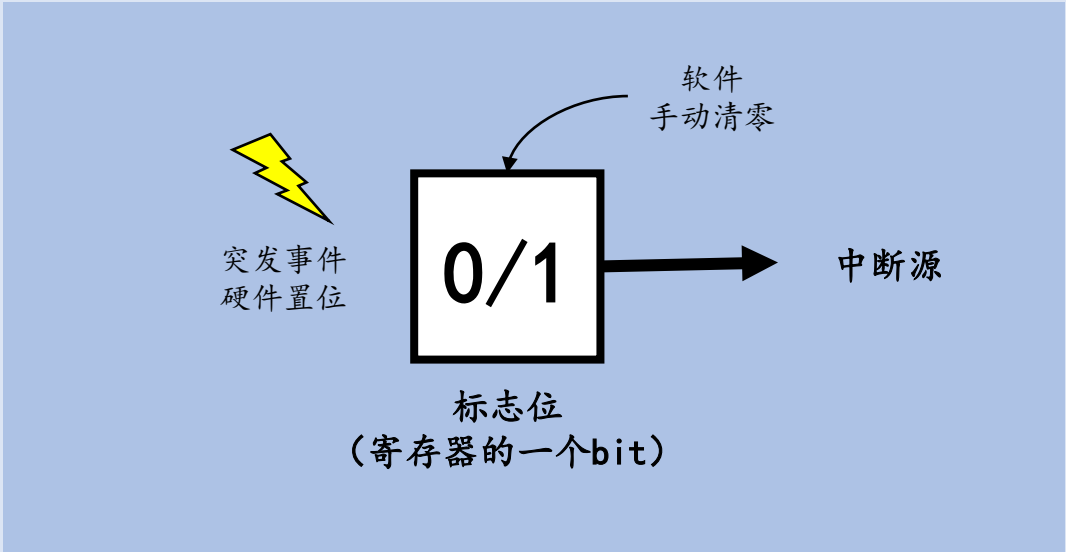
中断触发



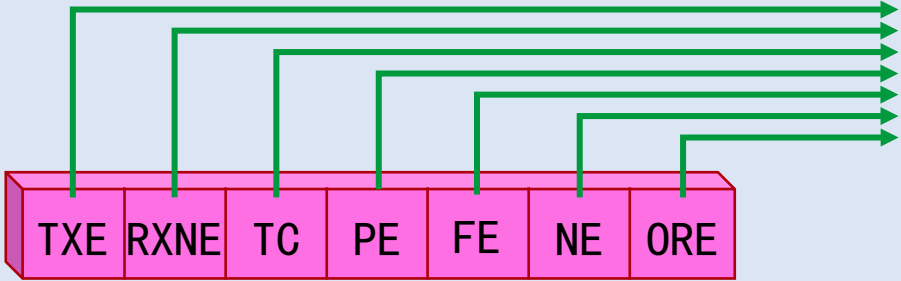
来电铃声
(电平型)



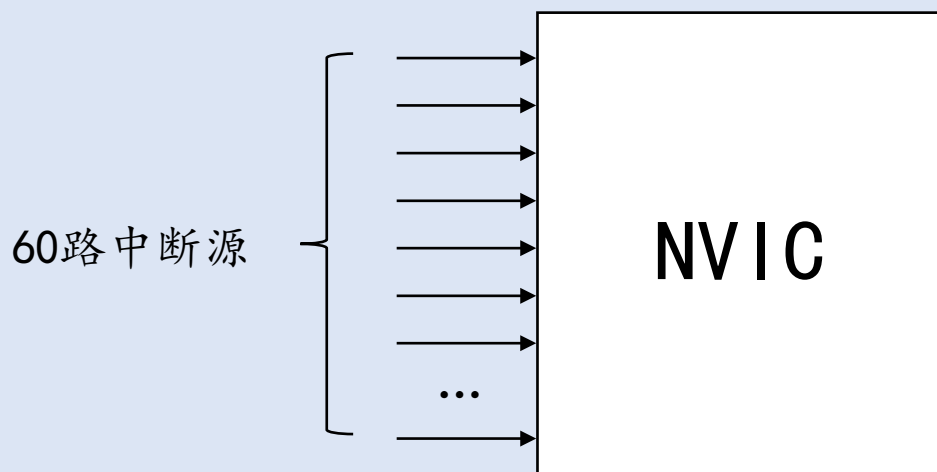
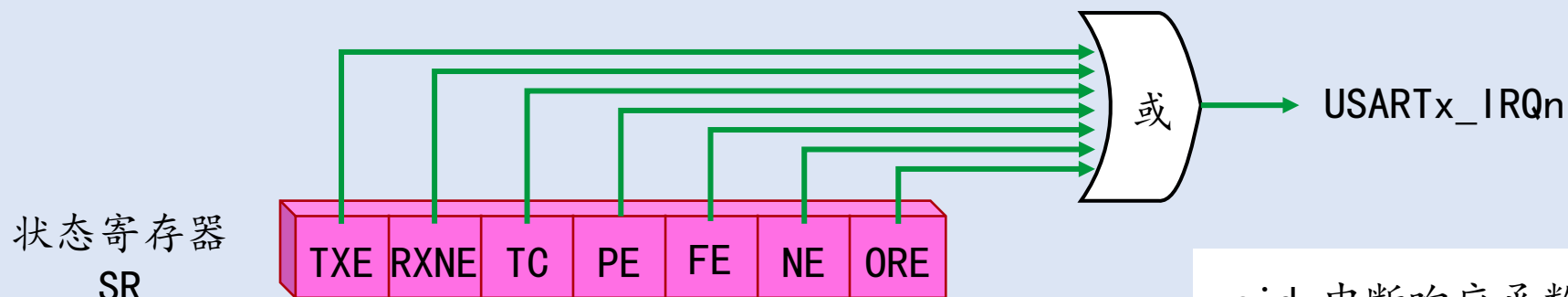
中断触发



状态寄存器
SR

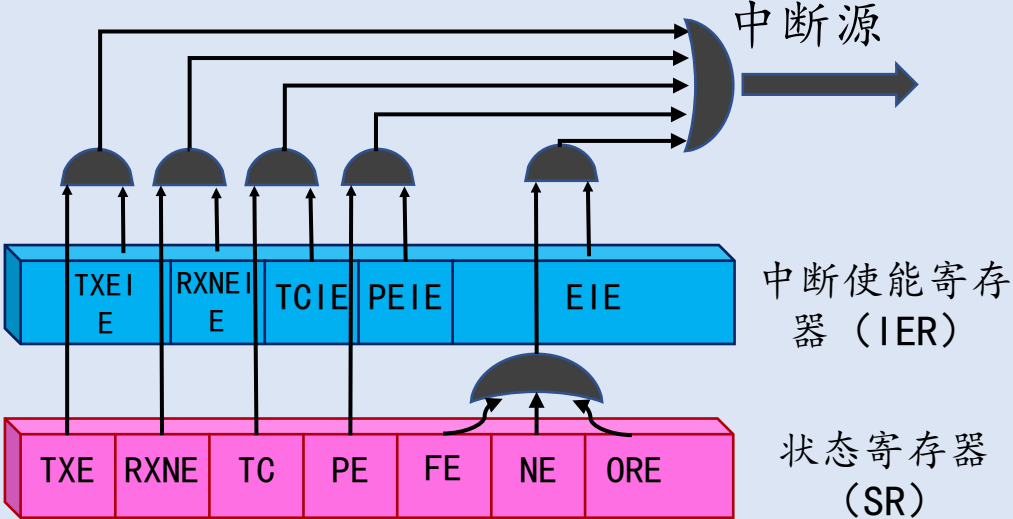
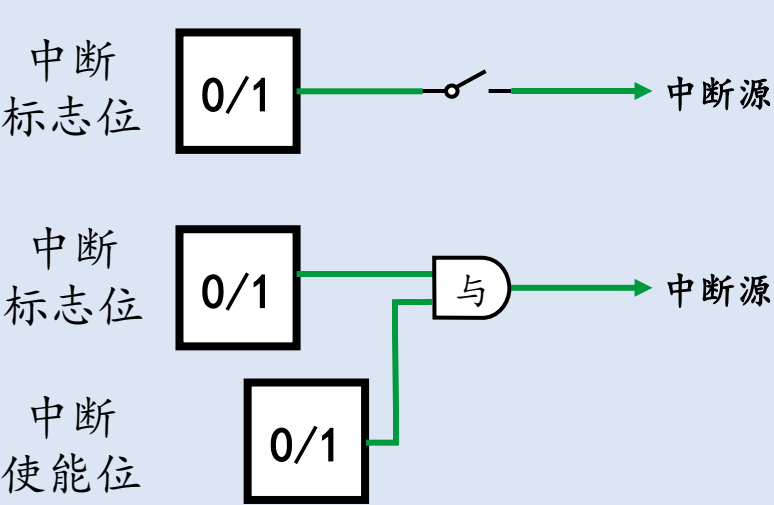
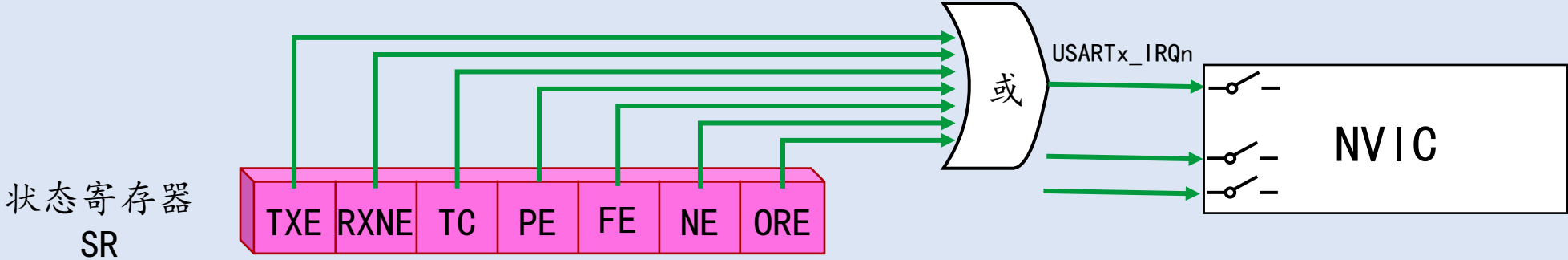


4.2.2. 中断共用



```
void 中断响应函数(void)
{
    if(TXE == 1) {} // 发送数据寄存器空
    if(RXNE == 1) {} // 接收数据寄存器非空
    if(TC == 1) {} // 数据发送完成
    if(PE == 1) {} // 奇偶校验错
    if(NE == 1) {} // 噪声错
    if(ORE == 1) {} // 过载错
}
```

4.2.3. 中断屏蔽



4.2.4. 编程接口



// 使能/禁止中断

void USART_ITConfig(...)



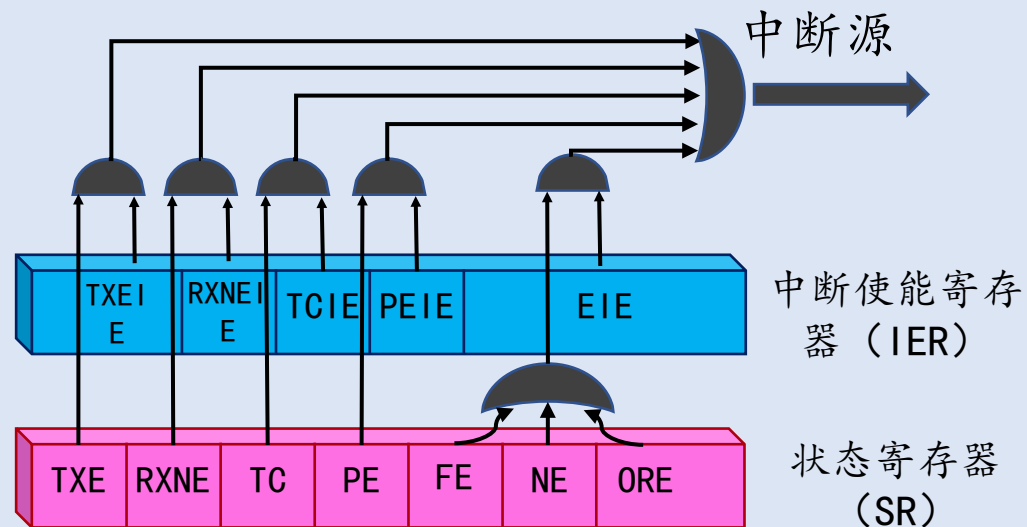
// 查询标志位(只查标志位)

ITStatus USART_GetITStatus(...);



// 清除标志位

void USART_ClearITPendingBit(...);



USART_ITConfig



// 简介：使能/禁止中断

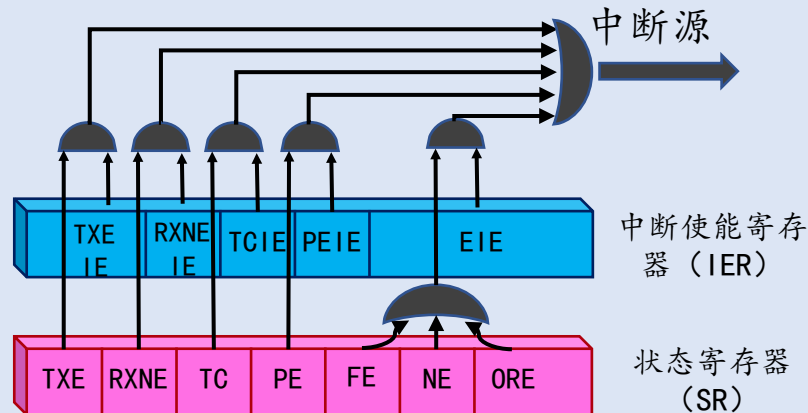
// 参数：USARTx - 要操作的串口的名称，可以是USART1..3或者UART4..5

// 参数：USART_IT - 要操作的中断标志的名称

USART_IT_TXE, USART_IT_TC, USART_IT_RXNE, USART_IT_PE, USART_IT_ERR

// 参数：NewState - 中断的使能或禁止，ENABLE表示使能，DISABLE表示禁止

void USART_ITConfig(USART_TypeDef* USARTx, uint16_t USART_IT, FunctionalState NewState);



示例代码

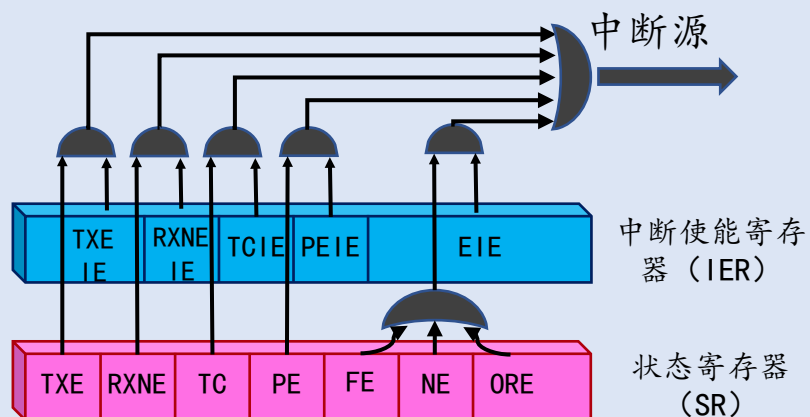
// 使能USART1的RXNE中断

```
USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);
```


USART_GetITStatus



```
// 简介：获取标志位的值（标志位&&中断使能位）  
// 参数：USARTx - 要操作的串口的名称，可以是USART1..3或者UART4..5  
// 参数：USART_IT - 要获取的中断标志的名称  
                USART_IT_TXE, USART_IT_TC, USART_IT_RXNE, USART_IT_PE  
                USART_IT_FE, USART_IT_NE, USART_IT_ORE  
// 返回：SET=1, RESET=0  
ITStatus USART_GetITStatus(USART_TypeDef* USARTx, uint16_t USART_IT)  
FlagStatus USART_GetFlagStatus(...); // 获取标志位的值（标志位）
```



示例代码

```
// 如果是RXNE引起的中断  
if(USART_GetITStatus(USART1, USART_RXNE)==SET)  
{  
    // 接收数据  
    c = USART_ReceiveData(USART1);  
}
```

USART_ClearITPendingBit



// 简介：清除标志位的值

// 参数：USARTx - 要操作的串口的名称，可以是USART1..3或者UART4..5

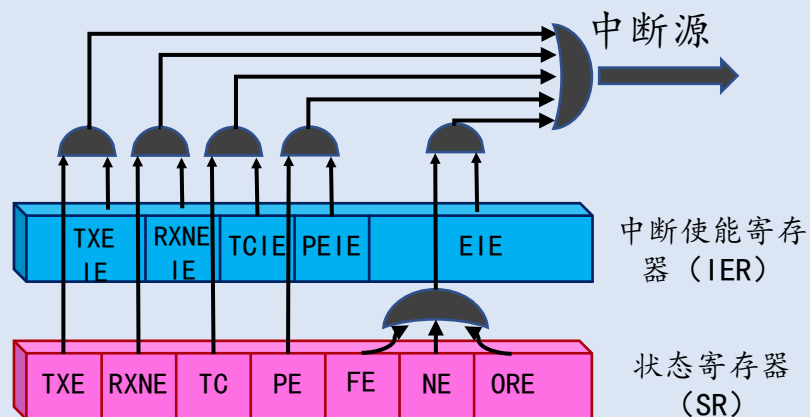
// 参数：USART_IT - 要获取的中断标志的名称

USART_IT_TXE, USART_IT_TC, USART_IT_RXNE, USART_IT_PE

USART_IT_FE, USART_IT_NE, USART_IT_ORE

void USART_ClearITPendingBit(USART_TypeDef* USARTx, uint16_t USART_IT)

void USART_ClearFlag(...); // 效果相同



裸机多任务模型



初始化

```
int main(void)
{
    xxx_Init();
    xxx_Init();
    ...
    xxx_Init();
}
```

任务进程
<5ms

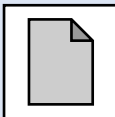
```
while(1)
{
    xxx_Proc();
    xxx_Proc();
    ...
    xxx_Proc();
}
```

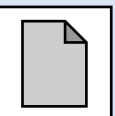
中断响应函数 < 1ms

```
void xxx_IRQHandler(void) {...}
void xxx_IRQHandler(void) {...}
...
void xxx_IRQHandler(void) {...}
```

	延迟函数	最大处理时间
初始化	可用	不限制
任务进程	禁用	单个任务<5ms 合计<30ms，越短越好
中断响应	禁用	<1ms

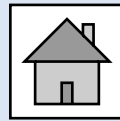
4.3. 中断接收的编程思路

4.3.1. 数据处理能够瞬间完成 

4.3.2. 数据处理耗时较长 

```
void 中断响应函数()  
{  
    // 1. 接收数据  
    uint8_t c = RDR; // 单个指令周期, 耗时0.1us左右  
    // 2. 处理数据  
    // 耗时不确定  
}
```

4.3.1. 数据处理能够瞬间完成



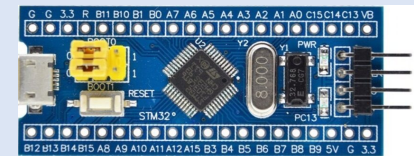
初始化

```
int main(void)
{
    ..
    USART_Recv_Init();
    ..
}
```

进程函数

```
while(1)
{
    ..
}
```

```
void 中断响应函数(void)
{
    c = RDR; // 接收数据
    // 处理数据
    if(c== '0' ) {} // 熄灭
    else if(c== '1' ) {} // 点亮
}
```

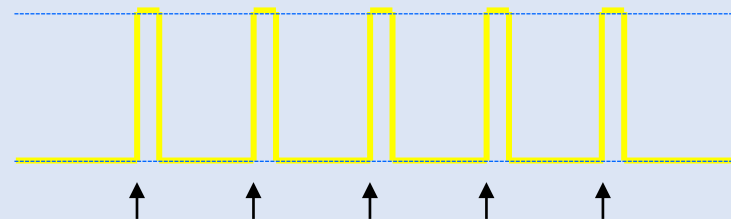


'0' 灭

'1' 亮

能够瞬间完成的
就地处理
(处理速度 >> 接收速度)

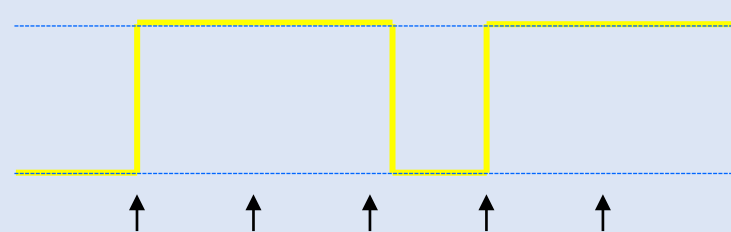
中断程序



常规程序

9600波特率
0.1ms

中断程序



常规程序

9600波特率
0.1ms

4.3.2. 数据处理耗时较长

初始化

```
int main(void)
{
    ..
    USART_Rx_Init();
    ..
}
```

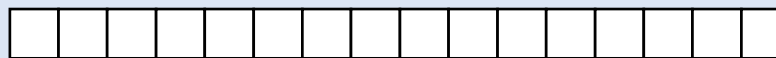
进程函数

```
while(1)
{
    ..
    USART_Rx_Proc();
    ..
}
```

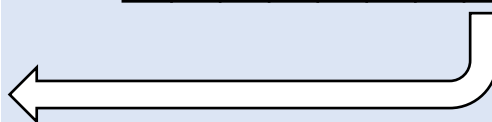
```
void 中断响应函数(void)
{
    c = RDR; // 接收数据
    // 转存数据
    // ...
}
```

短时间处理不完
存入缓冲区

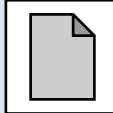
缓冲区(uint8_t buffer[100])

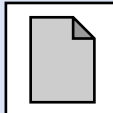


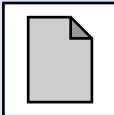
延迟处理

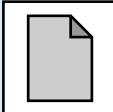


4.4. 改进的串口接收实验

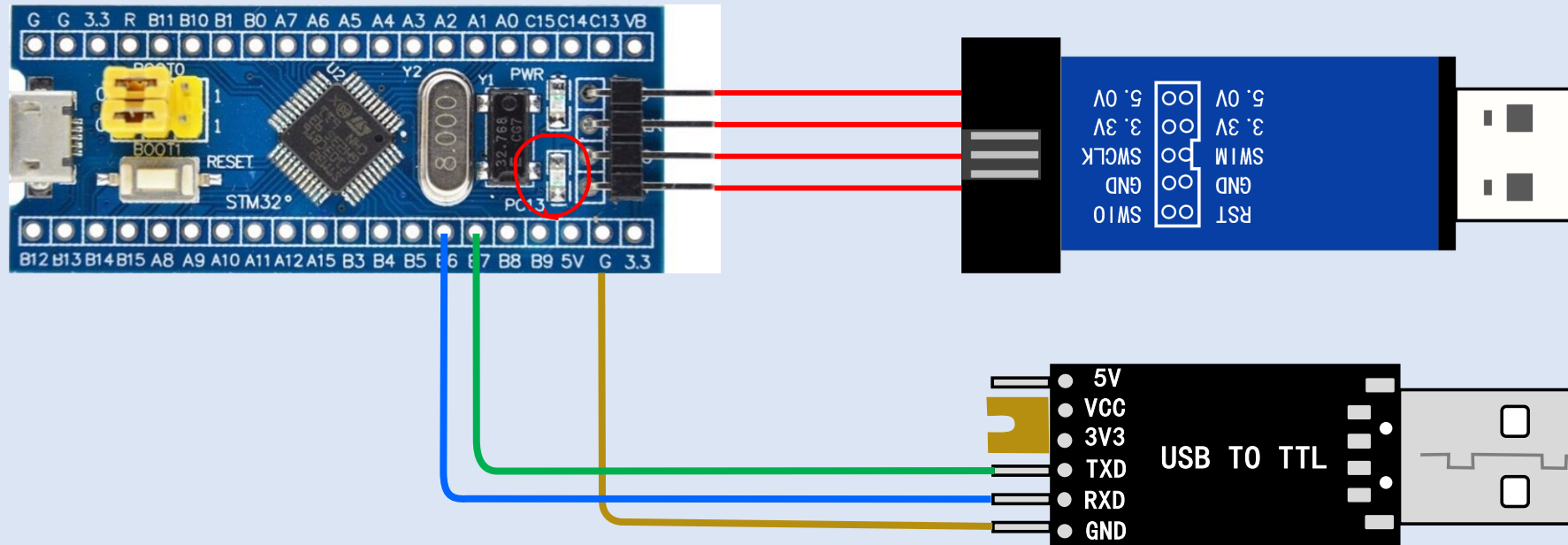
4.4.1. 实验简介 

4.4.2. 编程思路 

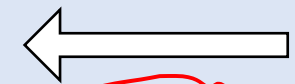
4.4.3. 初始化 

4.4.4. 编写中断响应函数 

4.4.1. 实验简介



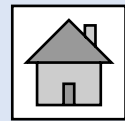
通过串口发指令



发1 - 点亮

发0 - 熄灭

4.4.2. 编程思路



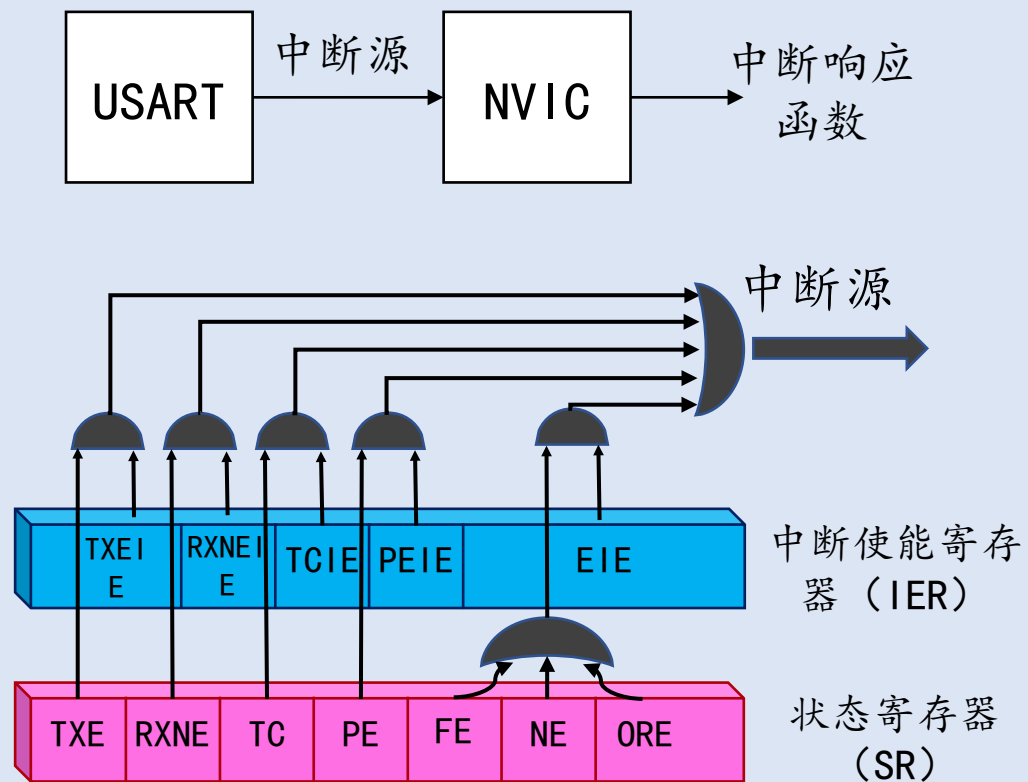
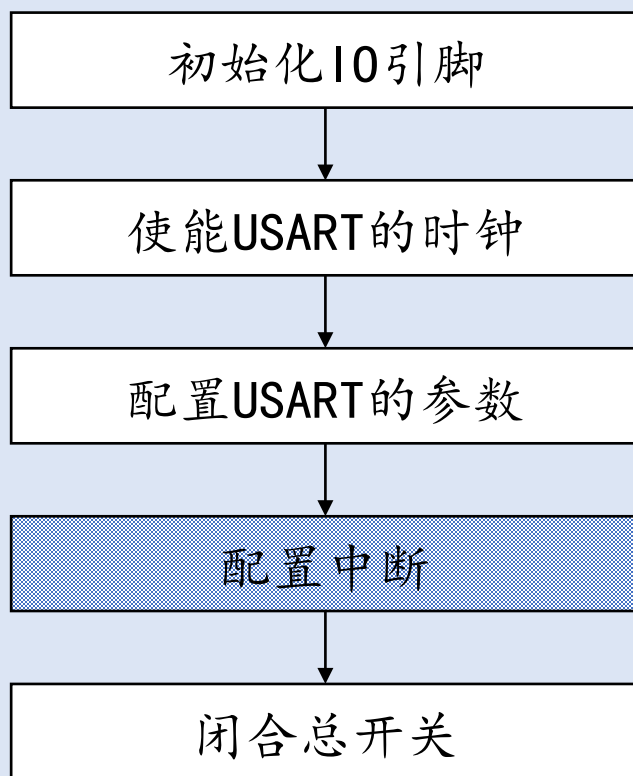
初始化

```
int main(void)
{
    ..
    USART_Recv_Init();
    ..
    while(1)
    {
        ..
    }
}
```

进程函数

```
void 中断响应函数(void)
{
    c = RDR; // 接收数据
    // 处理数据
    if(c== '0' ) {} // 熄灭
    else if(c== '1' ) {} // 点亮
}
```

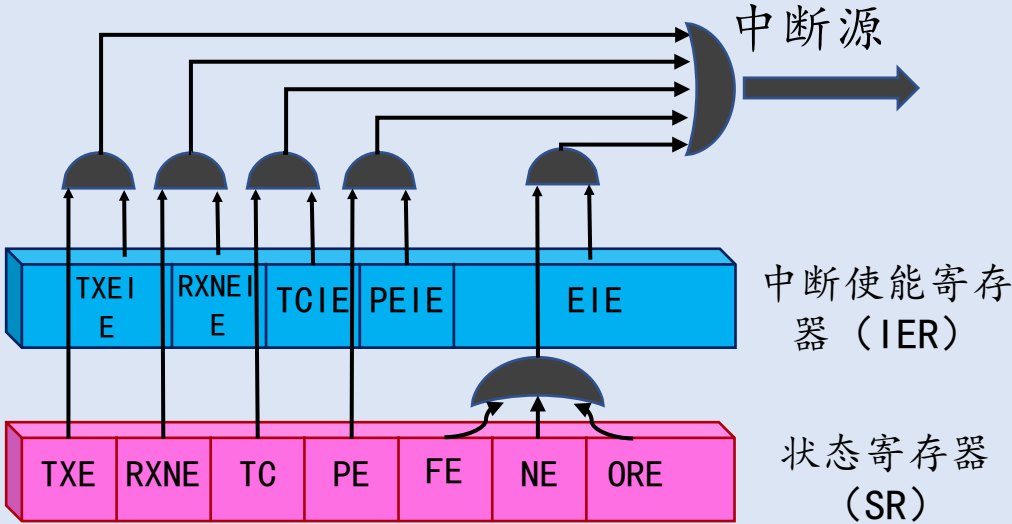
4.4.3. 初始化



4.4.4. 编写中断响应函数

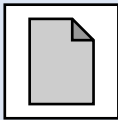


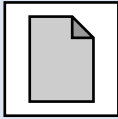
```
void USARTx_IRQHandler(void)
{
    // 1. 判断中断是由谁产生的
    // 2. 清除中断
    // 3. 读取数据
    // 4. 处理中断
}
```

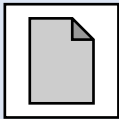


	TXE	TC	RXNE	PE	FE	NE	ORE
置位	TDR空	TDR空&移位寄存器空	收到数据	接收数据出错			
复位	写TDR	先读SR, 再写TDR	读取RDR	先读取SR, 再读取RDR			

4.5. Echo实验

4.5.1. 实验简介 

4.5.2. 思路梳理 

4.5.4. 改进 

4.5.1. 实验简介

echo - 百度翻译

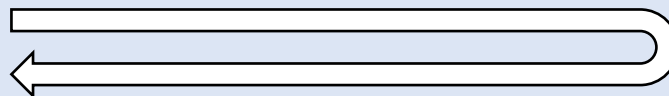
回声 常见释义

英['ekəʊ]   美['ekou]  

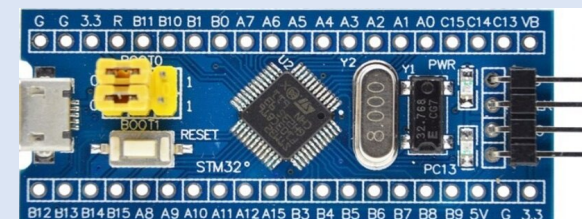
n. 回声; 回音; 回响; 重复; 附和; 共鸣; 反响; 暗示; 启示; 映现;



Hello world\r\n



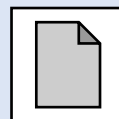
Hello world\r\n



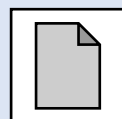
4.5.3. 思路梳理



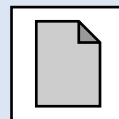
4.5.3.1. 就地处理



4.5.3.2. 队列



4.5.3.3. 延迟处理



4.5.3.1. 就地处理

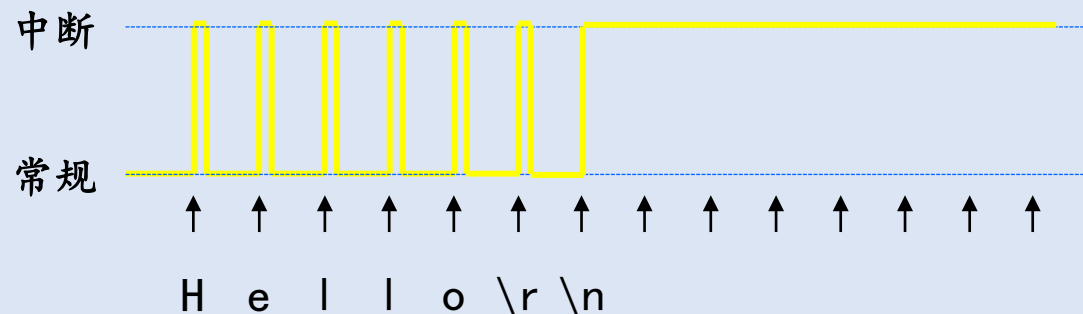
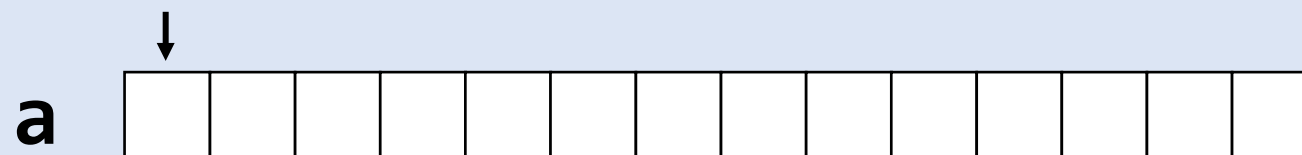


```
int main(void)
{
    USART_Echo_Init(); // 初始化
    while(1) {}
}

uint8_t a[]; // 数组
uint16_t cursor=0; // 游标

void USARTx_IRQHandler(void)
{
    uint8_t c = RDR; // 1. 读取
    a[cursor++] = c; // 2. 缓存
    // 如果遇到新行
    if(cursor>2
        && a[cursor-2]== '\r'
        && a[cursor-1]== '\n' )
    {
        SendString(a); // 3. 发送整行
        cursor=0; // 清空数组a
    }
}
```

Cursor



4.5.3.2. 队列

4.5.3.2.1. 队列简介

4.5.3.2.2. 队列的工作原理

4.5.3.2.3. 队列的C语言实现

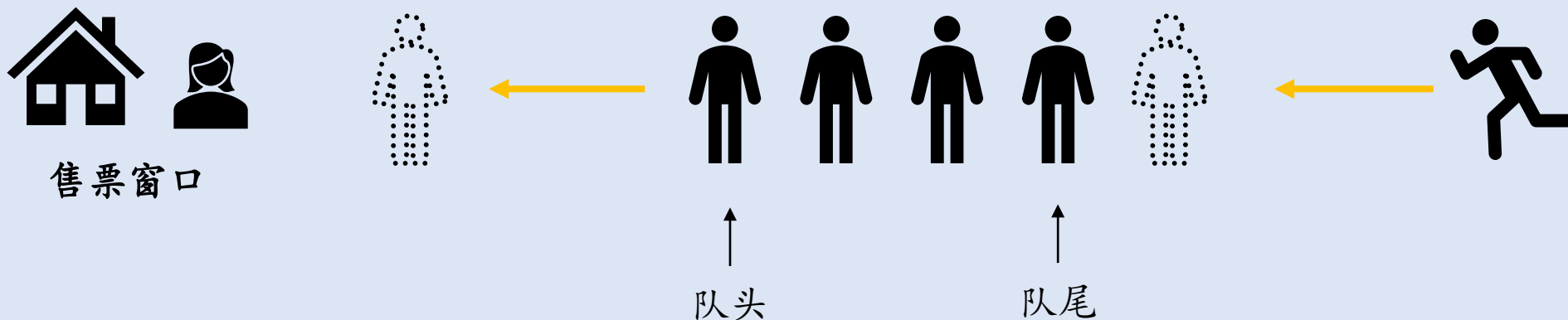
4.5.3.2.1. 队列简介



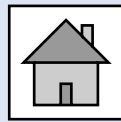
队列（Queue），一种常用的数据结构，经常作为缓冲区来使用，由于具有“先入先出”的特点所以也叫FIFO（First In First Out）

Enqueue - 入队，从队尾增加元素

Dequeue - 出队，将元素从队头取出



4.5.3.2.2. 队列的工作原理



Enqueue(...) - 入队 Dequeue() - 出队

Enqueue('H')

H						
---	--	--	--	--	--	--

Enqueue('e')

H	e					
---	---	--	--	--	--	--

Enqueue('l')

H	e	l				
---	---	---	--	--	--	--

'H' Dequeue()

e	l					
---	---	--	--	--	--	--

Enqueue('l')

e	l	l				
---	---	---	--	--	--	--

'e' Dequeue()

e	e					
---	---	--	--	--	--	--

Enqueue('o')

e	e	o				
---	---	---	--	--	--	--

'l' Dequeue()

e	o					
---	---	--	--	--	--	--

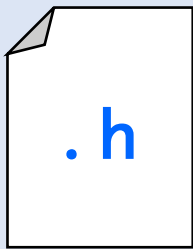
'e' Dequeue()

o						
---	--	--	--	--	--	--

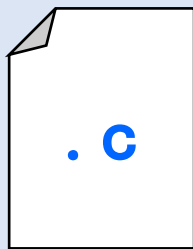
'o' Dequeue()

--	--	--	--	--	--	--

4.5.3.2.3. 队列的C语言实现



queue.h
头文件



queue.c
源文件

```
typedef struct {
    uint8_t Data[100];
    uint16_t Tail; // 队尾
} Queue_HandleTypeDef

void Queue_Init(...);
void Queue_Enqueue(...);
... Queue_Dequeue(...);
```

```
// 初始化
void Queue_Init(...) {
    Tail = 0;
}

// 入队
void Queue_Enqueue(e) {
    Data[Tail++] = e;
}

// 出队
ErrorStatus Queue_Dequeue(uint8_t *pElement) {
    if(Tail == 0) return ERROR;
    *pElement = Data[0];
    for(;;) Data[i] = Data[i+1];
    return SUCCESS;
}
```

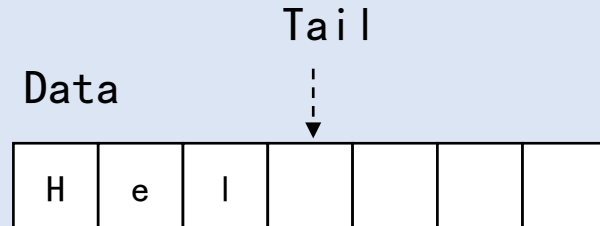
4.5.3.2.3. 队列的C语言实现

```
typedef struct {  
    uint8_t Data[100];  
    uint16_t Tail; // 队尾  
} Queue_HandleTypeDef
```

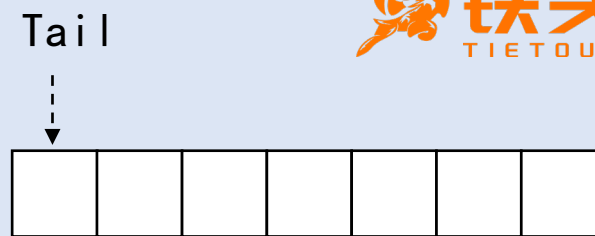
```
// 初始化  
void Queue_Init(...) {  
    Tail = 0;  
}
```

```
// 入队  
void Queue_Enqueue(e) {  
    Data[Tail++] = e;  
}
```

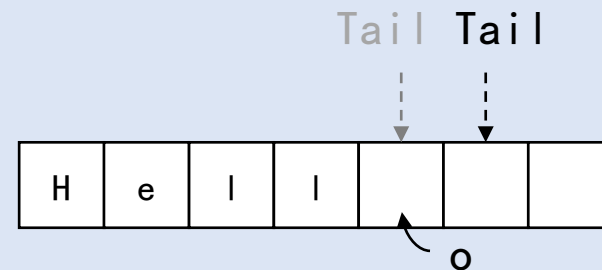
```
// 出队  
ErrorStatus Queue_Dequeue(uint8_t *pElement) {  
    if(Tail == 0) return ERROR;  
    *pElement = Data[0];  
    for(;;) Data[i] = Data[i+1];  
    Tail--;  
    return SUCCESS;  
}
```



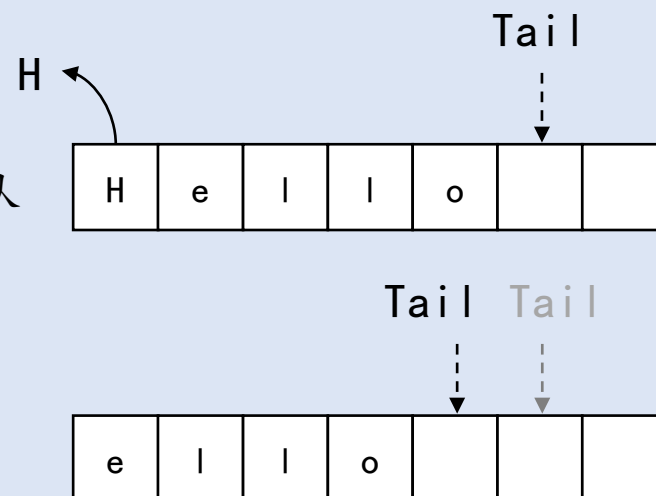
初始状态



入队



出队



4.5.3.3. 延迟处理



主程序

```
int main(void) {  
    USART1_Init();  
    while(1) {  
        USART1_Echo_Proc();  
    }  
}
```

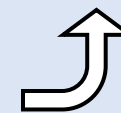
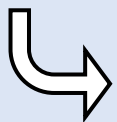
进程函数

```
void USART1_Echo_Proc(void) {  
    uint8_t c;  
    if(Dequeue(&hQueue, &c)) { // 出队  
        a[cursor++] = c;  
        if(cursor>2  
            &&...== '\r' &&...== '\n' )  
        {  
            SendString(buffer3);  
            cursor=0;  
        }  
    }  
}
```

中断函数

```
void USART1_IRQHandler(void) {  
    Enqueue(&hQueue, RDR); // 入队  
}
```

入队
Enqueue



出队
Dequeue

4.5.4. 改进



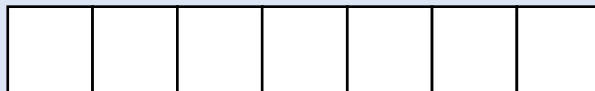
中断函数

```
void USART1_IRQHandler(void) {  
    Enqueue(&hQueue, RDR); // 入队  
}
```

进程函数

```
void USART1_Echo_Proc(void) {  
    Dequeue(&hQueue, &c);  
}
```

Dequeue的过程中发生中断



```
void USART1_Echo_Proc(void) {  
    USART_ITConfig(RXNE, DISABLE);  
    Dequeue(&hQueue, &c);  
    USART_ITConfig(RXNE, ENABLE);  
}
```

4.6. 随堂测试



题目1：简述两种数据接收方式：轮询标志位、中断

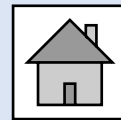
题目2：绘制产生USART中断源的电路图

题目3：分别简述如何清除TXE, RXNE, TC, 以及错误标志位

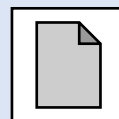
题目4：就地处理和延迟处理分别适用于哪些场景？

题目5：简述线性队列的实现方式

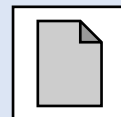
第五节、中断方式的数据发送



5.1. 轮询标志位的缺点



5.2. 中断发送的编程思路



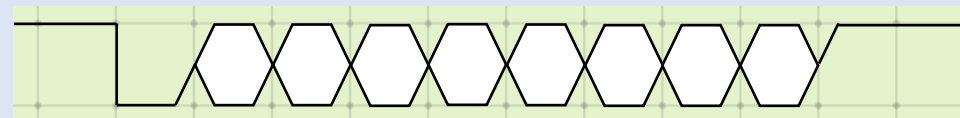
5.3. Echo实验改进

5.1. 轮询标志位的缺点



```
void USART_SendString(Str) {  
    for (...) {  
        while (TXE==0);  
        TDR=Str[i];  
    }  
    while (TC==0);  
}
```

轮询标志位



@9600 bps

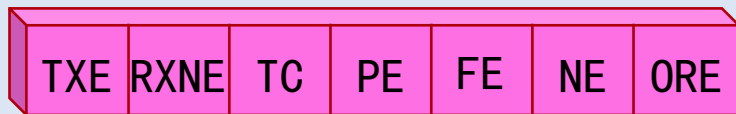
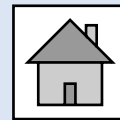
$$1/9600 * 10 * n \approx n \text{ 毫秒}$$

例子：ECHO实验

```
void USART_Echo_Proc()  
{  
    if (... ' \r' && ... ' \n' ) {  
        USART_SendString(Str);  
    }  
}
```

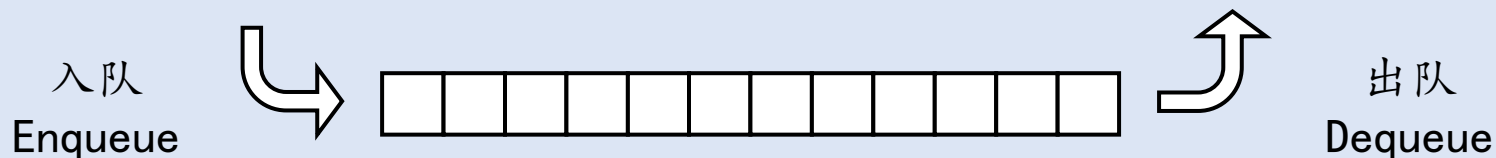
≤5毫秒

5.2. 中断发送的编程思路



```
void USART_SendString(Str) {  
    USART_ITConfig(..., TXE, DISABLE); // 1. 关中断  
    for (;;) Enqueue(Str[i]); // 2. 数据入队  
    USART_ITConfig(..., TXE, ENABLE); // 3. 开中断  
}
```

```
void USARTx_IRQHandler(void) {  
    if (Dequeue(&c) == SUCCESS)  
        TDR=c; // 发送数据  
    else // 数据发完  
        USART_ITConfig(..., TXE, DISABLE); // 关中断  
}
```



第六节 PAL库编程

第五节 中断发送

第四节 中断接收

第三节 标准库编程

第二节 USART

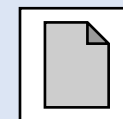
第一节 串口通信



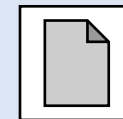
第六节、USART的PAL库编程



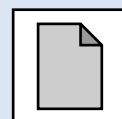
6. 1. USART驱动的工作原理



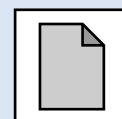
6. 2. USART驱动的使用方法



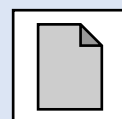
6. 3. 应用案例



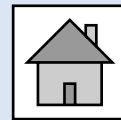
6. 4. 调整缓冲区的大小



6. 5. 随堂测试



6.1. USART驱动的工作原理



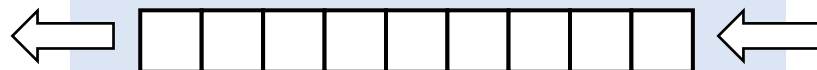
中断方式的数据接收

```
void PAL_USART_ReceiveData(...)  
{  
    Dequeue(...); // 出队  
}
```

```
void PAL_USART_SendData(...)  
{  
    Enqueue(...); // 入队  
}
```

中断方式的数据发送

接收队列
RxQueue

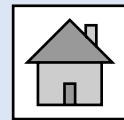


发送队列
TxQueue

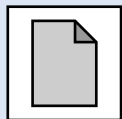


```
void USARTx_IRQHandler(void)  
{  
    uint8_t c;  
    if(RXNE != 0) // RXNE中断  
    {  
        c=RDR;  
        Enqueue(RDR); // 入队  
    }  
    else if(TXE != 0) // TXE中断  
    {  
        c = Dequeue(); // 出队  
        TDR = c;  
    }  
}
```

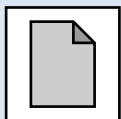
6.2. USART驱动的使用方法



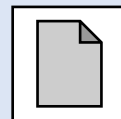
6.2.1. 驱动概述



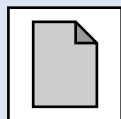
6.2.2. 初始化



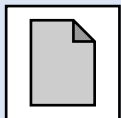
6.2.3. 编写中断响应函数



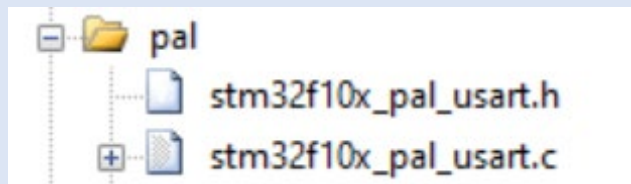
6.2.4. 数据的发送



6.2.5. 数据的接收



6.2.1. 驱动概述



// 初始化

```
ErrorStatus PAL_USART_Init(...);
```

// 数据发送

```
void PAL_USART_SendByte(...); // 发送单个字节
```

```
void PAL_USART_SendBytes(...); // 发送字节数组
```

```
void PAL_USART_PutChar(...); // 发送单个字符
```

```
void PAL_USART_SendString(...); // 发送字符串
```

```
void PAL_USART_Printf(...); // 发送格式化字符串
```

// 数据接收

```
int16_t PAL_USART_RecieveByte(...); // 接收单个字节
```

```
uint16_t PAL_USART_RecieveBytes(...); // 接收字节数组
```

```
uint16_t PAL_USART_ReadLine(...); // 接收一行字符串
```

// 杂项

```
void PAL_USART_IRQHandler(...); // 中断响应函数
```

```
void PAL_USART_Log(...); // 打印日志
```

```
uint16_t PAL_USART_GetTxBufferOccupancy(...) // 获取发送缓冲区占用率
```

```
uint16_t PAL_USART_GetRxBufferOccupancy(...) // 获取接收缓冲区占用率
```

6.2.2. 初始化



```
// @简介: 对USART进行初始化  
// @参数: Handle - USART的句柄  
// @返回值: SUCCESS表示初始化成功, ERROR表示初始化失败  
ErrorStatus PAL_USART_Init(PalUSART_HandleTypeDef *Handle)
```

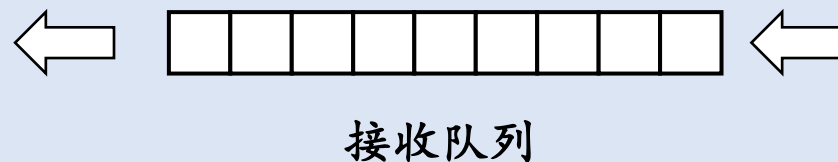
```
hUSART1.Init.USARTx = USART1; // 使用哪个串口  
hUSART1.Init.BaudRate = 9600; // 波特率  
hUSART1.Init.USART_WordLength = USART_WordLength_8b; // 数据位  
hUSART1.Init.USART_Parity = USART_Parity_No; // 校验位  
hUSART1.Init.USART_StopBits = USART_StopBits_1; // 停止位  
hUSART1.Init.USART_Mode = USART_Mode_Tx | USART_Mode_Rx; // 方向  
hUSART1.Init.USART_IRQ_PreemptionPriority = 0; // 中断抢占优先级  
hUSART1.Init.USART_IRQ_SubPriority = 0; // 子优先级  
hUSART1.Init.TxBufferSize = 128; // 发送缓冲区大小  
hUSART1.Init.RxBufferSize = 128; // 接收缓冲区大小  
hUSART1.Init.Advanced.Remap = 1; // 复用功能重映射  
PAL_USART_Init(&hUSART1);
```



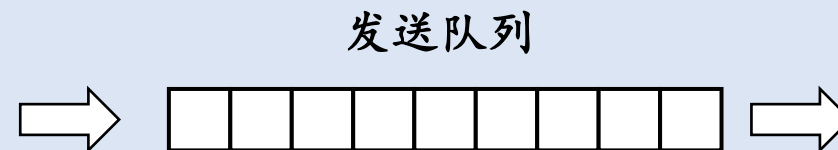
1. 初始化IO引脚
2. 配置IO重映射
3. 使能USART的时钟
4. 配置USART的参数
5. 配置中断
6. 初始化Tx、Rx队列
7. 闭合USART的总开关

发送缓冲区和接收缓冲区

```
int16_t PAL_USART_RecieveByte(...)  
{  
    Deuque(...);  
}
```



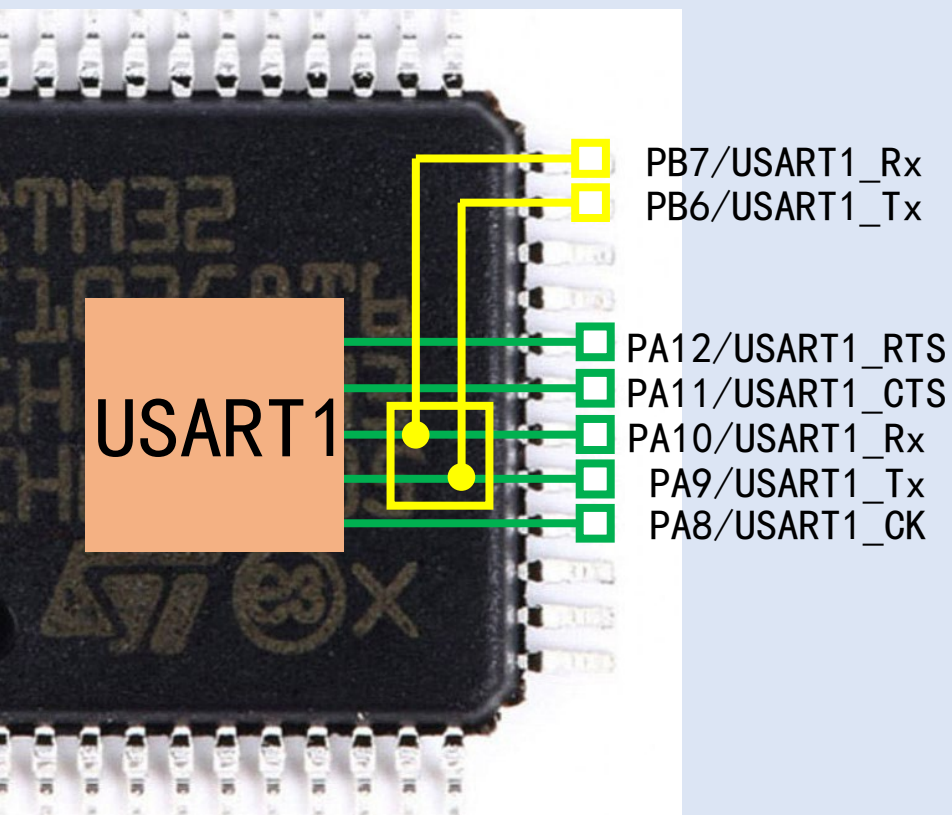
```
int16_t PAL_USART_SendByte(...)  
{  
    Enqueue(...);  
}
```



TxBufferSize: 最多有同时多少个元素待发送, 默认128即可

RxBufferSize: 最多有同时多少个元素待接收, 默认128即可

复用功能重映射



```
hUSART1.Init.Advanced.Remap = 1;
```

```
// 使能AFIO的时钟
```

```
RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO, ENABLE);
```

```
// 重映射 = 1
```

```
GPIO_PinRemapConfig(GPIO_Remap_USART1, ENABLE);
```

表47 USART1重映像

复用功能	USART1_REMAP = 0	USART1_REMAP = 1
USART1_TX	PA9	PB6
USART1_RX	PA10	PB7

6.2.3. 编写中断响应函数



```
// @简介: USART中断处理代码  
// @参数: Handle - USART的句柄  
// @注意: 需要在对应的中断响应函数中调用该方法  
void PAL_USART_IRQHandler (PalUSART_HandleTypeDef *Handle);
```

// 示例代码

```
void USART1_IRQHandler(void)  
{  
    PAL_USART_IRQHandler(&hUSART1);  
}
```

```
void USARTx_IRQHandler(void)  
{  
    uint8_t c;  
    if(RXNE != 0) // RXNE中断  
    {  
        c=RDR;  
        Enqueue(c); // 入队  
    }  
    else if(TXE != 0) // TXE中断  
    {  
        c = Dequeue(); // 出队  
        TDR = c;  
    }  
}
```

6.2.4. 数据的发送



以上编程接口仅将数据入队，发送工作由中断响应函数完成，因此调用后会立即返回

// 发送单个字节 0x5a

```
void PAL_USART_SendByte(PalUSART_HandleTypeDef *Handle, uint8_t Data);
```

// 发送字节数组 01 02 03 04 05

```
void PAL_USART_SendBytes(PalUSART_HandleTypeDef *Handle, const uint8_t *pData, uint16_t Size)
```

// 发送单个字符 ‘H’

```
void PAL_USART_PutChar(PalUSART_HandleTypeDef *Handle, char c);
```

// 发送字符串 Hello world\r\n

```
void PAL_USART_SendString(PalUSART_HandleTypeDef *Handle, const char *Str);
```

// 发送格式化字符串

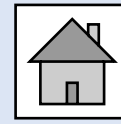
```
void PAL_USART_Printf(PalUSART_HandleTypeDef *Handle, const char* format, ...);
```

// 清空发送队列

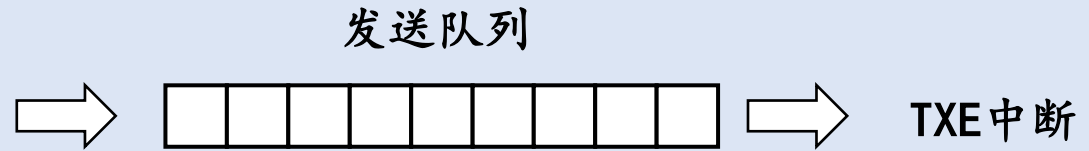
```
void PAL_USART_DiscardOutBuffer(PalUSART_HandleTypeDef *Handle);
```



PAL_USART_DiscardOutBuffer



```
int16_t PAL_USART_SendString(...)  
{  
    Enqueue(...);  
}
```



6.2.5. 数据的接收



// 接收单个字节



 int16_t PAL_USART_RecieveByte(PalUSART_HandleTypeDef *Handle, uint32_t Timeout);

// 接收字节数组

uint16_t PAL_USART_ReciveBytes(PalUSART_HandleTypeDef *Handle, uint8_t *pData, uint16_t Size, uint32_t Timeout);

// 读取一整行字符串

uint16_t PAL_USART_ReadLine(PalUSART_HandleTypeDef *Handle, char *pBuffer, uint16_t Size, uint32_t Timeout); 

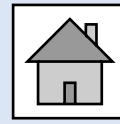
// 查询接收缓冲区里的字符个数

uint16_t PAL_USART_NumberOfBytesToRead(PalUSART_HandleTypeDef *Handle); 

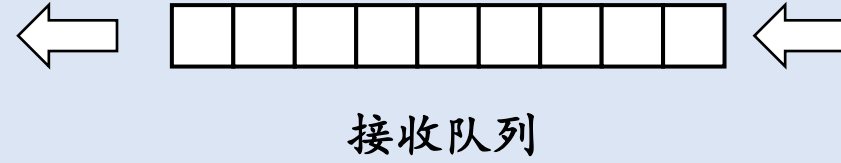
// 清空接收缓冲区

void PAL_USART_DiscardInBuffer(PalUSART_HandleTypeDef *Handle); 

PAL_USART_DiscardInBuffer



```
int16_t PAL_USART_RecieveByte(...)  
{  
    Deuque(...);  
}
```





超时值
最长等待时间



```
int16_t PAL_USART_RecieveByte(Timeout) {  
    while (Now < Start + Timeout)  
    {  
        if (Deuque(...) == SUCCESS) break;  
    }  
}
```



Timeout = 0	- 不等待，立即退出
Timeout > 0	- 等待一段时间，以ms为单位
Timeout = PAL_MAX_DELAY	- 一直等待下去，直到接受到数据为止

串口数据接收实验



```
[Rx][15:47:49.253] Receive single byte ...  
[Tx][15:48:02.047] 5A  
[Rx][15:48:02.086] Byte received: 0x5a  
[Rx][15:48:02.086] Receive 5 bytes ...  
[Tx][15:48:09.209] 01 02 03 04 05  
[Rx][15:48:09.256] 5 bytes received: 01 02 03 04 05  
[Rx][15:48:09.286] Receive a line...  
[Tx][15:48:19.604] I love stm32!  
[Rx][15:48:19.665] Line received: I love stm32!
```

接收1个字节
PAL_USART_ReceiveByte

接收5个字节
PAL_USART_ReceiveBytes

接收一行字符串 (\r\n)
PAL_USART_ReadLine

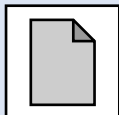
接收一行字符串

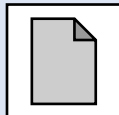


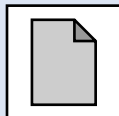
```
hUSART1.Init.USARTx = USART1; // 使用哪个串口
hUSART1.Init.BaudRate = 9600; // 波特率
hUSART1.Init.USART_WordLength = USART_WordLength_8b; // 数据位
hUSART1.Init.USART_Parity = USART_Parity_No; // 校验位
hUSART1.Init.USART_StopBits = USART_StopBits_1; // 停止位
hUSART1.Init.USART_Mode = USART_Mode_Tx | USART_Mode_Rx; // 方向
hUSART1.Init.USART_IRQ_PreemptionPriority = 0; // 中断抢占优先级
hUSART1.Init.USART_IRQ_SubPriority = 0; // 子优先级
hUSART1.Init.TxBufferSize = 128; // 发送缓冲区大小
hUSART1.Init.RxBufferSize = 128; // 接收缓冲区大小
hUSART1.Init.Advanced.Remap = 1; // 复用功能重映射
hUSART1.Init.Advanced.LineSeparator = LineSeparator_CRLF; // 换行符=\r\n
PAL_USART_Init(&hUSART1);
```

```
PAL_USART_ReadLine(...)
```

6.3. 应用案例

6.3.1. Echo实验 

6.3.2. 串口日志打印 

6.3.3. 串口波形监控 

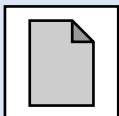
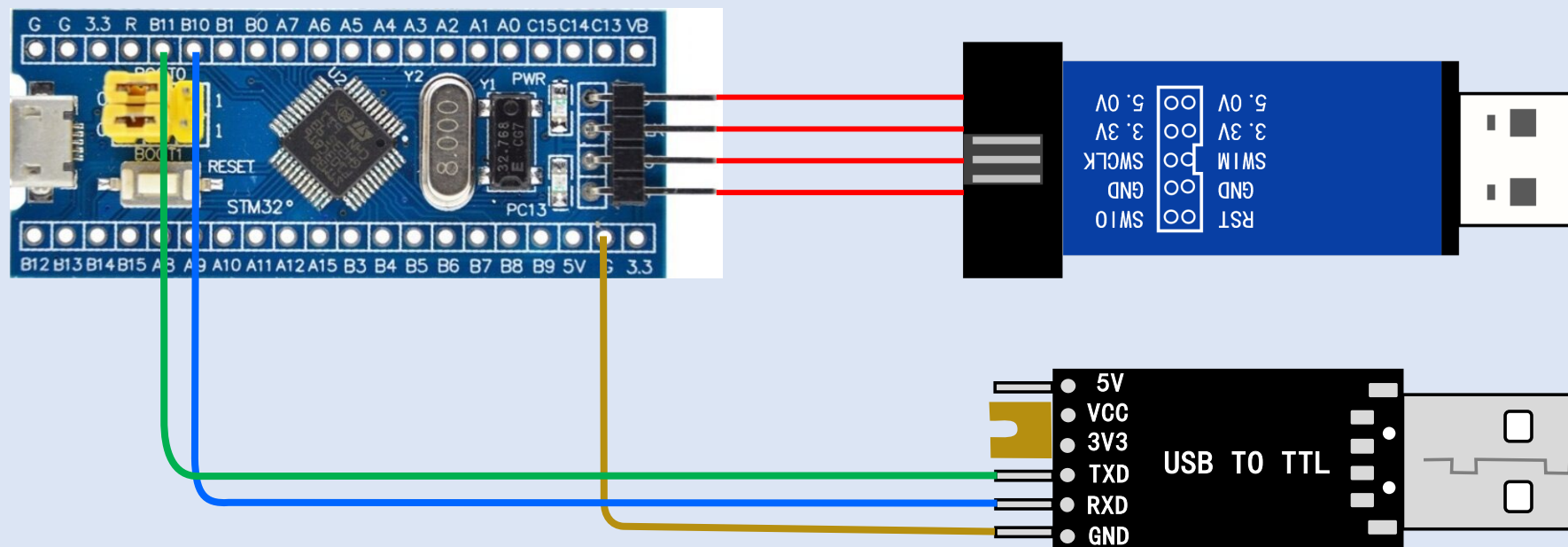
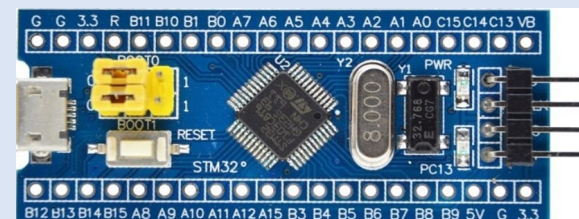
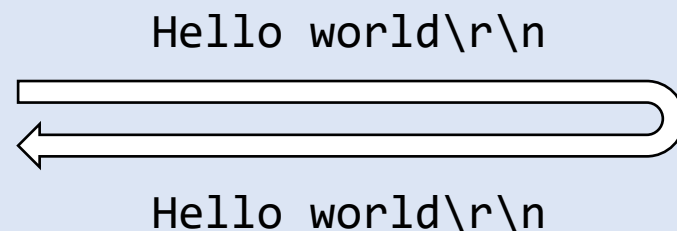
6.3.4. 其它用途 

表45 USART3重映像

复用功能	USART3_REMAP[1:0] = 00 (没有重映像)	USART3_REMAP[1:0] = 01 (部分重映像) ⁽¹⁾	USART3_REMAP[1:0] = 11 (完全重映像) ⁽²⁾
USART3_TX	PB10	PC10	PD8
USART3_RX	PB11	PC11	PD9
USART3_CK	PB12	PC12	PD10
USART3_CTS	PB13		PD11
USART3_RTS	PB14		PD12



6.3.1. Echo实验

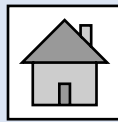


```
int main(void)
{
    // 初始化
    hUSART3.Init.USARTx=USART1;
    hUSART3.Init.Baudrate=xxx;
    ...
    PAL_USART_Init(...);
    // 进程函数
    while(1)
    {
        Echo_Proc(); // Echo进程函数
    }
}
```

```
// 进程函数
void EchoProc(void) {
    if(PAL_USART_ReadLine(..., 0) > 0)  如果收到了一行
    {
        PAL_USART_SendString(...); // 把它发送出去
    }
}
```

```
// 中断响应函数
void USART3_IRQHandler(void)
{
    PAL_USART_IRQHandler(...);
}
```

PAL_USART_ReadLine



返回值

实际接收到的
字符串长度

句柄指针

接收缓冲区

```
uint16_t PAL_USART_ReadLine(PalUSART_HandleTypeDef *Handle, char *pBuffer,  
uint16_t Size, uint32_t Timeout);
```

接收缓冲区大小

超时时间

设置换行
符的格式

```
hUSART1.Init.Advanced.LineSeparator = LineSeparator_CRLF; // 换行符=\r\n  
PAL_USART_Init(&hUSART1);
```

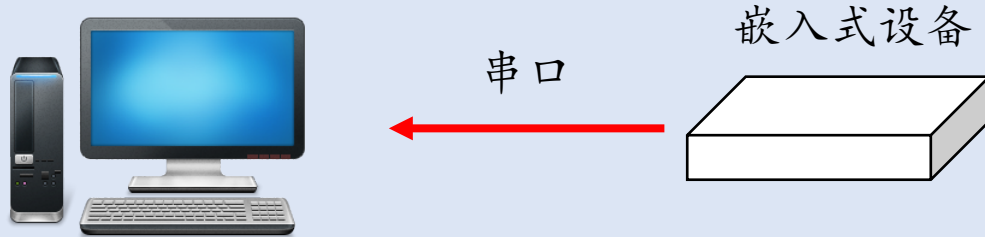
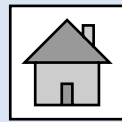
PAL_USART_ReadLine(···, pBuffer, 0);



PAL_USART_ReadLine(···, pBuffer, 0);



6.3.2. 打印日志



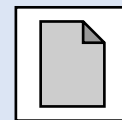
```
[00:00:01.115] Info: 底层硬件初始化结束
[00:00:01.115] Info: 初始化蓝牙模块...
[00:00:01.115] Info: 蓝牙模块初始化成功, rssi=-65 dB
[00:00:01.115] Warning: 蓝牙信号弱, 连接不稳定
[00:00:01.115] Info: 初始化OLED显示器...
[00:00:01.115] Error: OLED显示器启动失败!
[00:00:01.115] Error: 发生严重错误, 程序关闭!
```

```
// @简介: 按一定格式打印日志, 日志格式[hh:mm:ss.sss] 警告级别: 日志内容
// @参数: Handle - 串口句柄指针
// @参数: WarningLevel - 警告级别。
//          WarningLevel_Info - 信息; WarningLevel_Warning - 警告; WarningLevel_Error - 错误
// @参数: Format - 日志格式字符串
// @参数: ... - 可变参数
void PAL_USART_Log(PalUSART_HandleTypeDef *Handle, uint16_t WarningLevel, const char *Format, ...)
```

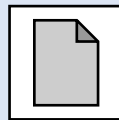
6.3.3. 波形监控



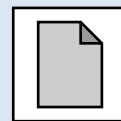
6.3.3.1. 波形监控的使用场景



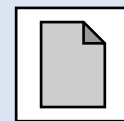
6.3.3.2. 实验介绍



6.3.3.3. 波特率估算



6.3.3.4. 周期性任务的实现方法



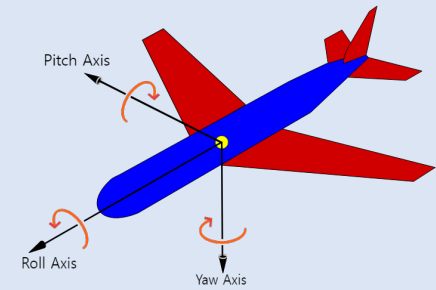
6.3.3.1. 波形监控的使用场景



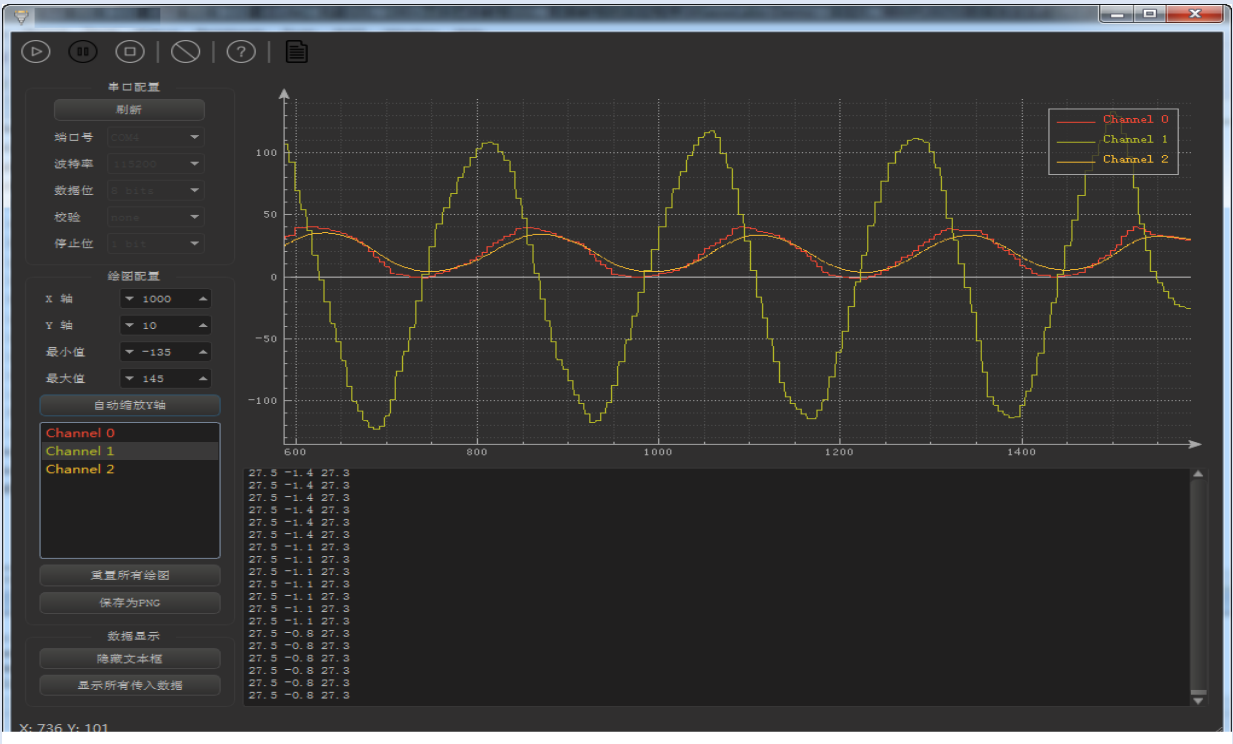
运动传感器
MPU6050



测量运动姿态

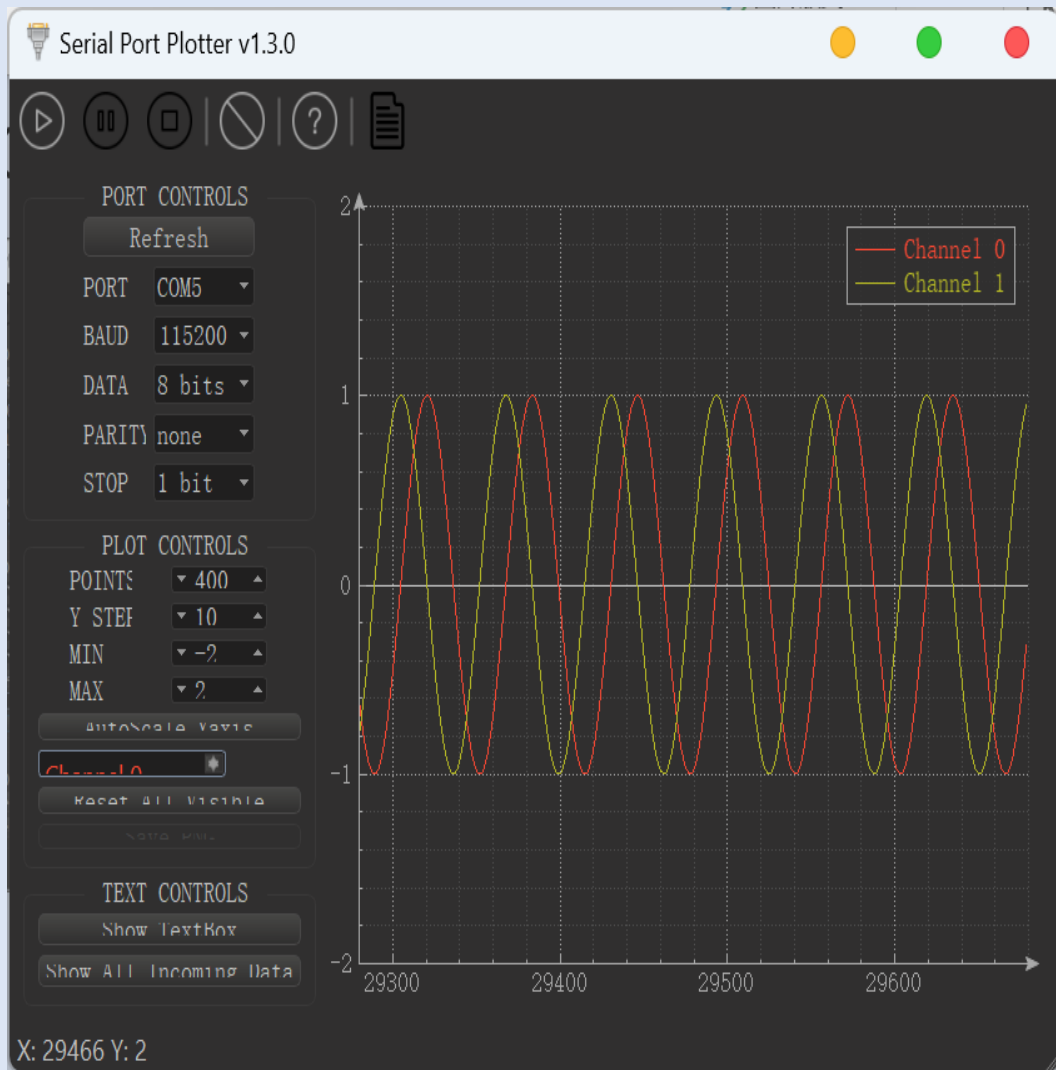


Serial Port Plotter



```
PAL_USART_Printf(..., "$%f %f;", line1Value, line2Value);
```

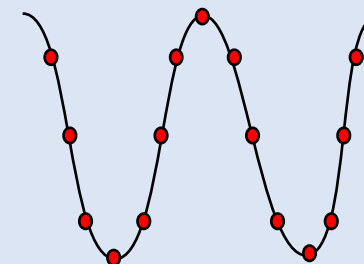
6.3.3.2. 实验介绍



波形1 - $\sin(10*t)$

波形2 - $\cos(10*t)$

要求每10ms显示一个数据点



```
int main(void) {  
    while(1) {  
        WaveformGenerator_Proc();  
        PAL_Delay(10);  
    }  
}  
  
void WaveformGenerator_Proc(void)  
{  
    t = PAL_GetTick() * 0.001;  
    y1 = sin(10*t);  
    y2 = cos(10*t);  
    PAL_USART_Printf(&hUSART1, "$%.3f %.3f;", y1, y2);  
}
```

每10ms执行一次

6.3.3.3. 波特率估算



```
$-0.774 0.633;  
$-0.707 0.707;  
$-0.633 0.774;  
$-0.552 0.834;  
$-0.466 0.885;  
$-0.376 0.927;  
$-0.282 0.960;  
$-0.184 0.983;  
$-0.085 0.996;
```

```
PAL_USART_Printf(&hUSART1, "$%.3f %.3f;", y1, y2);
```

波特率 $> (7 * 2 + 1) * 10 * 100 = 15000$

7 - 每个通道7个字节

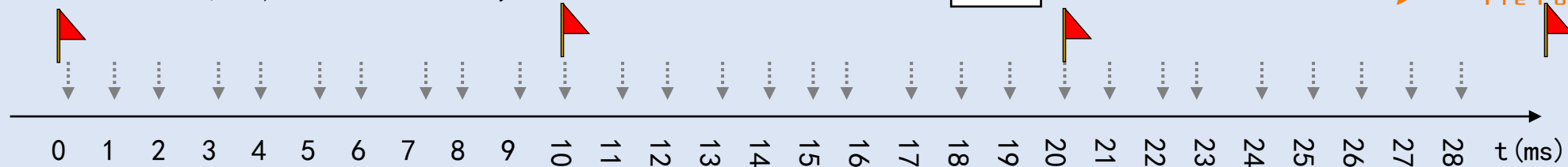
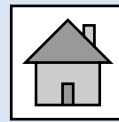
2 - 每次传输2个通道 (y1和y2)

1 - \$和;

10 - 每个数据帧10个bit

100 - 每秒钟传输100个点

6.3.3.4. 周期性任务的实现方法



```
int main(void) {
    while(1) {
        WaveformGenerator_Proc();
        PAL_Delay(10);
    }
}

void WaveformGenerator_Proc(void)
{
    x = PAL_GetTick() * 0.001;
    y1 = sin(x);
    y2 = cos(x);
    PAL_USART_Printf(...);
}
```

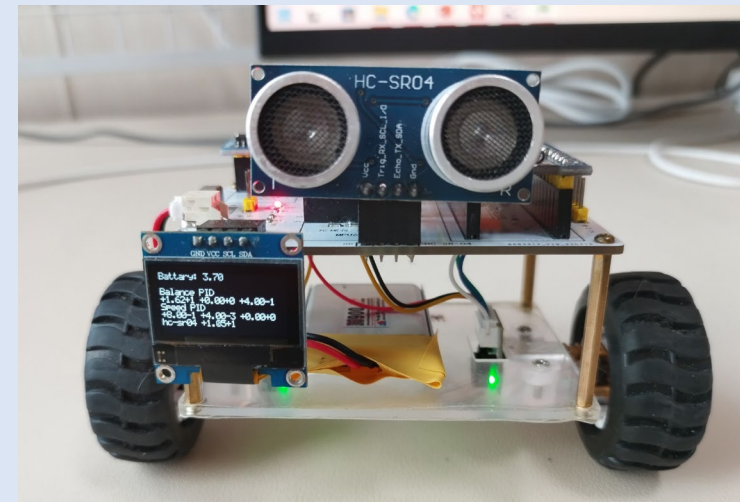
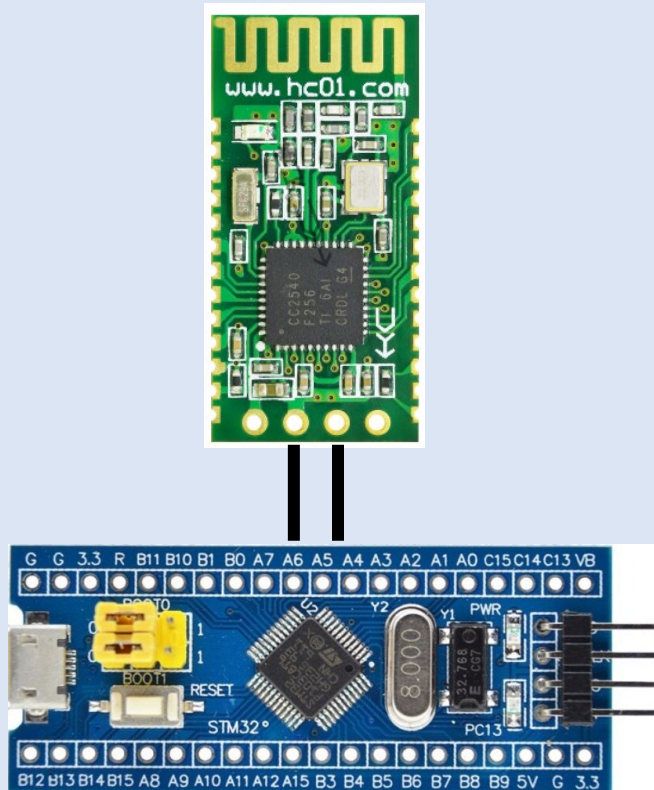
```
int main(void) {
    while(1) {
        WaveformGenerator_Proc();
        xxx_Proc();
        xxx_Proc();
    }
}

void WaveformGenerator_Proc(void)
{
    static uint64_t nextExecTime = 0;
    if(PAL_GetTick() >= nextExecTime) {
        x = PAL_GetTick() * 0.001;
        y1 = sin(x);
        y2 = cos(x);
        PAL_USART_Printf(...);
        nextExecTime += 10;
    }
}
```

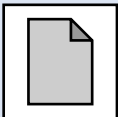
≈ 1ms

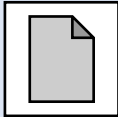
下次的执行时间

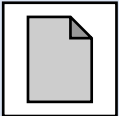
6.3.4. 其它用途

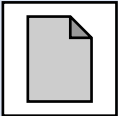


6.4. 调整缓冲区的大小

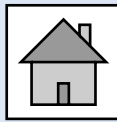
6.4.1. 缓冲区溢出的危害 

6.4.2. 缓冲区占用率的影响因素 

6.4.3. 选择合适的波特率 

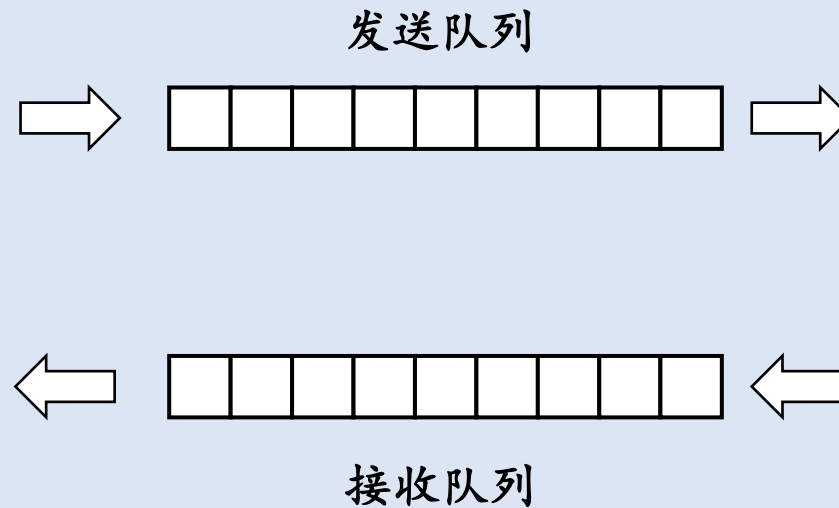
6.4.4. GetBufferMaxOccupancy 

6.4.1. 缓冲区溢出的危害



```
int16_t PAL_USART_SendByte(...)  
{  
    Enqueue(...);  
}
```

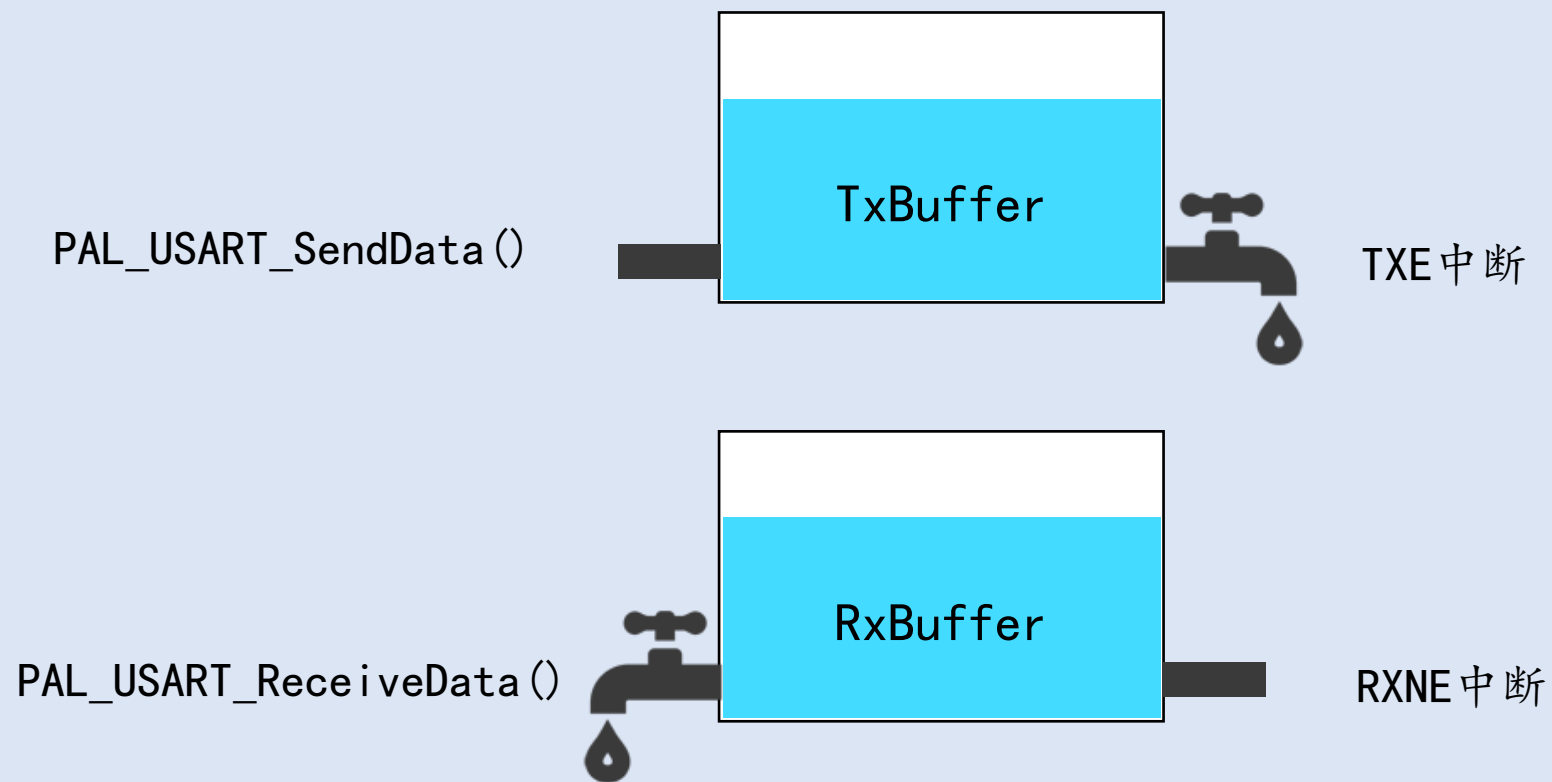
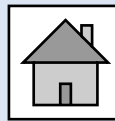
```
int16_t PAL_USART_RecieveByte(...)  
{  
    Deuque(...);  
}
```



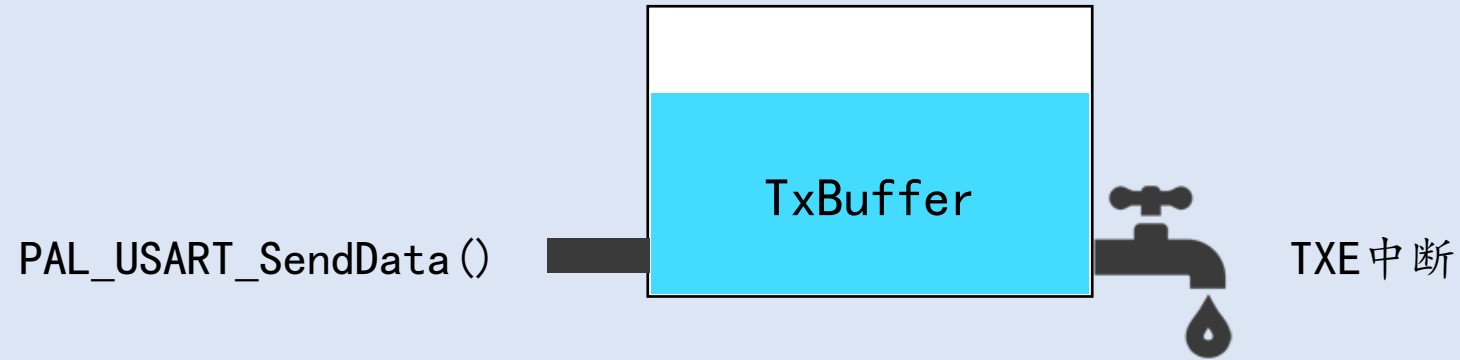
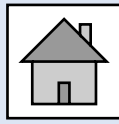
```
PAL_USART_SendString(..., " Hello\r\n" );
```

```
PAL_USART_SendString(..., " Nihao" );
```

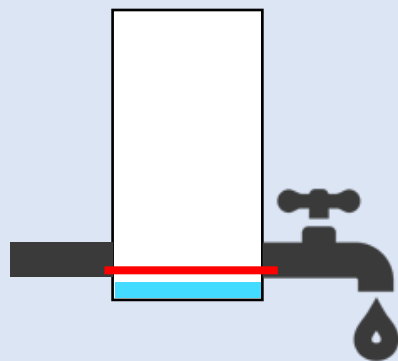
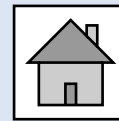
6.4.2. 缓冲区占用率的影响因素



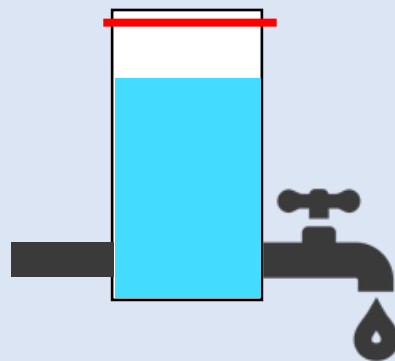
6.4.3. 选择合适的波特率



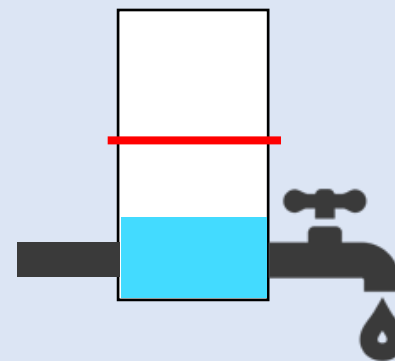
6.4.4. GetBufferMaxOccupancy



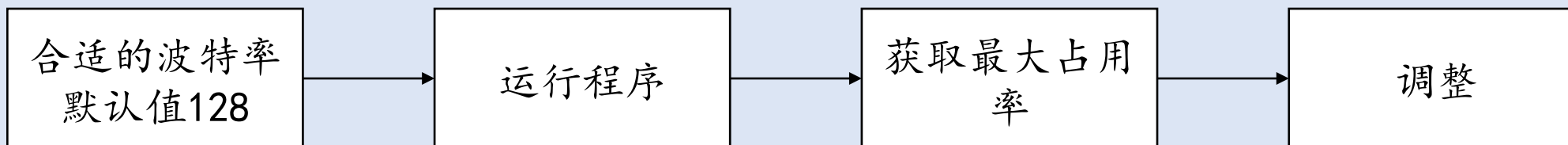
缓冲区利用率不足
浪费RAM资源



缓冲区利用率过高
有溢出的风险



缓冲区设置合理



PAL_USART_GetTxBufferMaxOccupancy(...)

PAL_USART_GetRxBufferMaxOccupancy(...)

6.6. 随堂测试



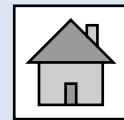
题目1：简述PAL库USART发送/接收的数据流向

题目2：如何使用USART驱动的IO重映射？

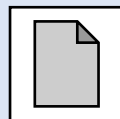
题目3：使用USART发送格式化字符串应当选择哪个编程接口？

题目4：简述Serial Port Plotter软件的数据格式

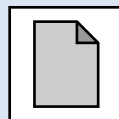
第七节、设备间的指令通信



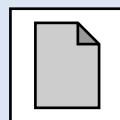
7.1. 指令通信简介



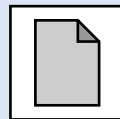
7.2. 指令通信方案概述



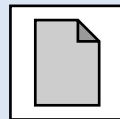
7.3. 单字符指令



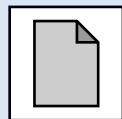
7.4. 字符串指令



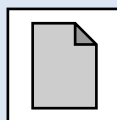
7.5. 二进制指令

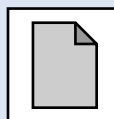


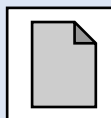
7.6. 随堂测试



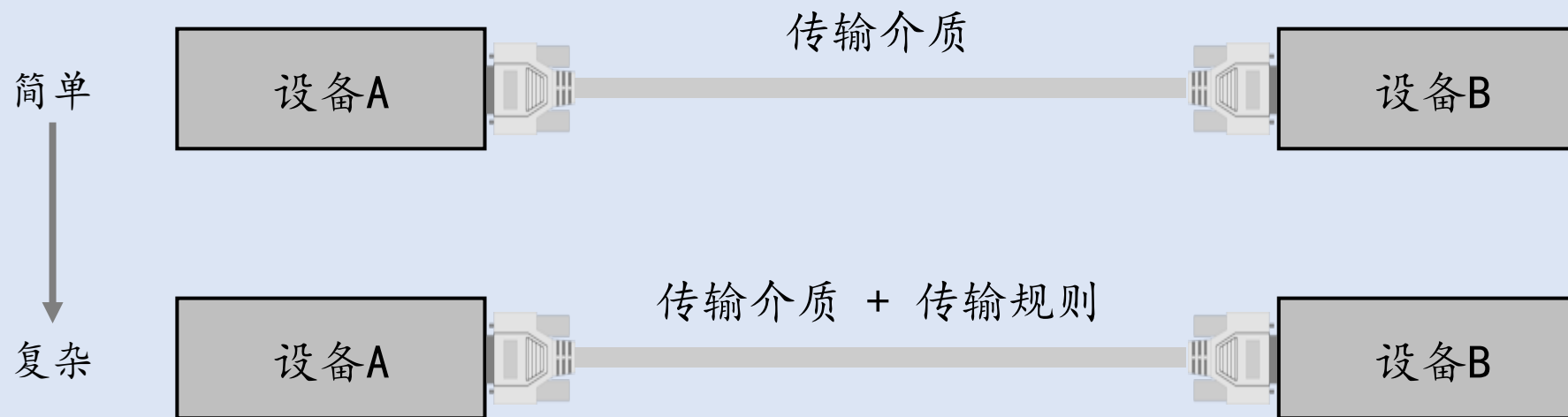
7.1. 指令通信简介

7.1.1. 指令通信的概念 

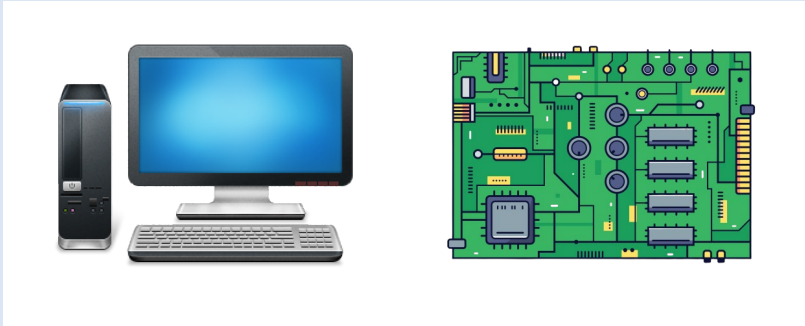
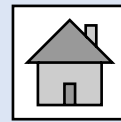
7.1.2. 指令通信的应用场景 

7.1.3. 学习指令通信的必要性 

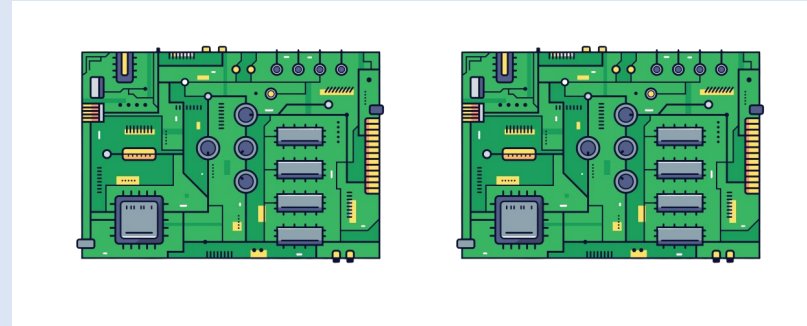
7.1.1. 指令通信的概念



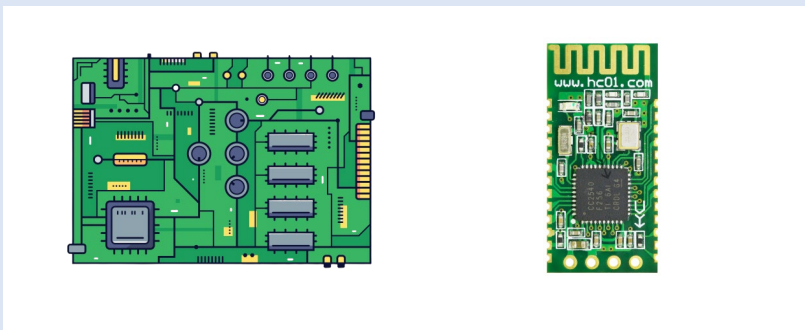
7.1.2. 指令通信的应用场景



上位机与MCU



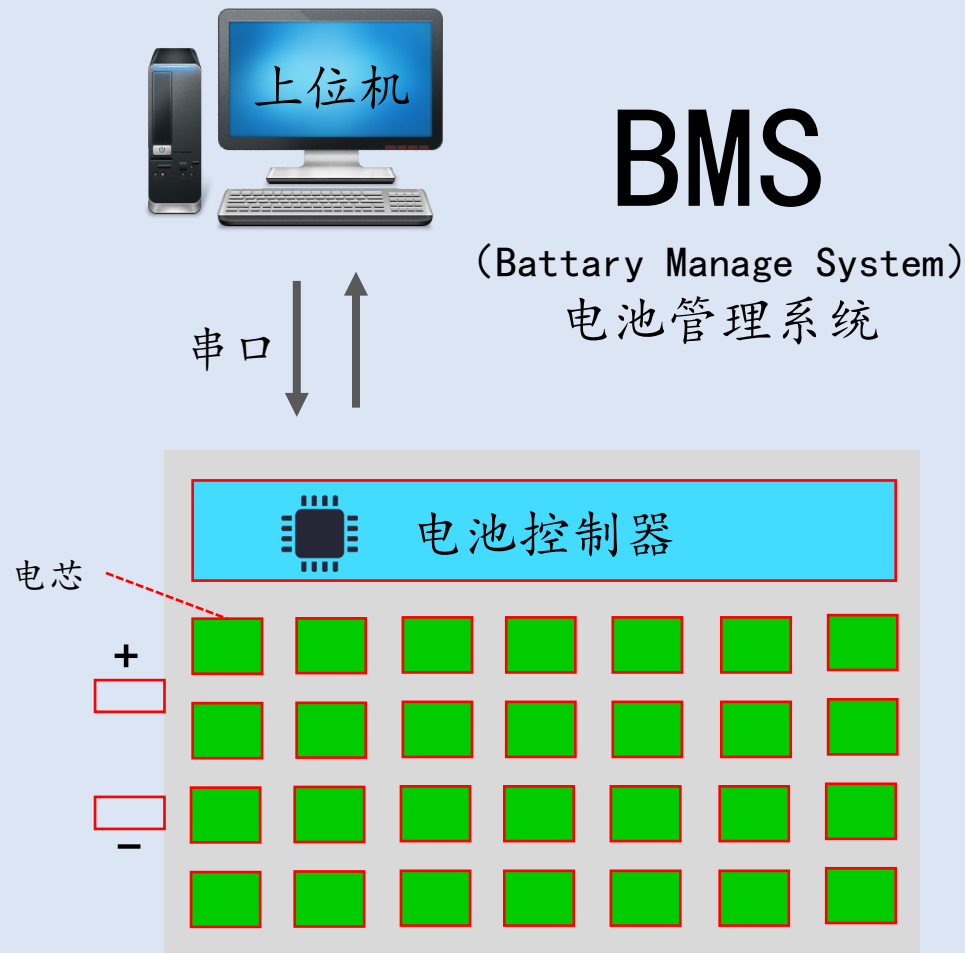
MCU与MCU



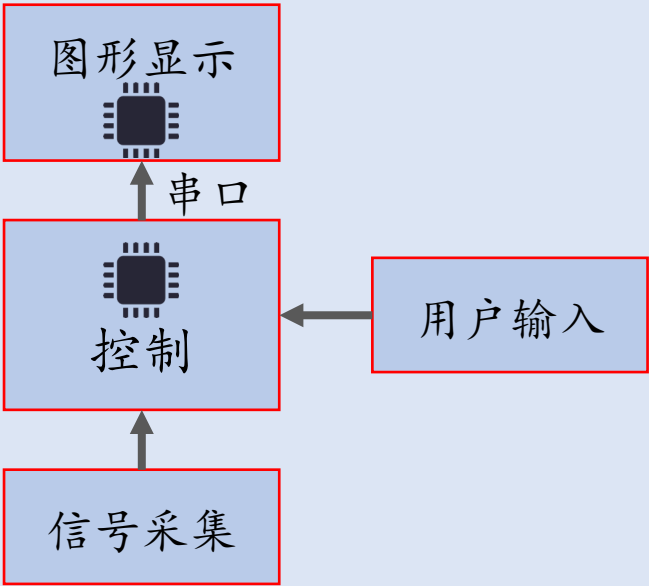
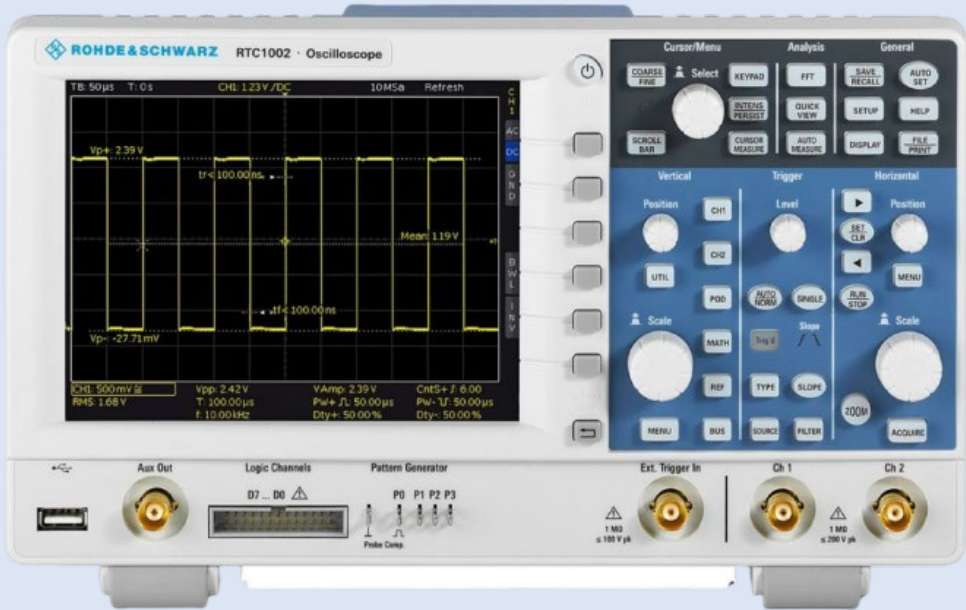
MCU与外部模块

上位机与MCU之间

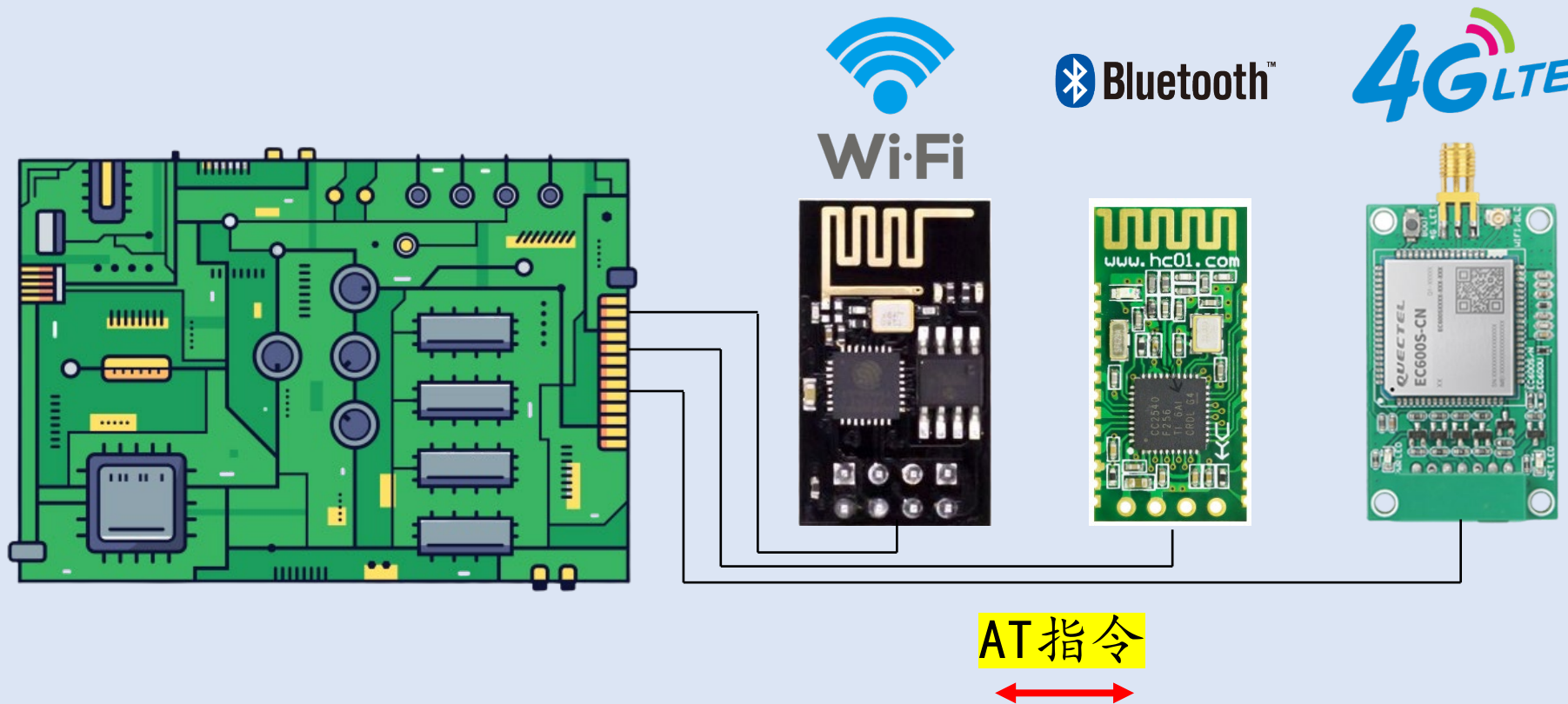
上位机软件



MCU与MCU之间



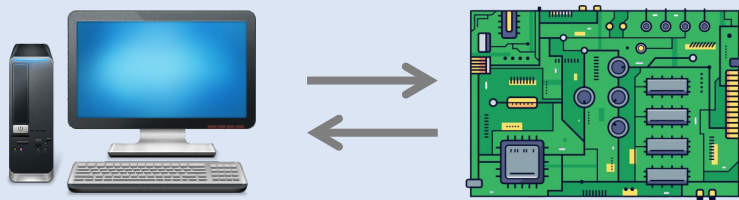
MCU与外部模块之间



7.1.3. 学习指令通信的必要性



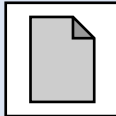
1. 单片机开发离不开指令通信

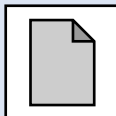


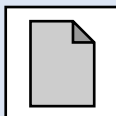
2. 自创的指令格式很难稳定

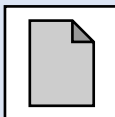
3. 学习的过程可以锻炼自己的编程思维

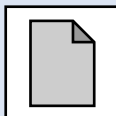
7.2. 指令通信方案概述

7.2.1. 单字符指令 

7.2.2. 字符串指令 

7.2.3. 二进制指令 

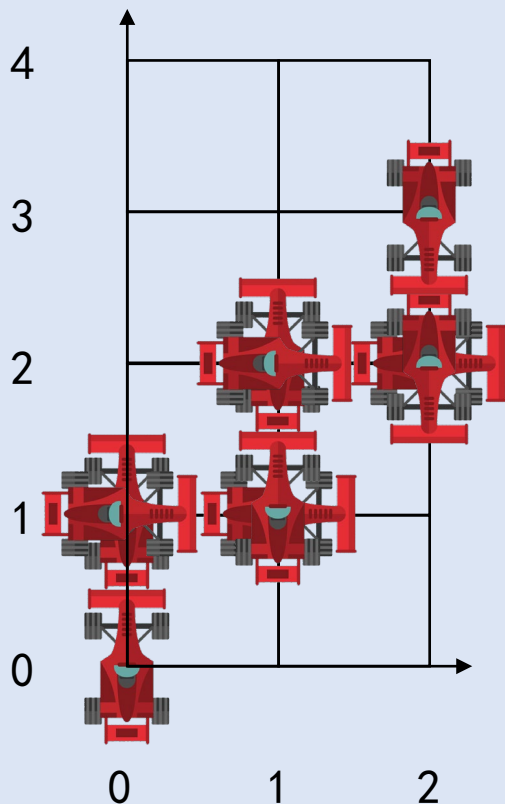
7.2.4. AT指令 

7.2.5. 总结与对比 

7.2.1. 单字符指令



单字符指令：所有的指令使用单个字母表示

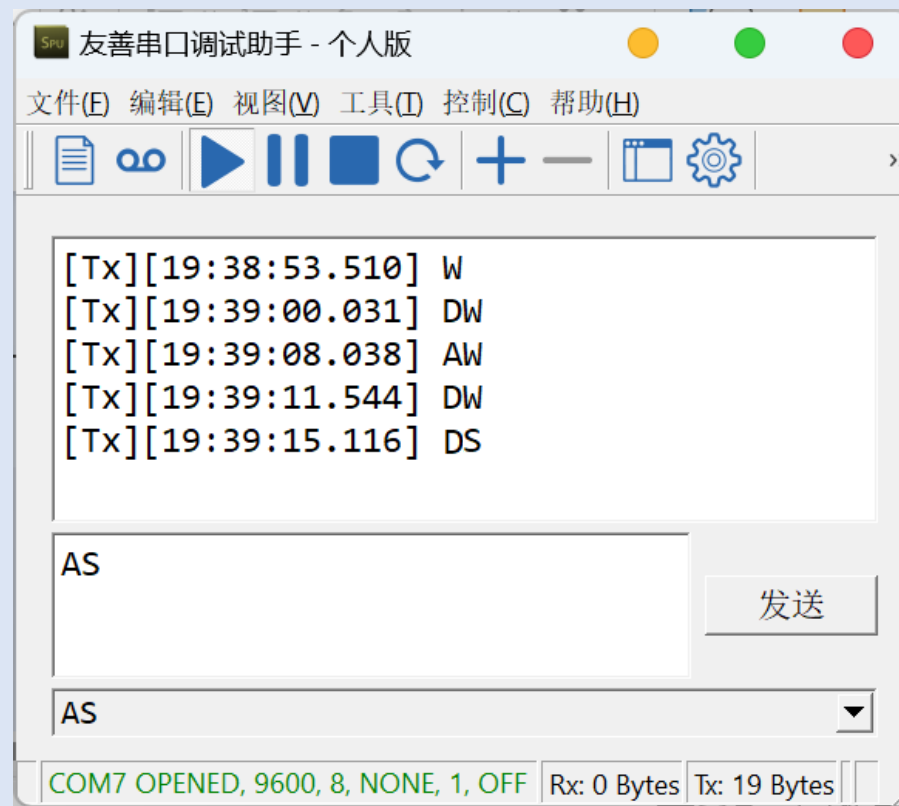
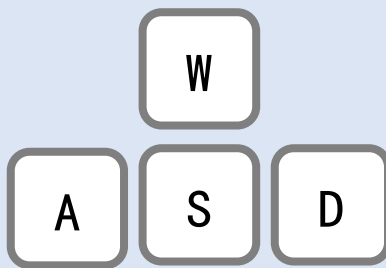


W - 前进1米

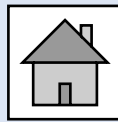
S - 后退1米

A - 左转

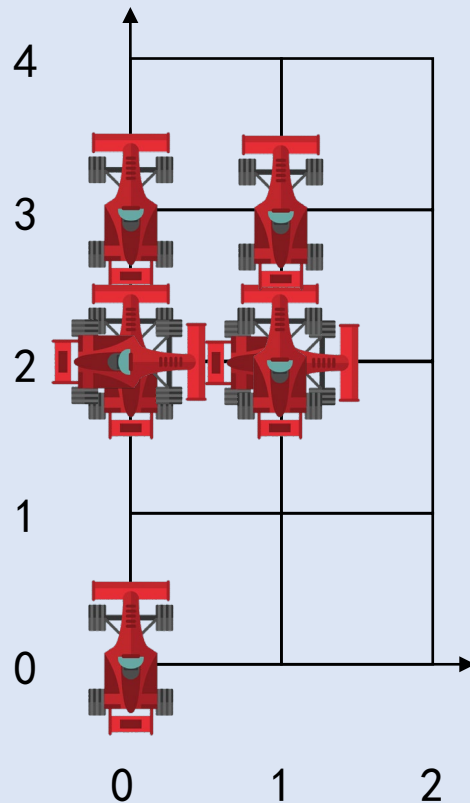
D - 右转



7.2.2. 字符串指令



字符串指令：指令名称 参数1 参数2 ...\r\n



walk [方向] [距离]

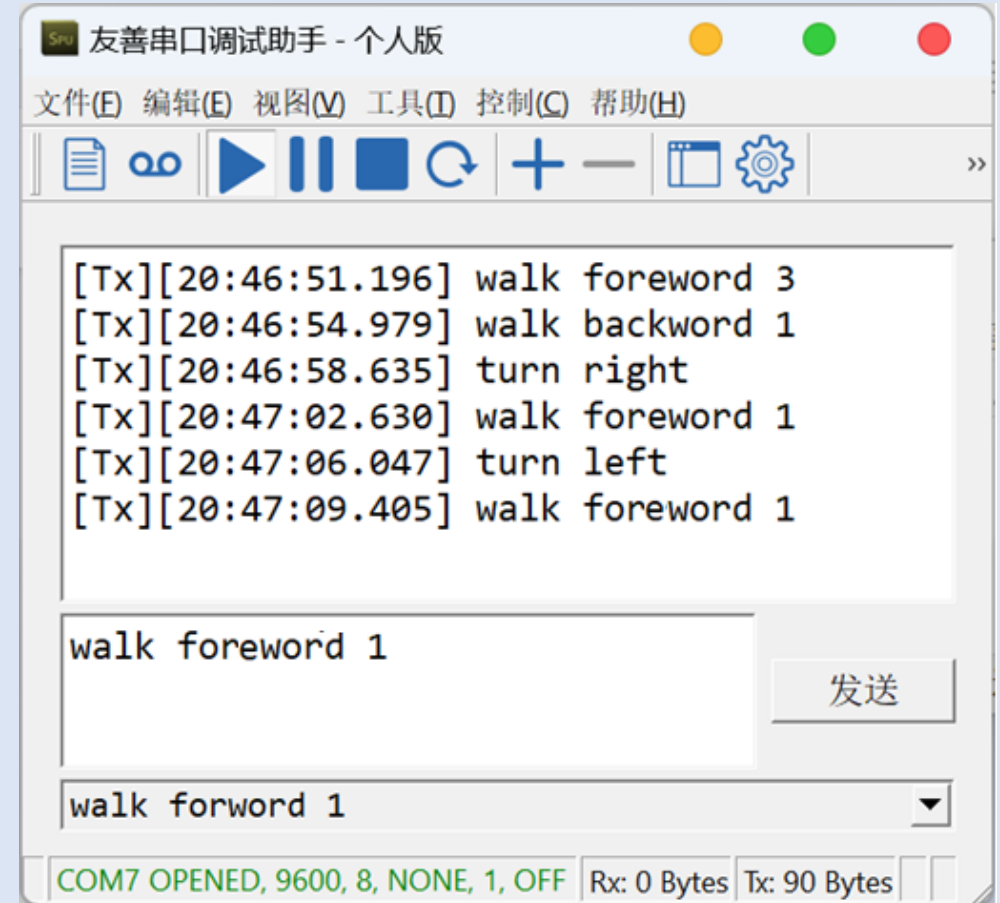
foreword - 前

Backward - 后

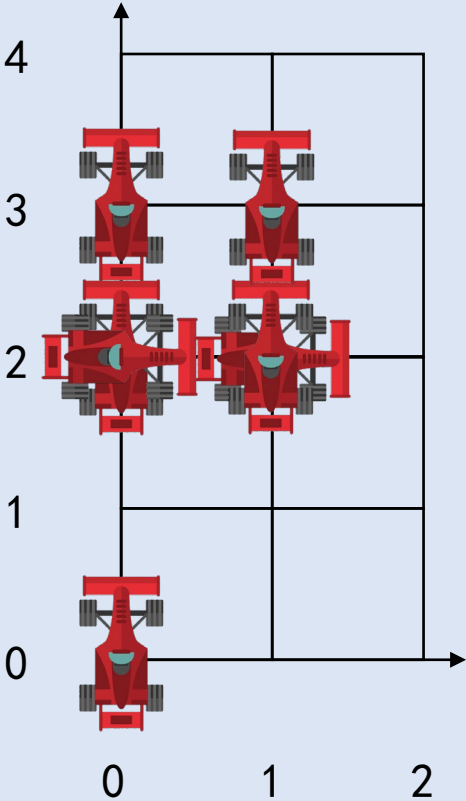
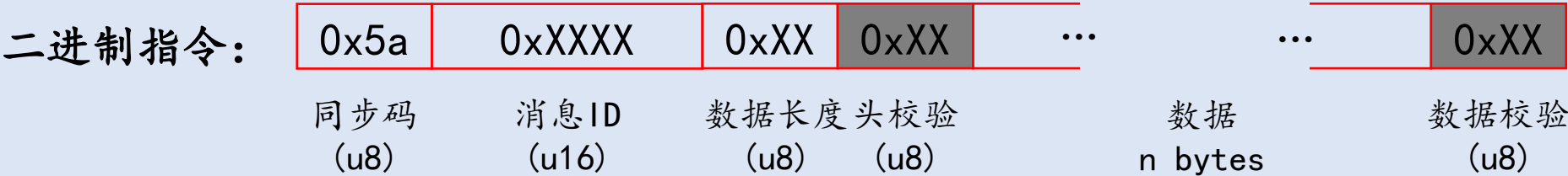
turn [方向]

left - 左

right - 右



7.2.3. 二进制指令

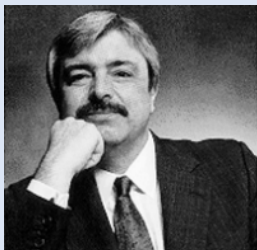


ID	作用	参数
0x0000	行走 (walk)	[DIR(u8)] [DIS(u8)] DIR - 方向, 0x00 - 前, 0x01 - 后 DIS - 距离
0x0001	转弯 (turn)	[DIR(u8)] DIR - 方向, 0x00 - 左, 0x01 - 右

```
walk foreword 3 \r\nwalk backward 1 \r\nturn right \r\nwalk foreword 1 \r\nturn left \r\nwalk foreword 1 \r\n
```

0x5a	0x00	0x00	0x02	0x00	0x03
0x5a	0x00	0x00	0x02	0x01	0x01
0x5a	0x01	0x00	0x01	0x01	
0x5a	0x00	0x00	0x02	0x00	0x01
0x5a	0x01	0x00	0x01	0x00	
0x5a	0x00	0x00	0x02	0x00	0x01

7.2.4. AT指令

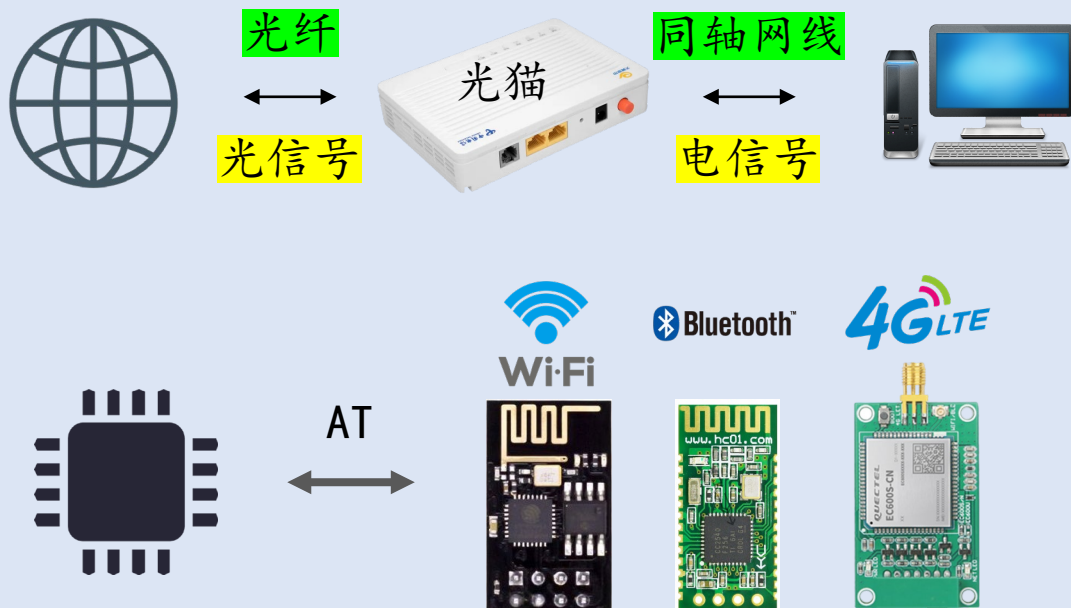


一种字符串格式的指令集
丹尼斯·汉斯在1981年发明



AT是Attention(注意)的缩写
这种指令常以AT开头

首次应用在Hayes Smart 300 MODEM(调制解调器)上



// EC600通信指令示例

[Tx] AT // 测试设备可用性
[Rx] OK

[Tx] AT+CSQ // 查询信号质量
[Rx] +CSQ: 26, 99 // 信号强度, 误码率
[Rx] OK

[Tx] AT+CPIN? // 检查SIM卡状态
[Rx] +CPIN: READY
[Rx] OK

7.2.5. 总结与对比

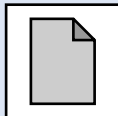


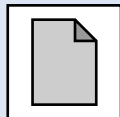
本节课学习

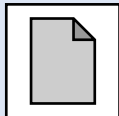
指令方案	格式	优点	缺点	适用场景
单字符	单个字母	简单	不能传输复杂指令	简单调试
字符串	[名称] [参数1] [参数2]... \r\n	便于阅读和记忆	传输效率低 容错率低	用户与设备交互
二进制	[同步码][ID][长度][头校验][数据][数据校验]	安全、高效	不易阅读 不易记忆	设备间交互
AT	AT+xxx\r\n	与字符串指令相同 + 指令处理困难		MODEM或某些通信模块

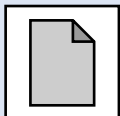
walk foreword 3\r\n 0x5a 0x00 0x00 0x02 0x00 0x00 0x03

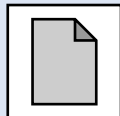
7.3. 单字符指令

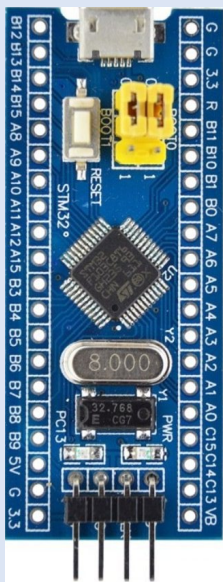
7.3.1. LED闪灯实验 

7.3.2. 闪灯进程的编码思路 

7.3.3. 调节闪灯参数 

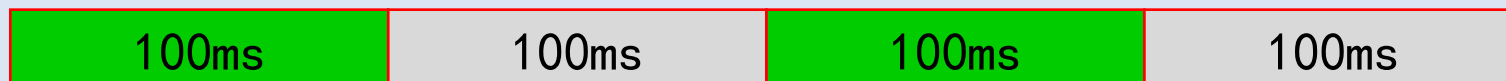
7.3.4. 单字符指令实验 

7.3.5. 指令进程的编码思路 



慢闪

$0_n=100, 0_{ff}=100$



短闪

$0_n=10$, $0_{ff}=100$

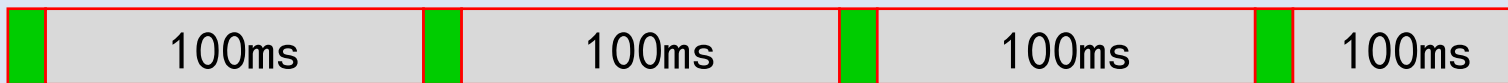


快闪

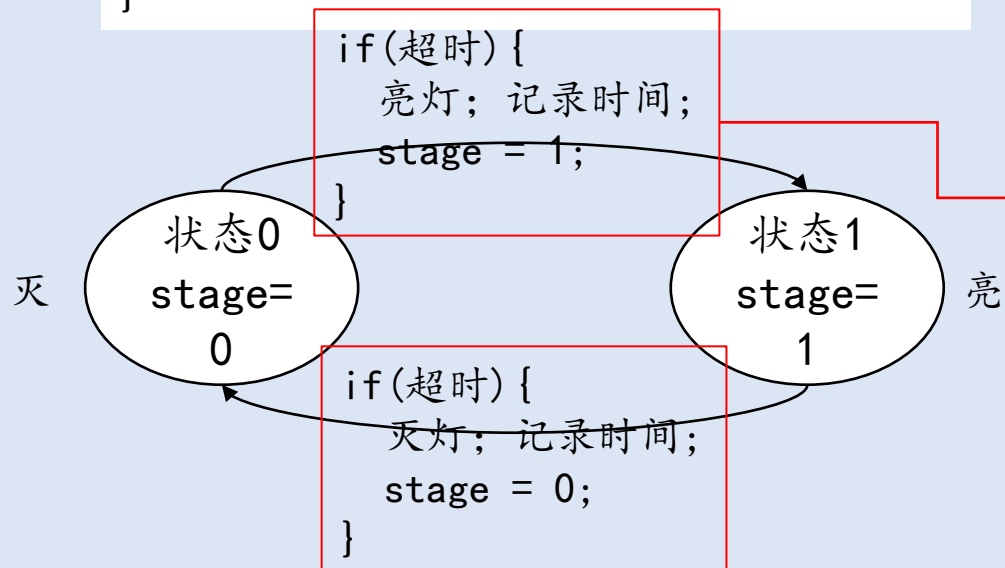
$0_n=50$, $0_{ff}=50$



7.3.2. 闪灯进程的编码思路



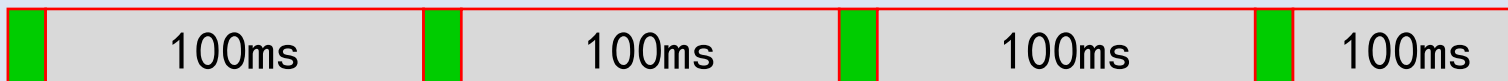
```
#include "app_blinky_led.h"
int main(void) {
    App_BlinkyLED_Init();
    while(1) {
        App_BlinkyLED_Proc();
    }
}
```



 app_blinky_led.c/h

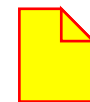
```
static uint32_t onTime = 10;
static uint32_t offTime = 100;
static uint16_t stage = 0; // LED状态: 灭, 亮
static uint64_t lastToggleTime = 0; // 上次切换的时间
void App_BlinkyLED_Proc()
{
    switch(stage) {
        case 0: // stage =0, 灭灯
            if(PAL_GetTick() > lastToggleTime + offTime) {
                GPIO_ResetBits(...);
                lastToggleTime = PAL_GetTick();
                stage = 1;
            }
        case 1: // stage =1, 亮灯
            ...
    }
}
```

7.3.3. 调节闪灯参数



```
#include "app_blinky_led.h"
int main(void) {
    App_BlinkyLED_Init();
    App_BlinkyLED_SetOnTime(10);
    App_BlinkyLED_SetOffTime(1000);
    while(1) {
        App_BlinkyLED_Proc();
    }
}
```

```
static uint32_t onTime = 100;
static uint32_t offTime = 100;
```



app_blinky_led.c/h

// 设置亮灯时间

```
void App_BlinkyLED_SetOnTime(uint32_t Val) {onTime = Val;}
```

// 设置灭灯时间

```
void App_BlinkyLED_SetOffTime(uint32_t Val) {offTime = Val;}
```

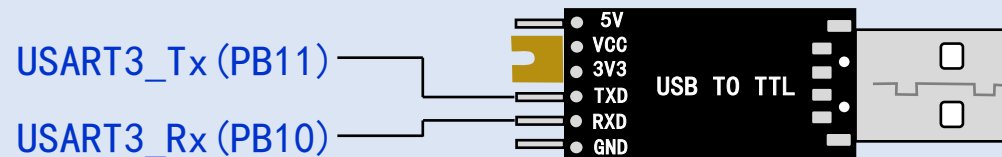
// 读取亮灯时间

```
uint32_t App_BlinkyLED_GetOnTime(void) {return onTime;}
```

// 读取灭灯时间

```
uint32_t App_BlinkyLED_GetOffTime(void) {return OffTime;}
```

7.3.4. 单字符指令实验

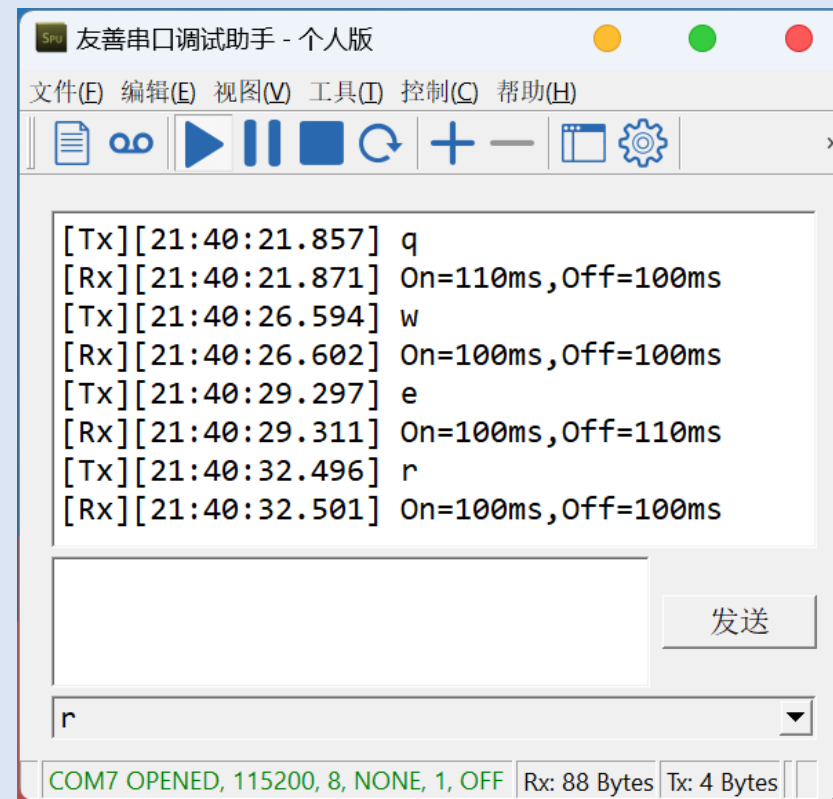


q : 开启时间+10ms

w : 开启时间-10ms

e : 关闭时间+10ms

r : 关闭时间-10ms



7.3.5. 指令进程的编码思路

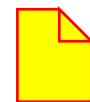


```
#include "app_cmd.h"

int main(void)
{
    App_BlinkyLED_Init();
    App_Cmd_Init();
    while(1) {
        App_BlinkyLED_Proc();
        App_Cmd_Proc();
    }
}
```

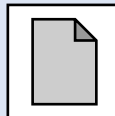
```
void USART3_IRQHandler(void) {}
```

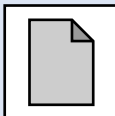
```
void App_Cmd_Proc(void) {
    int16_t tmp = PAL_USART_ReceiveByte(..., 0);
    if(tmp >= 0) {
        char c = (char)tmp;
        switch(c) {
            case 'q' : App_BlinkyLED_SetOnTime(...+10); break;
            case 'w' : App_BlinkyLED_SetOnTime(...-10); break;
            case 'e' : App_BlinkyLED_SetOffTime(...+10); break;
            case 'r' : App_BlinkyLED_SetOffTime(...-10); break;
        }
    }
}
```

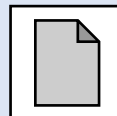


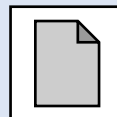
app_cmd.c/h

7.4. 字符串指令

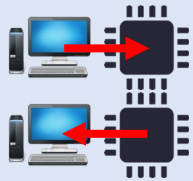
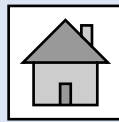
7.4.1. 字符串指令的格式 

7.4.2. 实验介绍 

7.4.3. 使用标准库收发指令 

7.4.4. 使用PAL库收发指令 

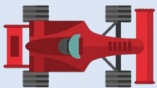
7.4.1. 字符串命令的格式



发送: **【指令名称】 【参数1】 【参数2】 ...\r\n** 各部分内部不允许有空格

接收: **OK\r\n | ERROR, 【错误描述】 \r\n** ← 错误信息

【附加信息】 \r\n ← 附加信息



发: Walk **【方向】 【距离】** \r\n
收: OK\r\n
 \r\n

发: Turn **【方向】** \r\n
收: OK\r\n
 \r\n

发: GetPos\r\n
收: OK\r\n
 【x】 , 【y】 \r\n

例子1: 行走命令

发送: Walk forward 1\r\n
接收: OK\r\n
 \r\n

例子2: 名称错误

发送: Wake forward 1\r\n
接收: ERROR, 未识别的命令 “Wake” \r\n
 \r\n

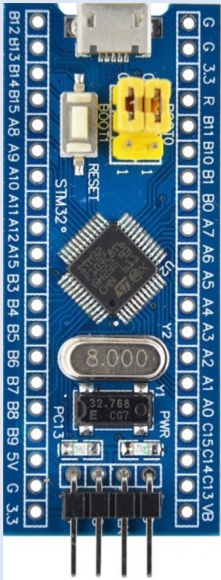
例子3: 参数数量错误

发送: Walk foreword\r\n
接收: ERROR, 参数数量错误\r\n
 \r\n

例子4: 查询当前位置

发送: GetPos\r\n
接收: OK\r\n
 0, 1\r\n

7.4.2. 实验介绍



命令	作用	参数	示例
SetBlinkParam	设置闪灯数值	OnTime - 非负整数 OffTime - 非负整数	SetBlinkParam 10 1000\r\n OK\r\n \r\n
GetBlinkParam	获取闪灯数值	无	GetBlinkParam\r\n OK\r\n On=10, Off=1000\r\n

7.4.3. 使用标准库收发指令

7.4.3.1. 流程拆解与分析 

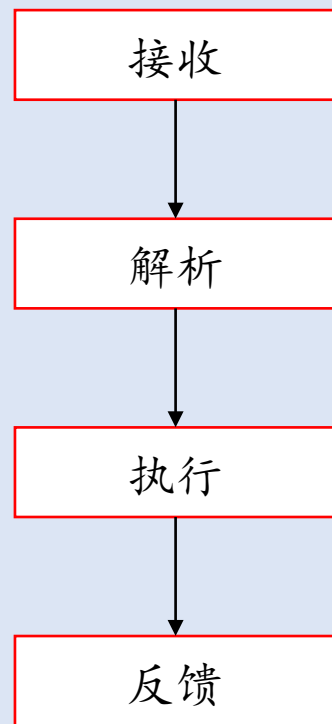
7.4.3.2. 程序的整体结构 

7.4.3.3. 字符串指令的接收 

7.4.3.4. 预处理 

7.4.3.5. 解析指令名称 

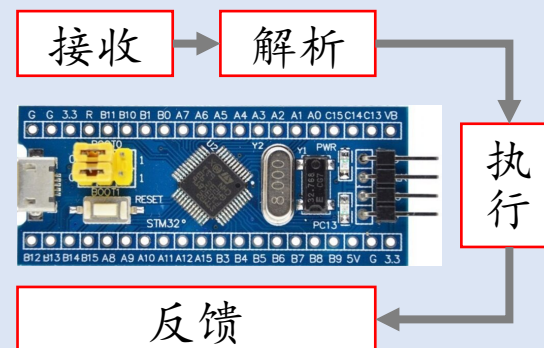
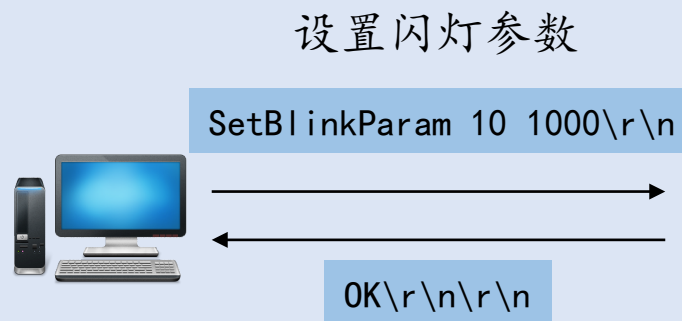
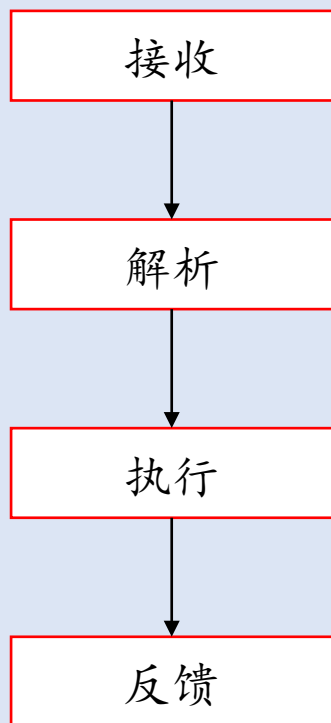
7.4.3.6. 解析参数 



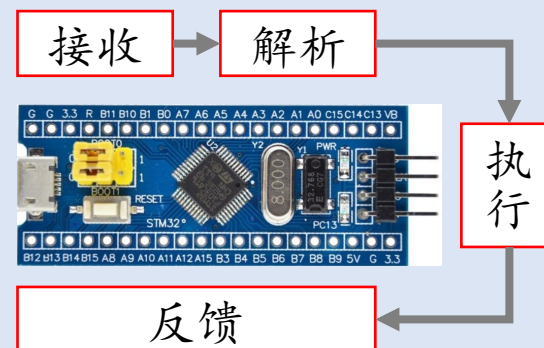
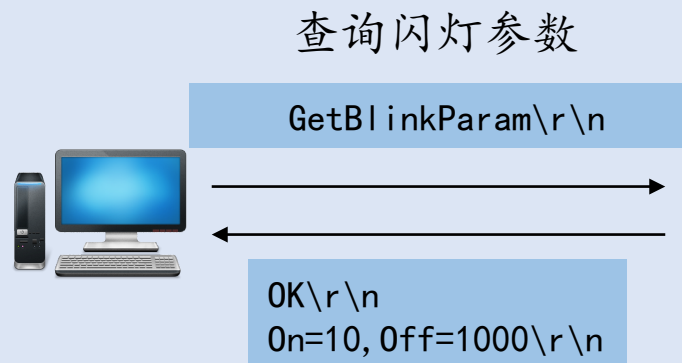
分割提取有效信息

SetBlinkParam 10 1000\r\n

7.4.3.1. 流程拆解与分析

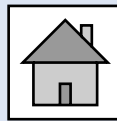


App_BlinkyLED_SetOnTime(...)
App_BlinkyLED_SetOffTime(...)



App_BlinkyLED_GetOnTime(...)
App_BlinkyLED_GetOffTime(...)

7.4.3.2. 程序的整体结构



main.c

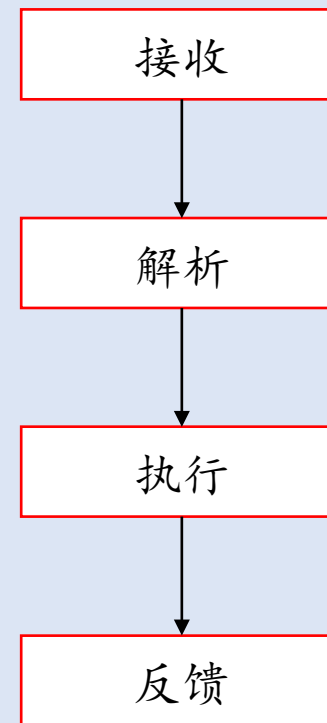
```
int main(void) {  
    App_BlinkyLED_Init();  
    App_Cmd_Init();  
    while(1) {  
        App_BlinkyLED_Proc();  
        App_Cmd_Proc();  
    }  
}  
void USART3_IRQHandler() {}
```



app_blinky_led.c/h



app_cmd.c/h



7.4.3.3. 字符串指令的接收

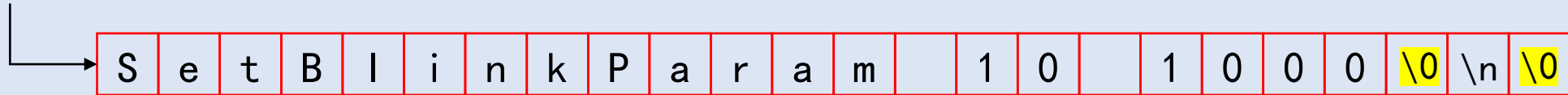
【字符串指令格式】 指令名称 参数1 参数2 ...\r\n

```
void App_Cmd_Proc(void) {  
    char cmdLine[64];  
    if (PAL_USART_ReadLine(···, cmdLine, 0) > 0) {  
  
    }  
}
```

7.4.3.4. 预处理



cmdLine



```
void App_Cmd_Proc(void) {  
    char cmdLine[64];  
    if (PAL_USART_ReadLine(..., cmdLine) > 0) {  
        cmdLine[strlen(cmdLine)-2] = '\0'; // 删除掉\r\n  
    }  
}
```


7.4.3.5. 解析指令名称

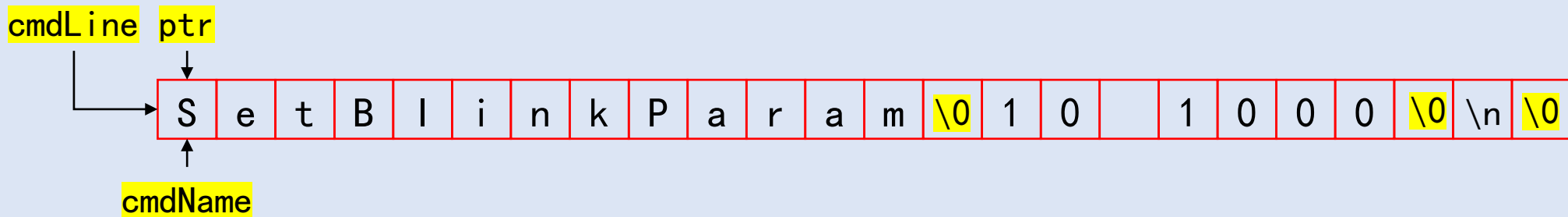


【字符串指令格式】 指令名称 参数1 参数2 ...\r\n

```
// @文件: string.h
// @简介: 在字符串中查找某个字符的位置
// @参数: str - 被查找的字符串
// @参数: c   - 要查找的字符
// @返回值: 字符所在位置的指针, 或0
char *strchr(const char *str, int c);

// @简介: 对str1和str2比较, 相等则返回0
int strcmp(const char *str1, const char *str2);
```

```
char cmdLine[64]; // 接收到的指令字符串
char *ptr = cmdLine;
const char *cmdName = ptr; // 指令名称
ptr = strchr(cmdLine, ' '); // 查找空格
if(ptr!=0) { *ptr= '\0' ;ptr++; } // 写入\0
// 根据指令名称作相应的处理
if(strcmp(cmdName, " SetBlinkParam" )==0) {...}
else
if(strcmp(cmdName, " GetBlinkParam" )==0) {...}
else {...} // ERROR, Unsupported cmd %s\r\n\r\n
```



7.4.3.6. 解析参数

【字符串指令格式】指令名称 参数1 参数2 ...\r\n

```
// @文件: string.h
// @简介: 在字符串中查找某个字符的位置
// @参数: str - 被查找的字符串
// @参数: c   - 要查找的字符
// @返回值: 字符所在位置的指针, 或0
char *strchr(const char *str, int c);

// @简介: 将字符串转化为整数
int atoi(const char *str);
```

```
const char *arg1,*arg2;
```

```
// 解析参数1
```

```
arg1=ptr;
```

```
ptr=strchr(ptr, ' ');
```

```
*ptr= '\0' ;
```

```
ptr++;
```

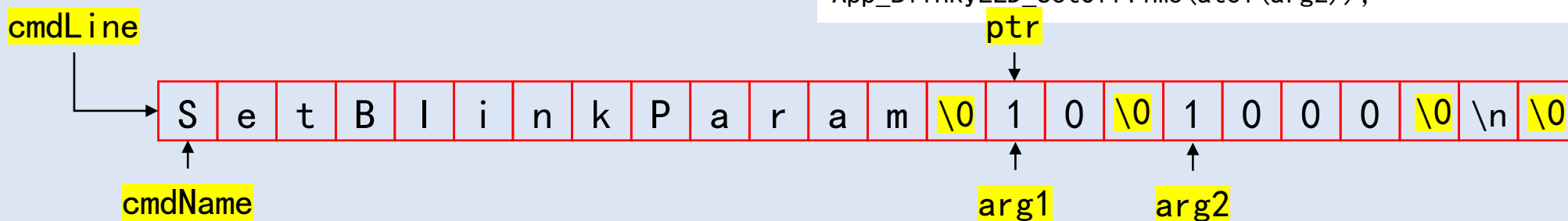
```
// 解析参数2
```

```
arg2=ptr;
```

```
// 执行
```

```
App_BlinkyLED_SetOnTime(atoi(arg1));
```

```
App_BlinkyLED_SetOffTime(atoi(arg2));
```



7.4.4. 使用PAL库收发字符串指令

7.4.4.1. 编程思路

7.4.4.2. 字符串指令编解码器

7.4.4.3. 初始化编解码器

7.4.4.4. 接收指令

7.4.4.5. 获取指令名称和参数

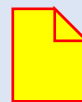
7.4.4.6. 改进

7.4.4.1. 编程思路



```
#include "app_blink_led.h"
#include "app_cmd.h"
int main(void) {
    App_BlinkyLED_Init();
    App_Cmd_Init();
    while(1) {
        App_BlinkyLED_Proc();
        App_Cmd_Proc();
    }
}
```

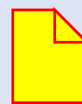
```
void USART3_IRQHandler(void) {}
```



main.c

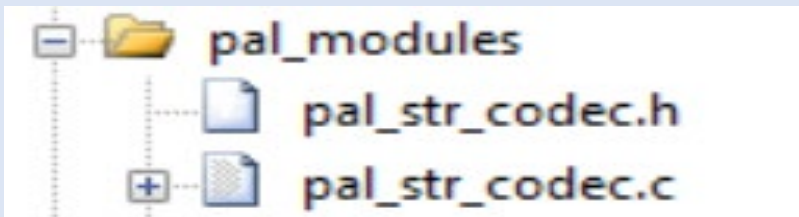


app_blink_led.c/h



app_cmd.c/h

7.4.4.2. 字符串指令编解码器



字符串指令编解码器

解析

初始化

接收

获取
参数

```
// @简介: 初始化字符串命令编解码器
void PAL_StrCodec_Init(...);
// @简介: 从串口接收命令行
ErrorStatus PAL_StrCodec_Receive(...);
// @简介: 获取参数的个数
uint16_t PAL_StrCodec_GetNumberOfArgs(...);
// @简介: 获取命令的名称字符串
const char *PAL_StrCodec_GetName(...);
// @简介: 获取参数字符串
const char *PAL_StrCodec_ReadArgStr(...);
// @简介: 获取参数, 并将其转化为int类型
ErrorStatus PAL_StrCodec_ReadArgInt(...);
// @简介: 获取参数, 并将其转化为float类型
ErrorStatus PAL_StrCodec_ReadArgFloat(...);
```

SetBlinkParam 10 1000\r\n

const char *cmdName

uint32_t offTime

uint32_t onTime

①

初始化

②

接收指令

获取指令的
名称和参数

名称
参数个数
参数

7.4.4.3. 初始化编解码器



// @简介: 初始化字符串命令编解码器

```
void PAL_StrCodec_Init(PalStrCodec_HandleTypeDef *Handle);
```

```
static PalUSART_HandleTypeDef hUSART3;
```

← 串口句柄

```
static PalStrCodec_HandleTypeDef hStrCodec;
```

← 编解码器句柄

```
void App_Cmd_Init(void) {
```

```
    // 初始化串口
```

```
    hUSART3.Init.Baudrate=115200;
```

```
    ...
```

```
    hUSART3.Init.LineSeparator=LineSeparator_CRLF;
```

```
    PAL_USART_Init(&hUSART3);
```

← 初始化串口

```
    // 初始化编解码器
```

```
    hStrCodec.Init.hUSART = &hUSART3;
```

```
    PAL_StrCodec_Init(&hStrCodec);
```

← 初始化编解码器

```
}
```

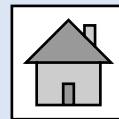
7.4.4.4. 接收指令



```
// @简介: 从串口接收命令行 (超时值为0)  
// @返回值: SUCCESS-接收到有效命令行 ERROR-未接收到有效命令行  
ErrorStatus PAL_StrCodec_Receive(PalStrCodec_HandleTypeDef *Handle);
```

```
void App_Cmd_Porc(void) {  
    if (PAL_StrCodec_Receive(&hStrCodec) == SUCCESS)  
    {  
        // 对接收到的指令进行处理  
        // ...  
    }  
}
```

7.4.4.5. 获取指令的名称和参数



// @简介: 获取参数的个数

```
uint16_t PAL_StrCodec_GetNumberOfArgs(PalStrCodec_HandleTypeDef *Handle);
```

// @简介: 获取命令的名称字符串

```
const char *PAL_StrCodec_GetName(PalStrCodec_HandleTypeDef *Handle);
```

// @简介: 读取参数

```
const char *PAL_StrCodec_ReadArgStr(PalStrCodec_HandleTypeDef *Handle, uint16_t Index);
```

字符串

```
ErrorStatus PAL_StrCodec_ReadArgInt(PalStrCodec_HandleTypeDef *Handle, uint16_t Index, int32_t *pValOut);
```

整数

```
ErrorStatus PAL_StrCodec_ReadArgFloat(PalStrCodec_HandleTypeDef *Handle, uint16_t Index, float *pValOut);
```

浮点数 (小数)

```
void App_Cmd_Porc(void) {
```

```
    if (PAL_StrCodec_Receive(&hStrCodec) == SUCCESS) {
```

```
        // 对接收到的指令进行处理
```

```
        const char *cmdName = PAL_StrCodec_GetName(&hStrCodec);
```

```
        if (strcmp(cmdName, "SetBlinkParam") == 0) {
```

```
            int32_t onTime, offTime;
```

```
            PAL_StrCodec_ReadArgInt(&hStrCodec, 0, &onTime);
```

```
            PAL_StrCodec_ReadArgInt(&hStrCodec, 1, &offTime);
```

```
            App_BlinkyLED_SetOnTime(onTime);
```

```
            App_BlinkyLED_SetOffTime(offTime);
```

```
        } else if (strcmp(cmdName, "GetBlinkParam") == 0) {}
```

SetBlinkParam 10 1000\r\n

7.4.4.6. 改进



```
if(strcmp(cmdName, " SetBlinkParam" )==0) {  
}else  
if(strcmp(cmdName, " GetBlinkParam" )==0) {  
}else {// 打印错误信息}
```

```
if(PAL_StrCodec_GetNumberOfArgs(&hStrCodec) !=2) {  
    //打印错误信息  
}
```

```
int32_t onTime, offTime;  
if(PAL_StrCodec_ReadArgInt(..., 0, &onTime) ==ERROR) {}  
if(PAL_StrCodec_ReadArgInt(..., 1, &offTime) ==ERROR) {}
```

// 指令名称错误

SeatBlinkParam 10 1000\r\n

ERROR, unsupported cmd name\r\n\r\n

// 指令数量错误

SetBlinkParam 10\r\n

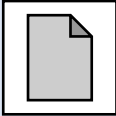
ERROR, incorrect num of args\r\n\r\n

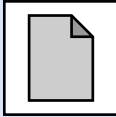
// 指令格式错误

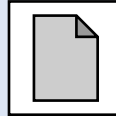
SetBlinkParam 10.5 1000\r\n

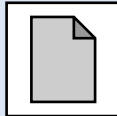
ERROR, invalid arg\r\n\r\n

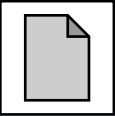
7.5. 二进制指令

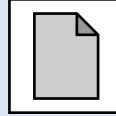
7.5.1. 二进制指令的格式 

7.5.2. 实验介绍 

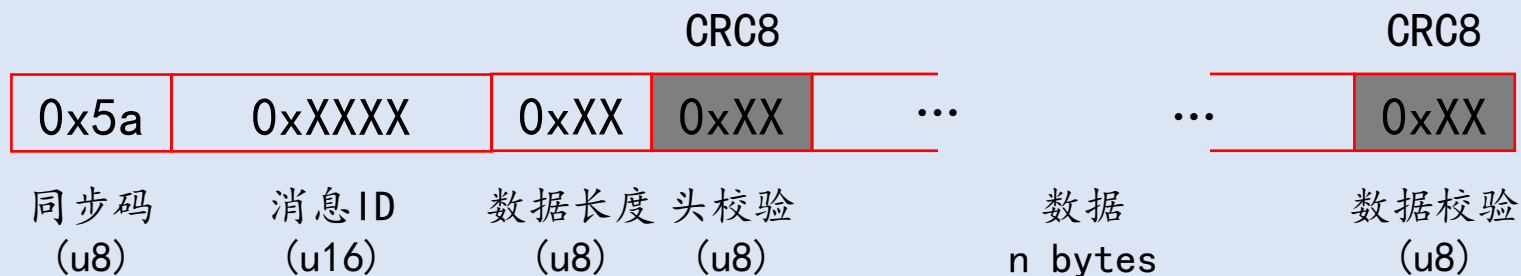
7.5.3. 二进制指令的处理流程 

7.5.4. 状态机与指令解析 

7.5.5. 数据类型转换 

7.5.6. 使用PAL库收发二进制指令 

7.5.1. 二进制指令的格式



同步码 - u8, 表示二进制数据包的开始, 接收方解码时使用

消息ID - u16, 用来区别消息的类型 (0~0xffff)

数据长度 - u8, 数据部分的字节数量 (最长64字节)

头校验码 - u8, CRC8, 接收方使用此校验码检验包头是否正确

数据 - n*u8, 有效数据, 一般用于存储参数

数据校验 - u8, CRC8, 接收方使用此校验码检验数据部分是否正确

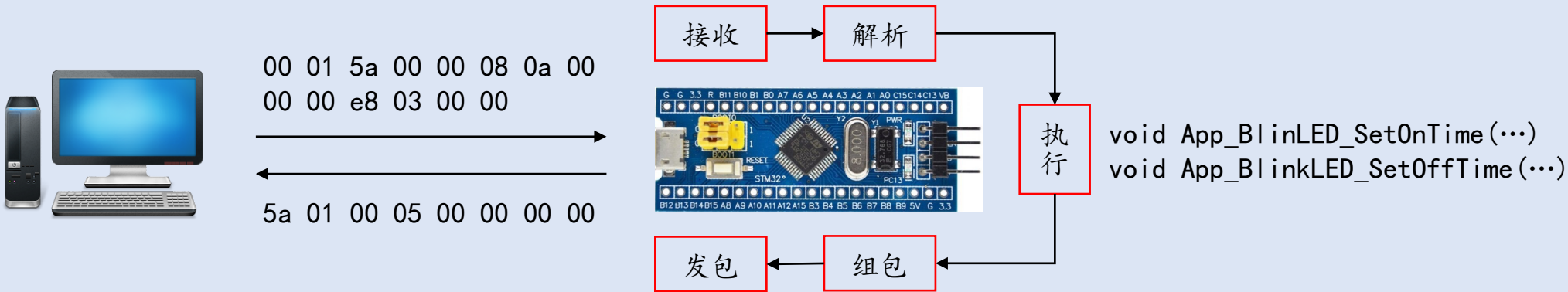
7.5.2. 实验介绍



ID	作用	参数	示例
0x0000	设置闪灯参数 (SetBlinkParam)	[OnTime:u32] [OffTime:u32]	5a 00 00 08 0a 00 00 00 e8 03 00 00
0x0001	ACK (Acknowledge)	[ACK(0)/NAK(1):u8] [ErrorCode:u32]	5a 01 00 05 00 00 00 00 00
0x0002	读取闪灯参数 (GetBlinkParam)	空	5a 02 00 00
0x0003	回复闪灯参数 (Rsp:GetBlinkParam)	[OnTime:u32] [OffTime:u32]	5a 03 00 08 0a 00 00 00 e8 03 00 00

7.5.3. 二进制指令的处理流程

ID	作用	参数	示例
0x0000	设置闪灯参数 (SetBlinkParam)	[OnTime:u32] [OffTime:u32]	5a 00 00 08 0a 00 00 00 e8 03 00 00
0x0001	ACK (Acknowledge)	[ACK(0)/NAK(1):u8] [ErrorC ode:u32]	5a 01 00 05 00 00 00 00 00
0x0002	读取闪灯参数 (GetBlinkParam)	空	5a 02 00 00
0x0003	回复闪灯参数 (Rsp:GetBlinkParam)	[OnTime:u32] [OffTime:u32]	5a 03 00 08 0a 00 00 00 e8 03 00 00

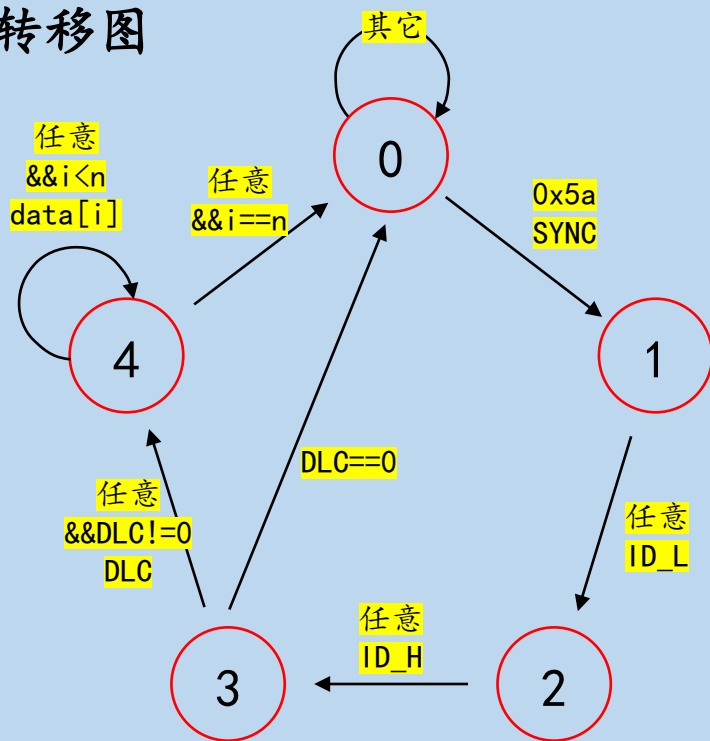


7.5.4. 状态机与指令解析



阶段0 等0x5a	阶段1 等ID_L	阶段2 等ID_H	阶段3 等DLC	阶段4 接收n个数据
0x5a	0xXXXX	0xXX	...	...
同步码 (u8)	消息ID (u16)	数据长度 (u8)	数据 n bytes	

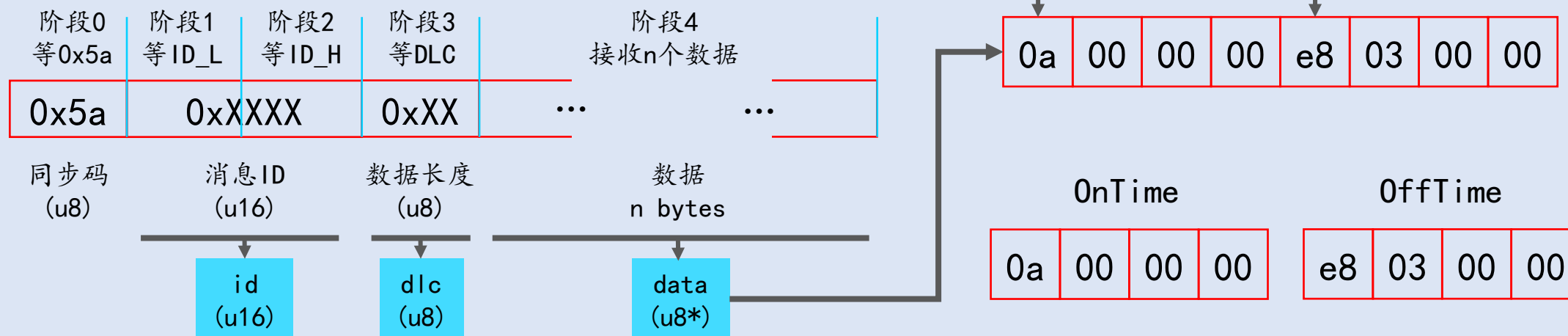
状态转移图



```
uint16_t stage=0; uint16_t id; uint8_t dlc;
uint8_t data[64]; uint16_t i;
```

```
void App_Cmd_Proc(void) {
    int16_t c;
    // 一次性把缓冲区里的数据全部取出
    while((c=PAL_USART_ReceiveByte(...))>0) {
        switch(stage)
        {
            case 0: if(c==0x5a) stage=1; break;
            case 1: id =c;          stage=2; break;
            case 2: id|=c<<8;      stage=3; break;
            case 3: dlc=c;
                    if(dlc!=0) {i=0; stage=4;}
                    else {stage=0;...// 处理消息}
                    break;
            case 4: data[i++]=c;
                    if(i==dlc-1) { stage=0;... // 处理消息}
                    break;
        }
    }
}
```

7.5.5. 数据类型转换



方法1

```
uint32_t onTime, offTime;  
onTime=0;  
onTime+=data[0]*1  
onTime+=data[1]*256^1  
onTime+=data[2]*256^2  
onTime+=data[3]*256^3  
  
offTime=0;  
offTime+=data[4]*1  
offTime+=data[5]*256^1  
offTime+=data[6]*256^2  
offTime+=data[7]*256^3
```

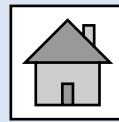
方法2

```
uint32_t *p1, *p2;  
uint32_t onTime, offTime;  
p1=(uint32_t *)&data[0];  
p2=(uint32_t *)&data[3];  
  
onTime=*p1;  
offTime=*p2;
```

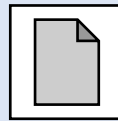
方法2精简

```
uint32_t onTime, offTime;  
onTime =  
    *(uint32_t*)&data[0];  
offTime =  
    *(uint32_t*)&data[3];
```

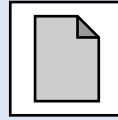
7.5.6. 使用PAL库收发二进制指令



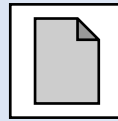
7.5.6.1. 编程接口



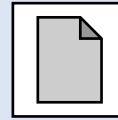
7.5.6.2. 编程思路



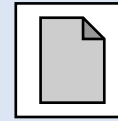
7.5.6.3. 初始化



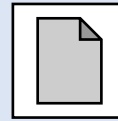
7.5.6.4. 数据包的接收原理



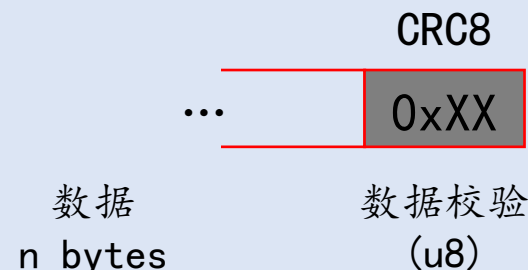
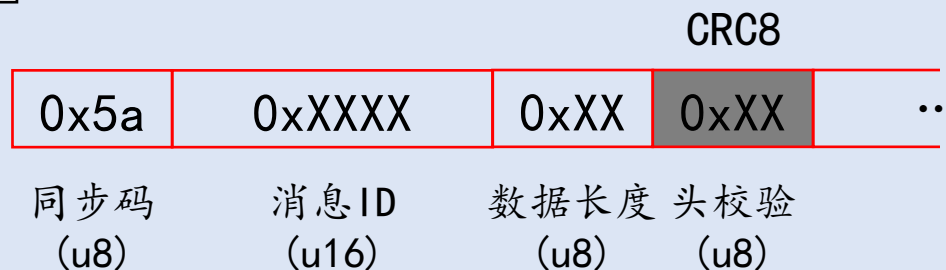
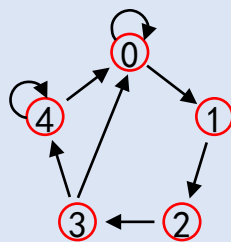
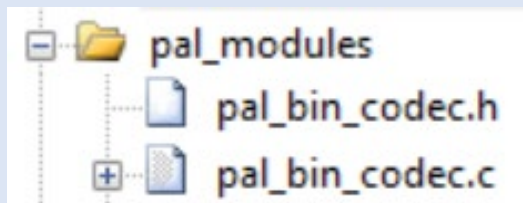
7.5.6.5. 指令处理进程



7.5.6.6. 数据包的发送原理



7.5.6.1. 编程接口



初始化

```
void PAL_BinCodec_Init(PalBinCodec_HandleTypeDef *Handle);
```

解码

```
// @简介: 接收消息
void PAL_BinCodec_Receive(...);
// @简介: 获取数据长度
uint16_t PAL_BinCodec_GetDataLength(...);
// @简介: 获取数据crc校验
FlagStatus PAL_BinCodec_GetDataCRC(...);
// @简介: 读取各种类型的数据
uint8_t PAL_BinCodec_ReadByte(...); // 字节
... PAL_BinCodec_ReadBytes(...); // 多字节
...
uint32_t PAL_BinCodec_ReadUInt32(...); // UInt32
float PAL_BinCodec_ReadFloat(...); // 单浮点数
double PAL_BinCodec_ReadDouble(...); // 双浮点数
const char *PAL_BinCodec_ReadString(...); // 字符串
```

编码

```
// @简介: 开始发送
void PAL_BinCodec_StartSend(...);
// @简介: 向参数中写入各种类型的数据
void PAL_BinCodec_WriteByte(..., uint8_t);
void PAL_BinCodec_WriteBytes(..., uint8_t[]);
...
void PAL_BinCodec_WriteUInt32(..., uint32_t);
void PAL_BinCodec_WriteFloat(..., float);
void PAL_BinCodec_WriteDouble(..., double);
void PAL_BinCodec_WriteString(..., const char *);
// @简介: 将二进制数据包发送出去
void PAL_BinCodec_EndSend(...);
```

* 支持的参数类型: byte bytes u8 u16 u32 u64 i8 i16 i32 i64 float double string

7.5.6.2. 编程思路

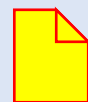


```
#include "app_blink_led.h"
#include "app_bin_cmd.h"
int main(void) {
    App_BlinkyLED_Init();
    App_BinCmd_Init();
    while(1) {
        App_BlinkyLED_Proc();
        App_BinCmd_Proc();
    }
}
```

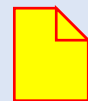
```
void USART3_IRQHandler(void) {}
```



main.c



app_blink_led.c/h



app_bin_cmd.c/h

7.5.6.3. 初始化



```
#include "app_blink_led.h"
#include "app_bin_cmd.h"
int main(void)
{
    App_BlinkyLED_Init();
    App_BinCmd_Init();
    while(1) {
        App_BlinkyLED_Proc();
        App_BinCmd_Proc();
    }
}
```

// @简介: 初始化字符串命令编解码器

```
void PAL_BinCodec_Init(PalBinCodec_HandleTypeDef *Handle);
```

```
static PalUSART_HandleTypeDef hUSART3;
```

```
static PalStrCodec_HandleTypeDef hStrCodec;
```

```
void App_BinCmd_Init(void) {
```

```
    // 初始化串口
```

```
    hUSART3.Init.Baudrate=115200;
```

```
    ...
```

```
    hUSART3.Init.LineSeparator=LineSeparator_CRLF;
```

```
    PAL_USART_Init(&hUSART3);
```

```
    // 初始化编解码器
```

```
    hStrCodec.hUSART = hUSART3;
```

```
    PAL_BinCodec_Init(&hStrCodec);
```

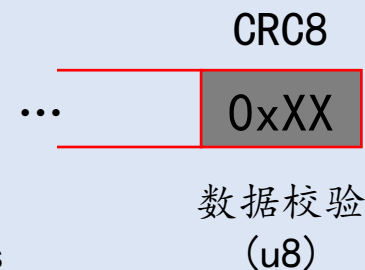
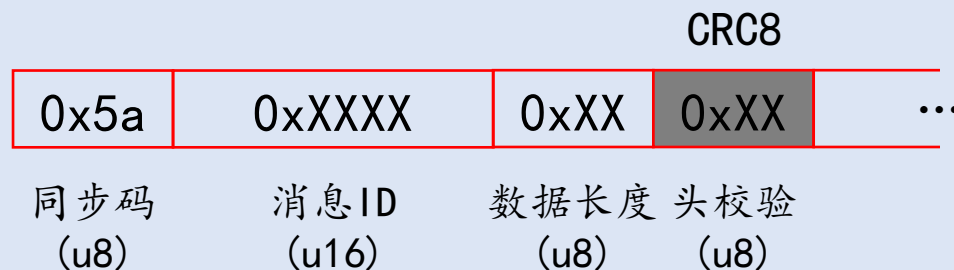
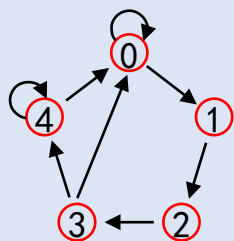
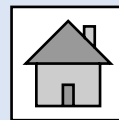
```
}
```

初始化串
口



初始化编
解码器

7.5.6.4. 数据包的接收原理



解码 // @简介: 接收消息

① ErrorStatus PAL_BinCodec_Receive(...);

// @简介: 获取消息ID

② uint16_t PAL_BinCodec_GetID(...);

// @简介: 获取数据长度

uint16_t PAL_BinCodec_GetDataLength(...);

// @简介: 获取数据crc校验

FlagStatus PAL_BinCodec_GetDataCRC(...);

// @简介: 读取各种类型的数据

uint8_t PAL_BinCodec_ReadByte(...); // 字节

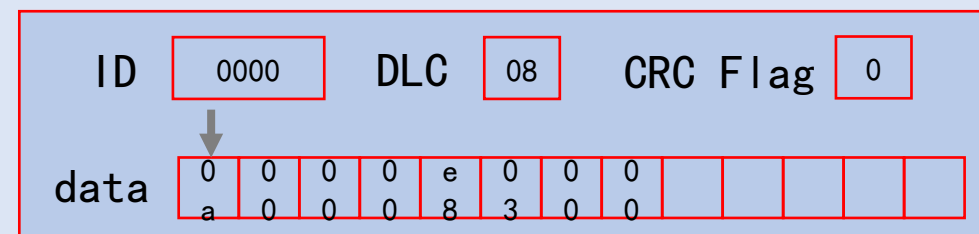
...

uint32_t PAL_BinCodec_ReadUInt32(...); // UInt32

float PAL_BinCodec_ReadFloat(...); // 单浮点数

double PAL_BinCodec_ReadDouble(...); // 双浮点数

const char *PAL_BinCodec_ReadString(...); // 字符串



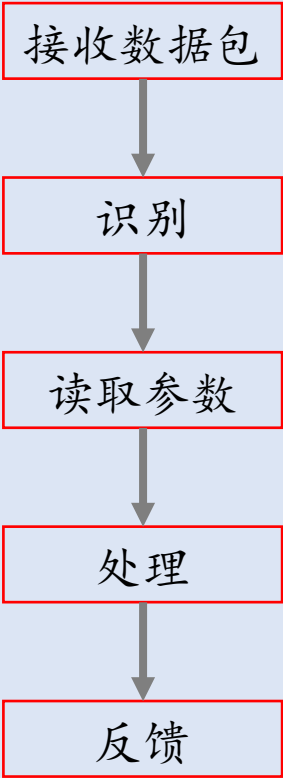
```
if (PAL_BinCodec_Receive(...) == SUCCESS) {
    switch (PAL_BinCodec_GetID(...)) {
        case 0: // 设置闪光灯参数
            onTime = PAL_BinCodec_ReadUInt32(...);
            offTime = PAL_BinCodec_ReadUInt32(...);
            ...
            break;
        case 1: ...
    }
}
```

7.5.6.5. 指令处理进程



ID	作用	参数	示例
0x0000	设置闪灯参数	[OnTime:u32][OffTime:u32]	5a 00 00 08 0a 00 00 00 e8 03 00 00
0x0001	ACK (Acknowledge)	[ErrorCode:u32]	5a 01 00 04 00 00 00 00
0x0002	读取闪灯参数	空	5a 02 00 00
0x0003	回复闪灯参数	[OnTime:u32][OffTime:u32]	5a 03 00 08 0a 00 00 00 e8 03 00 00

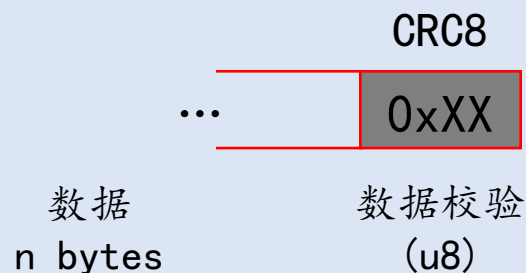
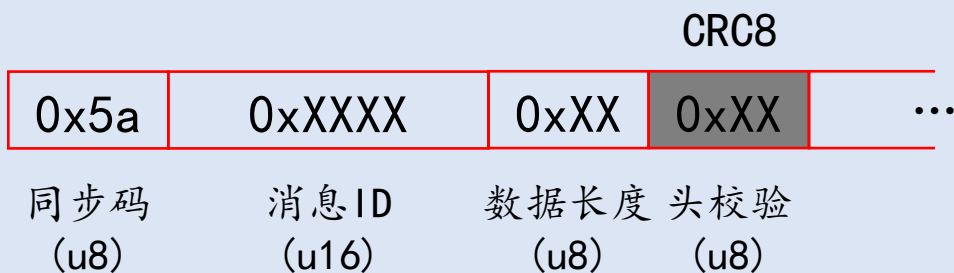
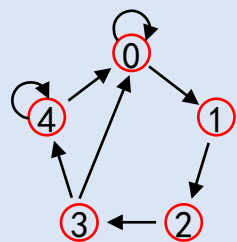
```
#include "app_blink_led.h"
#include "app_bin_cmd.h"
int main(void) {
    App_BlinkyLED_Init();
    App_BinCmd_Init();
    while(1) {
        App_BlinkyLED_Proc();
        App_BinBinCmd_Proc();
    }
}
```



```
void App_BinCmd_Proc(void)
{
    uint32_t onTime, offTime;

    if(PAL_BinCodec_Receive(...) == SUCCESS) {
        switch(PAL_BinCodec_GetID(...)) {
            case 0:
                onTime = PAL_BinCodec_ReadUInt32(...);
                offTime = PAL_BinCodec_ReadUInt32(...);
                ...
                break;
            case 2: ...
        }
    }
}
```

7.5.6.4. 数据包的接收原理



5a 02 00 00 //Get...
// Rsp: Get...
5a 03 00 08 0a 00 00
00 e8 03 00 00

编码

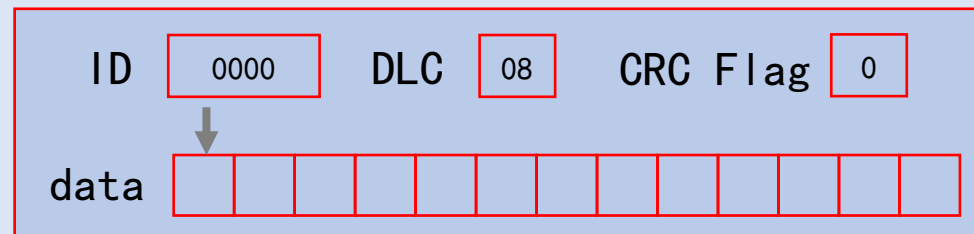
// @简介: 开始发送

① void PAL_BinCodec_StartSend(uint16_t ID);

② // @简介: 向参数中写入各种类型的数据
void PAL_BinCodec_WriteByte(..., uint8_t);
void PAL_BinCodec_WriteBytes(..., uint8_t[]);
...
void PAL_BinCodec_WriteUInt32(..., uint32_t);
void PAL_BinCodec_WriteFloat(..., float);
void PAL_BinCodec_WriteDouble(..., double);
void PAL_BinCodec_WriteString(..., const char *);

// @简介: 将二进制数据包发送出去

③ void PAL_BinCodec_EndSend(...);



```
PAL_BinCodec_StartSend(..., 03);
```

```
PAL_BinCodec_WriteUInt32(..., 10);    // 0n  
PAL_BinCodec_WriteUInt32(..., 1000); // 0ff
```

```
PAL_BinCodec_EndSend(...);
```

7.6. 随堂测试

题目1：简述PAL库USART发送/接收的数据流向